

PROYECTO EMPRESA ALIADA:

ENTREGA FINAL



Escuela Británica de Artes Creativas y Tecnología
Profesión Científico de Datos v2
Ing. Juan Carlos Kern Ochoátegui

PROYECTO EMPRESA ALIADA

¿Qué tal practicar?

Antes de Empezar

Esta actividad es la culminación de los proyectos previos que has completado en las diferentes etapas del análisis de datos. Asumirás el rol de un **Científico de Datos** en una compañía de productos de consumo, donde tu objetivo principal es proporcionar insights clave sobre el rendimiento de ventas de productos y realizar una predicción de ventas futuras para ayudar a la compañía a tomar decisiones estratégicas. Durante este proyecto, has trabajado con herramientas como **SQL, Python y PowerBI** para llevar a cabo un análisis exhaustivo de los datos. Ahora, deberás presentar los hallazgos obtenidos y las predicciones generadas en una presentación estructurada que muestre tanto tu capacidad técnica como la habilidad para comunicar resultados de manera efectiva.

PROYECTO EMPRESA ALIADA

Paso a Paso:

1. Introducción del Proyecto

En la introducción, debes proporcionar una descripción clara y concisa del proyecto que realizarás. Asegúrate de incluir los siguientes puntos:

- **Contexto del Proyecto:** Explica el propósito general del análisis.
- **Relevancia del Análisis:** Menciona por qué este análisis es importante para la estrategia de la empresa.
- **Herramientas Utilizadas:** Indica las herramientas y tecnologías que usarás durante el proyecto, como SQL, Python, PowerBI, etc.

2. Limpieza y Transformación de Datos con Python

- **Carga y Limpieza de Datos:** Los datos fueron cargados en **Pandas** desde archivos CSV y Excel. Se realizó una limpieza exhaustiva para manejar valores nulos, eliminar duplicados y corregir inconsistencias en los datos.
- **Transformaciones:** Se aplicaron transformaciones para normalizar los datos y estandarizar formatos de fechas y categorías.
- **Capturas y Código:** Incluir fragmentos de código que demuestran el proceso de limpieza y consolidación de los datos en **Python**.

PROYECTO EMPRESA ALIADA

3. Análisis Exploratorio de Datos (EDA) y Visualizaciones

- **Visualización de la Distribución de Ventas:** Se crearon histogramas y boxplots para visualizar la distribución de ventas por categoría de producto y región.
- **Análisis de Tendencias:** Gráficos de líneas permitieron observar tendencias de ventas en el tiempo, filtrando por categorías clave.
- **Identificación de Outliers:** Se usaron gráficos de dispersión y de caja para identificar posibles valores atípicos que puedan requerir un análisis más detallado.
- **Capturas de Visualizaciones:** Incluir ejemplos de las visualizaciones generadas con **Matplotlib** y **Seaborn** en Python.

PROYECTO EMPRESA ALIADA

4. Segmentación de Productos o Regiones con Clustering

- **Aplicación de Clustering con K-Means:** Se aplicó el algoritmo K-Means para segmentar los productos o regiones en función de características clave como ventas totales, categorías y regiones.
- **Número Óptimo de Clusters:** Se utilizó el método del codo para determinar el número óptimo de clusters, logrando identificar patrones de rendimiento.
- **Interpretación de los Clusters:** Se analizó cada cluster para identificar áreas de alto rendimiento y segmentos con oportunidades de mejora.
- **Visualizaciones de Clusters:** Se generaron gráficos de dispersión para visualizar los clusters y entender mejor las relaciones entre las diferentes variables.
- **Capturas de Pantalla y Código:** Incluir capturas de pantalla de los gráficos y el código utilizado para implementar el modelo de clustering en Python.

PROYECTO EMPRESA ALIADA

5. Análisis de Datos con SQL

- **Creación de la Base de Datos:** La base de datos fue diseñada e implementada en **SQL Server** utilizando las tablas de ventas proporcionadas (**DIM_CATEGORY, DIM_PRODUCT, DIM_SEGMENT, DIM_CALENDAR, FACT_SALES**).
- **Consultas Clave:** Se ejecutaron consultas básicas para verificar la correcta carga de datos y se realizaron uniones entre las tablas para obtener insights sobre las ventas por categoría y región.
- **Capturas de Pantalla y Código:** Incluir capturas de pantalla del proceso de creación de la base de datos y ejemplos de queries ejecutados en **SQL** para obtener insights clave.

6. Creación de un Dashboard en PowerBI

- **Estructura del Dashboard:** El tablero incluye secciones clave como resumen de ventas, desempeño por categoría, y análisis geográfico. Las visualizaciones fueron diseñadas para ser interactivas, permitiendo a los usuarios explorar los datos mediante filtros.
- **Visualizaciones Interactivas:** Se utilizaron gráficos de barras, líneas, mapas y tablas dinámicas para representar los insights obtenidos. Los usuarios pueden filtrar los resultados por región, segmento y fechas.
- **Capturas de Pantalla y Enlace:** Incluir capturas de pantalla del dashboard en Power BI, junto con un enlace al tablero publicado si es necesario.

PROYECTO EMPRESA ALIADA

7. Predicción de Ventas con Machine Learning

- **Selección del Modelo:** Se utilizó un modelo de **regresión lineal múltiple** y un modelo de **series de tiempo ARIMA** para predecir las ventas futuras. La selección del modelo se basó en los patrones de ventas observados en los datos históricos.
- **Validación del Modelo:** El modelo fue validado utilizando métricas como el **Error Absoluto Medio (MAE)** y el **Error Cuadrático Medio (MSE)**. Se realizaron ajustes para optimizar la precisión del modelo.
- **Predicciones Futuras:** Las predicciones de ventas para los próximos meses se presentaron en forma de gráficos de líneas comparativos entre los datos reales y las predicciones del modelo.
- **Capturas y Explicaciones:** Incluir gráficos comparativos y fragmentos de código que explican la implementación del modelo en **Python**.

PROYECTO EMPRESA ALIADA

8. Conclusiones y Recomendaciones

En las conclusiones, debes resumir tus hallazgos más importantes y proporcionar recomendaciones basadas en tu análisis. Aquí tienes lo que debes incluir:

- **Resumen de los Hallazgos Clave:** Ofrece un resumen de los principales insights obtenidos a lo largo del proyecto.
- **Impacto del Análisis en la Estrategia Empresarial:** Explica cómo los resultados pueden influir en las decisiones futuras de la compañía.
- **Recomendaciones:** Proporciona sugerencias específicas para mejorar la estrategia de la empresa.
- **Futuras Mejoras:** Indica posibles mejoras o pasos adicionales que podrían tomarse en el futuro.

PROYECTO EMPRESA ALIADA

Archivo Final

- **Formato de Entrega:** La presentación debe entregarse en un archivo **PowerPoint (.pptx)** o mediante un enlace a **Google Slides**. El archivo debe incluir todas las capturas de pantalla, gráficos y visualizaciones mencionadas.

¿Cómo presentar su entrega?

Un archivo Jupyter Notebook (.ipynb) que contenga el código, los gráficos y las explicaciones de cada paso del proceso de construcción y validación del modelo.

Tiempo estimado de resolución: 150 minutos

1 – INTRODUCCIÓN DEL PROYECTO

Este proyecto forma parte del entregable final de Empresa Aliada de la Profesión Científico de Datos de EBAC.

A través de los diferentes entregables se aplicaron progresivamente las competencias desarrolladas durante el curso: preparación y análisis de datos, estadística, visualización, bases de datos, Machine Learning y modelado predictivo.

Esta presentación integra y resume los principales procesos, análisis, resultados y aprendizajes obtenidos a lo largo del proyecto

1.1 CONTEXTO DEL PROYECTO

- Productos y Detergentes de México (nombre ficticio para efectos del proyecto) requiere analizar su información histórica de ventas, integrando datos de productos, categorías, segmentos, calendario y transacciones.
- El proyecto busca transformar estos datos en información útil para comprender el desempeño comercial, identificar patrones y tendencias, desarrollar modelos que permitan proyectar el comportamiento futuro de las ventas, y tomar decisiones estratégicas para la empresa.
- Fuentes de información:
 - DIM_CATEGORY · DIM_PRODUCT · DIM_SEGMENT · DIM_CALENDAR · FACT_SALES

1.2 – RELEVANCIA DEL ANÁLISIS

- El análisis de datos permite convertir los registros históricos de ventas en información para la toma de decisiones basada en datos.
- Su aplicación permite:
 - Comprender el comportamiento histórico de las ventas.
 - Identificar tendencias, patrones y variables relevantes.
 - Analizar productos, categorías y segmentos.
 - Anticipar la demanda y proyectar ventas futuras.
 - Generar información que apoye la planeación comercial y estratégica.

1.3 – HERRAMIENTAS UTILIZADAS



- Durante los distintos entregables se utilizaron herramientas complementarias para cubrir el ciclo de análisis de datos:
 - Datos: Excel y CSV
 - Programación: Python y Jupyter Notebook
 - Manipulación: Pandas y NumPy
 - Visualización: Matplotlib, Seaborn y Power BI
 - Bases de datos: SQL Server y SSMS
 - Modelado: Estadística, Machine Learning y Series de Tiempo
- Estas herramientas permitieron avanzar desde la limpieza e integración de los datos hasta su análisis, visualización y modelado predictivo.



2 – LIMPIEZA Y TRANSFORMACIÓN DE DATOS CON PYTHON



2.1 – CARGA Y LIMPIEZA DE DATOS

- Se cargaron en Pandas las cinco fuentes de información: calendario, categorías, productos, segmentos y ventas. Se revisaron dimensiones, columnas, tipos de datos, valores nulos, duplicados e integridad de las llaves.
- Se detectaron 2 productos sin ItemCode y se comprobó que no existían códigos de producto duplicados reales. Para conservar la mayor cantidad de información posible, únicamente se excluyeron los registros sin ItemCode antes de integrar las tablas.

2.1 – CARGA Y LIMPIEZA DE DATOS

- Evidencia los archivos CSV/Excel utilizados y el entorno de trabajo.
- Se muestran las dimensiones y columnas de las cinco tablas cargadas.

```
import pandas as pd
import numpy as np
import os
```

```
os.listdir()
```

```
['.ipynb_checkpoints',
'DIM_CALENDAR (2).xlsx',
'DIM_CATEGORY (2).csv',
'DIM_PRODUCT (1).xlsx',
'DIM_SEGMENT (1).xlsx',
'FACT_SALES (1).csv',
'Proyecto Empresa Aliada Entregable 6-CD-Juan Carlos Kern.docx',
'Untitled.ipynb']
```

```
# 3. Cargar los cinco archivos que conforman la base de datos
```

```
dim_calendar = pd.read_excel('DIM_CALENDAR (2).xlsx')
dim_category = pd.read_csv('DIM_CATEGORY (2).csv')
dim_product = pd.read_excel('DIM_PRODUCT (1).xlsx')
dim_segment = pd.read_excel('DIM_SEGMENT (1).xlsx')
fact_sales = pd.read_csv('FACT_SALES (1).csv')
```

```
# 4. Revisar el número de filas, columnas y los nombres de las variables de cada archivo
```

```
datasets = {
    'DIM_CALENDAR': dim_calendar,
    'DIM_CATEGORY': dim_category,
    'DIM_PRODUCT': dim_product,
    'DIM_SEGMENT': dim_segment,
    'FACT_SALES': fact_sales
}

for nombre, df in datasets.items():
    print(f'\n{nombre}')
    print(f'Dimensiones: {df.shape}')
    print('Columnas:')
    print(df.columns.tolist())
```

```
DIM_CALENDAR
Dimensiones: (156, 5)
Columnas:
['WEEK', 'YEAR', 'MONTH', 'WEEK_NUMBER', 'DATE']
```

```
DIM_CATEGORY
Dimensiones: (5, 2)
Columnas:
['ID_CATEGORY', 'CATEGORY']
```

```
DIM_PRODUCT
Dimensiones: (505, 9)
Columnas:
['MANUFACTURER', 'BRAND', 'ITEM', 'ITEM_DESCRIPTION', 'CATEGORY', 'FORMAT', 'ATTR1', 'ATTR2', 'ATTR3']
```

```
DIM_SEGMENT
Dimensiones: (53, 6)
Columnas:
['CATEGORY', 'ATTR1', 'ATTR2', 'ATTR3', 'FORMAT', 'SEGMENT']
```

```
FACT_SALES
Dimensiones: (122002, 6)
Columnas:
['WEEK', 'ITEM_CODE', 'TOTAL_UNIT_SALES', 'TOTAL_VALUE_SALES', 'TOTAL_UNIT_AVG_WEEKLY_SALES', 'REGION']
```

2.1 – CARGA Y LIMPIEZA DE DATOS

```
# 7. Revisar Los tipos de datos y la cantidad de valores nulos
```

```
for nombre, df in datasets.items():
    print(f'\n{"="*50}')
    print(nombre)
    print(f'\n{"="*50}')

    print('\nTipos de datos:')
    print(df.dtypes)

    print('\nValores nulos:')
    print(df.isnull().sum())
```

```
=====
DIM_CALENDAR
=====
```

```
Tipos de datos:
WeekCode      object
Year          int64
Month         int64
WeekNumber    int64
Date          datetime64[ns]
dtype: object
```

```
Valores nulos:
WeekCode    0
Year        0
Month       0
WeekNumber  0
Date        0
dtype: int64
```

```
=====
DIM_CATEGORY
=====
```

```
Tipos de datos:
CategoryID    int64
CategoryName  object
dtype: object
```

```
Valores nulos:
CategoryID    0
CategoryName  0
dtype: int64
```

```
=====
DIM_PRODUCT
=====
```

```
Tipos de datos:
Manufacturer  object
Brand         object
ItemCode      object
ItemDescription object
CategoryID    int64
Format        object
Attr1         object
Attr2         object
Attr3         object
dtype: object
```

```
Valores nulos:
Manufacturer  0
Brand         0
ItemCode      2
ItemDescription 0
CategoryID    0
Format        0
Attr1         6
Attr2         0
Attr3         6
dtype: int64
```

```
=====
DIM_SEGMENT
=====
```

```
Tipos de datos:
CategoryID    int64
Attr1         object
Attr2         object
Attr3         object
Format        object
Segment       object
dtype: object
```

```
Valores nulos:
CategoryID    0
Attr1         0
Attr2         0
Attr3         1
Format        0
Segment       0
dtype: int64
```

- Se muestran la revisión de tipos de datos y valores nulos en todas las tablas.

```
=====
FACT_SALES
=====
```

```
Tipos de datos:
WeekCode      object
ItemCode      object
TotalUnitSales float64
TotalValueSales float64
TotalUnitAvgWeeklySales float64
Region        object
dtype: object
```

```
Valores nulos:
WeekCode      0
ItemCode      0
TotalUnitSales 0
TotalValueSales 0
TotalUnitAvgWeeklySales 0
Region        0
dtype: int64
```

2.1 – CARGA Y LIMPIEZA DE DATOS

```
# 8. Revisar si las llaves de las tablas dimensionales contienen duplicados

print('DIM_CALENDAR')
print('Filas:', len(dim_calendar))
print('WeekCode únicos:', dim_calendar['WeekCode'].nunique())
print('WeekCode duplicados:', dim_calendar['WeekCode'].duplicated().sum())

print('\nDIM_PRODUCT')
print('Filas:', len(dim_product))
print('ItemCode únicos:', dim_product['ItemCode'].nunique())
print('ItemCode duplicados:', dim_product['ItemCode'].duplicated().sum())

print('\nDIM_CATEGORY')
print('Filas:', len(dim_category))
print('CategoryID únicos:', dim_category['CategoryID'].nunique())
print('CategoryID duplicados:', dim_category['CategoryID'].duplicated().sum())

print('\nDIM_SEGMENT')

segment_keys = ['CategoryID', 'Attr1', 'Attr2', 'Attr3', 'Format']

print('Filas:', len(dim_segment))
print(
    'Combinaciones únicas:',
    dim_segment[segment_keys].drop_duplicates().shape[0]
)
print(
    'Combinaciones duplicadas:',
    dim_segment.duplicated(subset=segment_keys).sum()
)
```

```
DIM_CALENDAR
Filas: 156
WeekCode únicos: 156
WeekCode duplicados: 0
```

```
DIM_PRODUCT
Filas: 505
ItemCode únicos: 503
ItemCode duplicados: 1
```

```
DIM_CATEGORY
Filas: 5
CategoryID únicos: 5
CategoryID duplicados: 0
```

```
DIM_SEGMENT
Filas: 53
Combinaciones únicas: 53
Combinaciones duplicadas: 0
```

- Se demuestra la revisión de duplicados en las llaves.
- Se muestra visualmente los registros problemáticos de DIM_PRODUCT.

```
# 10. Revisar los registros con valores nulos en DIM_PRODUCT

productos_con_nulos = dim_product[
    dim_product[['ItemCode', 'Attr1', 'Attr3']].isnull().any(axis=1)
]

productos_con_nulos
```

	Manufacturer	Brand	ItemCode	ItemDescription	CategoryID	Format	Attr1	Attr2	Attr3
33	WHITE MAGIC	WHITE MAGIC	0079893431208	9WHITE MAGIC BOT.PLAS 3970ML IMP 0079893431208	1	LIQUIDO	NaN	CLORO	NaN
107	INDS. ALEN	CLORALEX	7501025401023	CLORALEX EXTRA SENSACION FRUTALES BOT PLAST 95...	1	LIQUIDO	NaN	CLORO	NaN
143	INDS. ALEN	BLANCATEL	7501025402259	BLANCATEL BLANQUEADOR BOTELLA 1.8LT 7501025402259	1	LIQUIDO	NaN	CLORO	NaN
343	CLOROX	CLOROX	7501071901591	CLOROX BLANQUEADOR PARAISO TROPICAL 3800 ML NA...	1	LIQUIDO	NaN	CLORO	NaN
469	OTHERS FABRICANTE UNIF.	OTHERS MARCA UNIF.	7502217411776	KLEEN CLORO BOTELLA 1000 ML NAL 7502217411776	1	LIQUIDO	NaN	CLORO	NaN
495	IBERIA	IBERIA	8411660040503	9IBERIA BLANQUEADOR POLVO 250 GR IMP 841166004...	1	POLVO	NaN	CLORO	NaN
502	INDS. ALEN	CLORALEX	NaN	CLORALEX EL RENDIDOR BOT PLAST 2LT	1	LIQUIDO	CLORO	CLORO	NO DEFINIDO
503	CLOROX	CLOROX	NaN	CLOROX MASCOTAS BLANQUEADOR+DETERGENTE GALON 10L	1	LIQUIDO	CLORO	CLORO	MASCOTAS

2.1 – CARGA Y LIMPIEZA DE DATOS

- Se confirma que los supuestos duplicados de ItemCode eran consecuencia de los valores nulos y que no existían duplicados reales

```
# 11. Verificar si existen ItemCode duplicados reales, excluyendo valores nulos
```

```
itemcode_duplicados_reales = dim_product[  
    dim_product['ItemCode'].notna() &  
    dim_product['ItemCode'].duplicated(keep=False)  
]  
  
print('ItemCode duplicados reales:', len(itemcode_duplicados_reales))
```

```
itemcode_duplicados_reales
```

```
ItemCode duplicados reales: 0
```

Manufacturer	Brand	ItemCode	ItemDescription	CategoryID	Format	Attr1	Attr2	Attr3
--------------	-------	----------	-----------------	------------	--------	-------	-------	-------

2.2 - TRANSFORMACIONES

- Se estandarizaron los nombres de las variables y se prepararon las tablas para su integración. Posteriormente se consolidaron las dimensiones con FACT_SALES mediante llaves de producto, calendario, categoría y segmento, utilizando relaciones many-to-one para controlar la integridad de las uniones.
- El resultado conservó las 122,002 observaciones originales, generando un dataset consolidado de 20 variables y sin valores nulos.

2.2 - TRANSFORMACIONES

5. Ajustar y estandarizar los nombres de las columnas

```
dim_calendar = dim_calendar.rename(columns={
    'WEEK': 'WeekCode',
    'YEAR': 'Year',
    'MONTH': 'Month',
    'WEEK_NUMBER': 'WeekNumber',
    'DATE': 'Date'
})

dim_category = dim_category.rename(columns={
    'ID_CATEGORY': 'CategoryID',
    'CATEGORY': 'CategoryName'
})

dim_product = dim_product.rename(columns={
    'MANUFACTURER': 'Manufacturer',
    'BRAND': 'Brand',
    'ITEM': 'ItemCode',
    'ITEM_DESCRIPTION': 'ItemDescription',
    'CATEGORY': 'CategoryID',
    'FORMAT': 'Format',
    'ATTR1': 'Attr1',
    'ATTR2': 'Attr2',
    'ATTR3': 'Attr3'
})

dim_segment = dim_segment.rename(columns={
    'CATEGORY': 'CategoryID',
    'ATTR1': 'Attr1',
    'ATTR2': 'Attr2',
    'ATTR3': 'Attr3',
    'FORMAT': 'Format',
    'SEGMENT': 'Segment'
})

fact_sales = fact_sales.rename(columns={
    'WEEK': 'WeekCode',
    'ITEM_CODE': 'ItemCode',
    'TOTAL_UNIT_SALES': 'TotalUnitSales',
    'TOTAL_VALUE_SALES': 'TotalValueSales',
    'TOTAL_UNIT_AVG_WEEKLY_SALES': 'TotalUnitAvgWeeklySales',
    'REGION': 'Region'
})
```

- Se muestra claramente la estandarización de nombres de columnas.

6. Verificar los nuevos nombres de las columnas

```
datasets = {
    'DIM_CALENDAR': dim_calendar,
    'DIM_CATEGORY': dim_category,
    'DIM_PRODUCT': dim_product,
    'DIM_SEGMENT': dim_segment,
    'FACT_SALES': fact_sales
}

for nombre, df in datasets.items():
    print(f'\n{nombre}')
    print(df.columns.tolist())

DIM_CALENDAR
['WeekCode', 'Year', 'Month', 'WeekNumber', 'Date']

DIM_CATEGORY
['CategoryID', 'CategoryName']

DIM_PRODUCT
['Manufacturer', 'Brand', 'ItemCode', 'ItemDescription', 'CategoryID', 'Format', 'Attr1', 'Attr2', 'Attr3']

DIM_SEGMENT
['CategoryID', 'Attr1', 'Attr2', 'Attr3', 'Format', 'Segment']

FACT_SALES
['WeekCode', 'ItemCode', 'TotalUnitSales', 'TotalValueSales', 'TotalUnitAvgWeeklySales', 'Region']
```

2.2 - TRANSFORMACIONES

- Valida que ItemCode y WeekCode de ventas tuvieran correspondencia con producto y calendario.
- Valida la correspondencia con categorías y segmentos.

9. Verificar que Las Llaves de FACT_SALES tengan correspondencia en Las dimensiones

```
items_sin_producto = fact_sales.loc[
    ~fact_sales['ItemCode'].isin(dim_product['ItemCode']),
    'ItemCode'
].nunique()

semanas_sin_calendario = fact_sales.loc[
    ~fact_sales['WeekCode'].isin(dim_calendar['WeekCode']),
    'WeekCode'
].nunique()

print('ItemCode de FACT_SALES sin correspondencia en DIM_PRODUCT:',
      items_sin_producto)

print('WeekCode de FACT_SALES sin correspondencia en DIM_CALENDAR:',
      semanas_sin_calendario)
```

```
ItemCode de FACT_SALES sin correspondencia en DIM_PRODUCT: 0
WeekCode de FACT_SALES sin correspondencia en DIM_CALENDAR: 0
```

12. Verificar La correspondencia de Los productos con categorías y segmentos

```
categorias_sin_correspondencia = dim_product.loc[
    ~dim_product['CategoryID'].isin(dim_category['CategoryID']),
    'CategoryID'
].nunique()

segment_check = dim_product.merge(
    dim_segment,
    on=['CategoryID', 'Attr1', 'Attr2', 'Attr3', 'Format'],
    how='left',
    indicator=True
)

productos_sin_segmento = segment_check[
    segment_check['_merge'] == 'left_only'
]

print(
    'CategoryID de DIM_PRODUCT sin correspondencia en DIM_CATEGORY:',
    categorias_sin_correspondencia
)

print(
    'Productos sin correspondencia en DIM_SEGMENT:',
    len(productos_sin_segmento)
)
```

```
CategoryID de DIM_PRODUCT sin correspondencia en DIM_CATEGORY: 0
Productos sin correspondencia en DIM_SEGMENT: 6
```

2.2 - TRANSFORMACIONES

- Muestra la preparación de DIM_PRODUCT, eliminando únicamente registros sin ItemCode.
- Se muestra la consolidación mediante los cuatro merge.

```
# 13. Preparar DIM_PRODUCT eliminando únicamente registros sin ItemCode
```

```
dim_product_merge = dim_product.dropna(
    subset=['ItemCode']
).copy()

print('Filas originales de DIM_PRODUCT:', len(dim_product))
print('Filas disponibles para el merge:', len(dim_product_merge))
print(
    'ItemCode duplicados:',
    dim_product_merge['ItemCode'].duplicated().sum()
)
```

```
Filas originales de DIM_PRODUCT: 505
Filas disponibles para el merge: 503
ItemCode duplicados: 0
```

```
# 14. Consolidar FACT_SALES con Las tablas dimensionales
```

```
df_consolidado = (
    fact_sales
    .merge(
        dim_product_merge,
        on='ItemCode',
        how='left',
        validate='many_to_one'
    )
    .merge(
        dim_calendar,
        on='WeekCode',
        how='left',
        validate='many_to_one'
    )
    .merge(
        dim_category,
        on='CategoryID',
        how='left',
        validate='many_to_one'
    )
    .merge(
        dim_segment,
        on=['CategoryID', 'Attr1', 'Attr2', 'Attr3', 'Format'],
        how='left',
        validate='many_to_one'
    )
)
```

```
# 15. Verificar que no se hayan perdido ni multiplicado registros
```

```
print('Filas originales de FACT_SALES:', len(fact_sales))
print('Filas del dataset consolidado:', len(df_consolidado))
print('Columnas del dataset consolidado:', df_consolidado.shape[1])

print('\nDimensiones del dataset consolidado:')
print(df_consolidado.shape)
```

```
Filas originales de FACT_SALES: 122002
Filas del dataset consolidado: 122002
Columnas del dataset consolidado: 20
```

```
Dimensiones del dataset consolidado:
(122002, 20)
```

2.2 - TRANSFORMACIONES

- Comprueba que el merge no perdió ni multiplicó registros
- Muestra el resultado final y verifica que el dataset consolidado quedó sin valores nulos.

```
# 16. Visualizar las primeras filas del dataset consolidado
df_consolidado.head()
```

	WeekCode	ItemCode	TotalUnitSales	TotalValueSales	TotalUnitAvgWeeklySales	Region	Manufacturer	Brand
0	34-22	7501058792808BP2	0.006	0.139	1.000	TOTAL AUTOS AREA 5	RECKITT	VANISH VANISHOXIACTIONROS
1	34-22	7501058715883	0.487	116.519	2.916	TOTAL AUTOS AREA 5	RECKITT	VANISH VANISH OXI ACTION
2	34-22	7702626213774	1.391	68.453	5.171	TOTAL AUTOS AREA 5	RECKITT	VANISH VANISH OXI ACTION
3	34-22	7501058716422	0.022	1.481	1.833	TOTAL AUTOS AREA 5	RECKITT	VANISH VANISH OXI ACTION
4	34-22	7501058784353	2.037	182.839	5.375	TOTAL AUTOS AREA 5	RECKITT	VANISH VANISH INTELLI

```
# 17. Revisar la cantidad de valores nulos en el dataset consolidado
df_consolidado.isnull().sum()

WeekCode      0
ItemCode      0
TotalUnitSales 0
TotalValueSales 0
TotalUnitAvgWeeklySales 0
Region        0
Manufacturer   0
Brand         0
ItemDescription 0
CategoryID    0
Format        0
Attr1         0
Attr2         0
Attr3         0
Year          0
Month         0
WeekNumber    0
Date          0
CategoryName  0
Segment       0
dtype: int64
```

3 – ANÁLISIS EXPLORATORIO DE DATOS (EDA) Y VISUALIZACIONES



3.1 – VISUALIZACIÓN DE LA DISTRIBUCIÓN DE VENTAS

- Se analizó la distribución de las ventas tanto en valor como en unidades mediante histogramas y boxplots con Matplotlib y Seaborn.
- Los histogramas muestran una distribución fuertemente asimétrica hacia la derecha: la mayoría de las observaciones se concentra en valores bajos y existe un número reducido de ventas considerablemente mayores.
- Los boxplots permitieron comparar la distribución entre categorías y regiones, identificando diferencias importantes en dispersión y magnitud de las ventas. La región TOTAL AUTOS SCANNING MEXICO presenta una mayor amplitud y niveles de venta superiores al resto.

3.1 – VISUALIZACIÓN DE LA DISTRIBUCIÓN DE VENTAS

- Código de histogramas y boxplots utilizando Matplotlib y Seaborn

```
[59]: # Paso 3: Visualizar la distribución de ventas

# Importamos las librerías necesarias para los gráficos
import matplotlib.pyplot as plt
import seaborn as sns

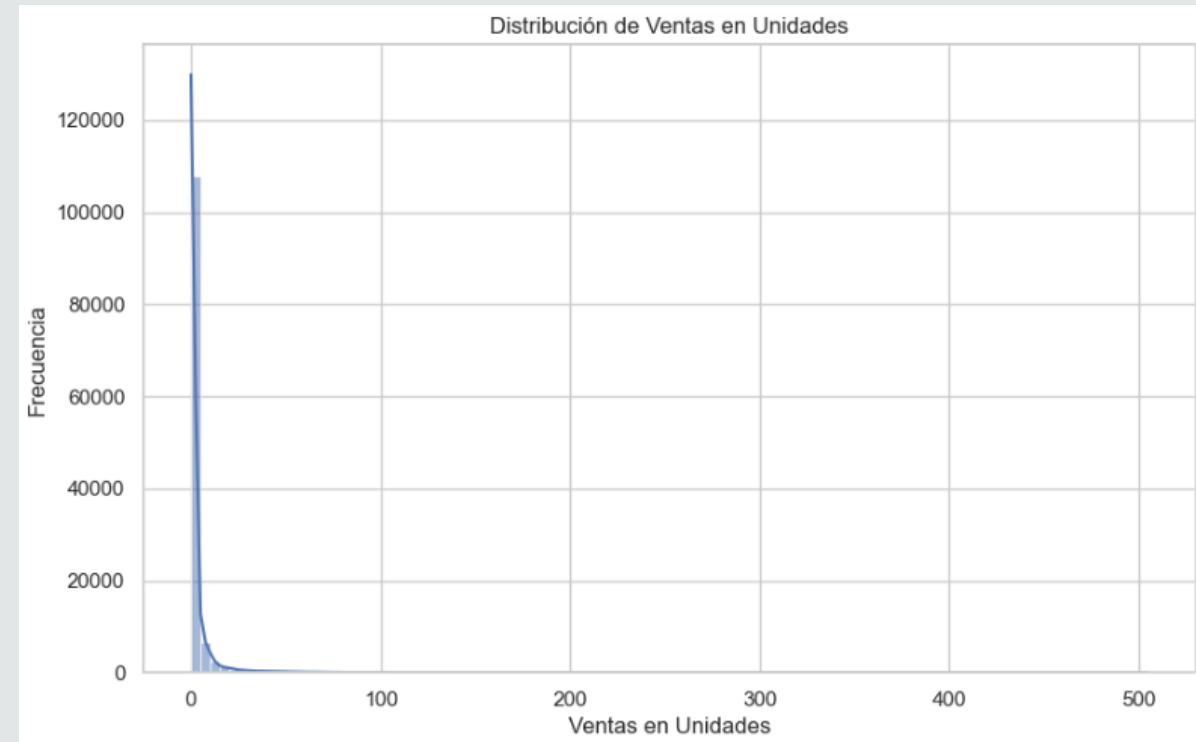
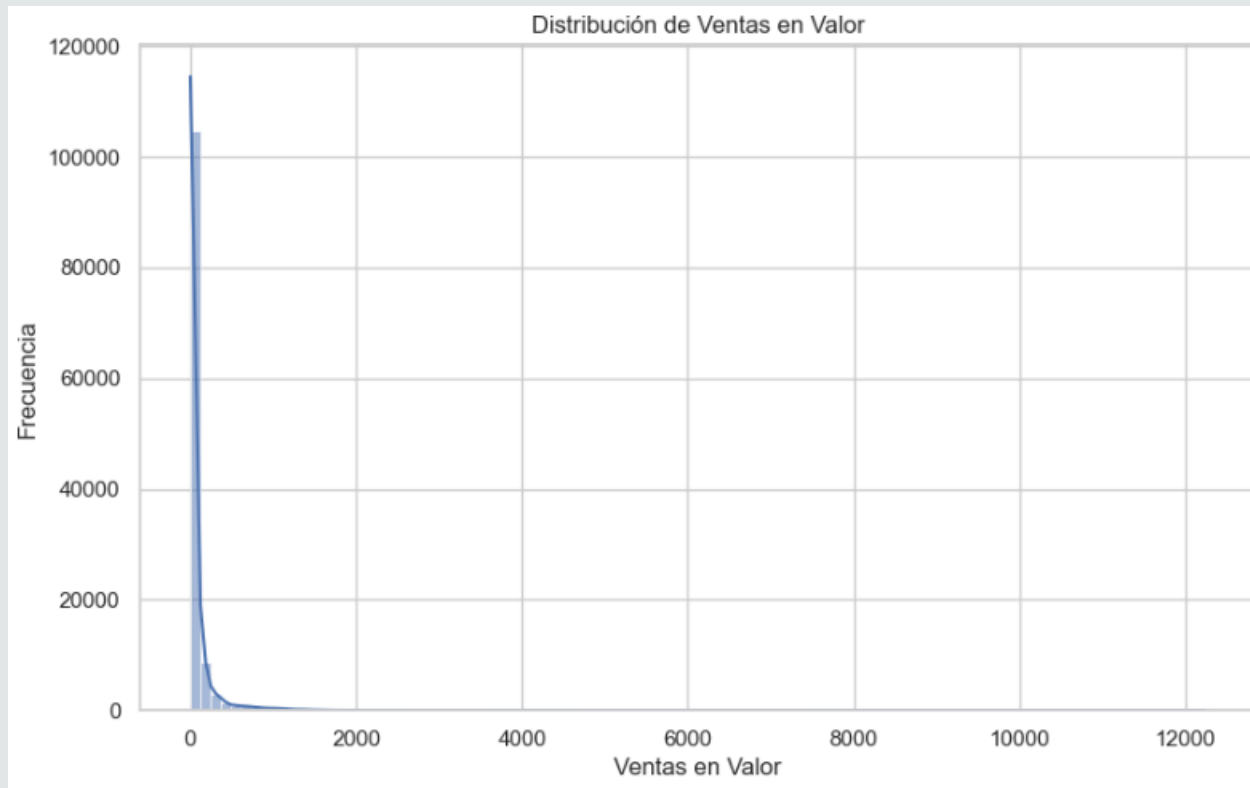
# Histograma de la distribución de ventas en valor
plt.figure(figsize=(10,6))
sns.histplot(df_consolidado['total_value_sales'], bins=100, kde=True)
plt.title('Distribución de Ventas en Valor')
plt.xlabel('Ventas en Valor')
plt.ylabel('Frecuencia')
plt.savefig('3.1-Distribución de Ventas en Valor.png', dpi=300, bbox_inches='tight')
plt.show()

# Histograma de la distribución de ventas en unidades
plt.figure(figsize=(10,6))
sns.histplot(df_consolidado['total_unit_sales'], bins=100, kde=True)
plt.title('Distribución de Ventas en Unidades')
plt.xlabel('Ventas en Unidades')
plt.ylabel('Frecuencia')
plt.savefig('3.2-Distribución de Ventas en Unidades.png', dpi=300, bbox_inches='tight')
plt.show()

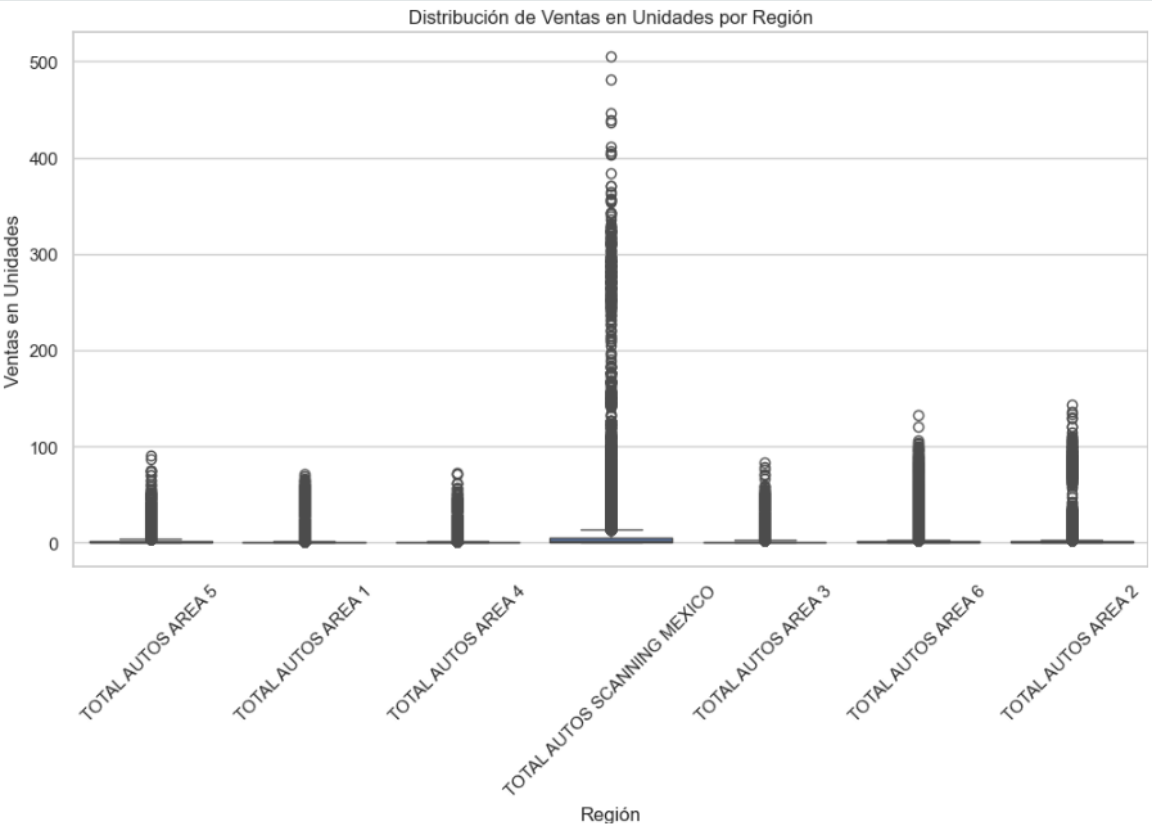
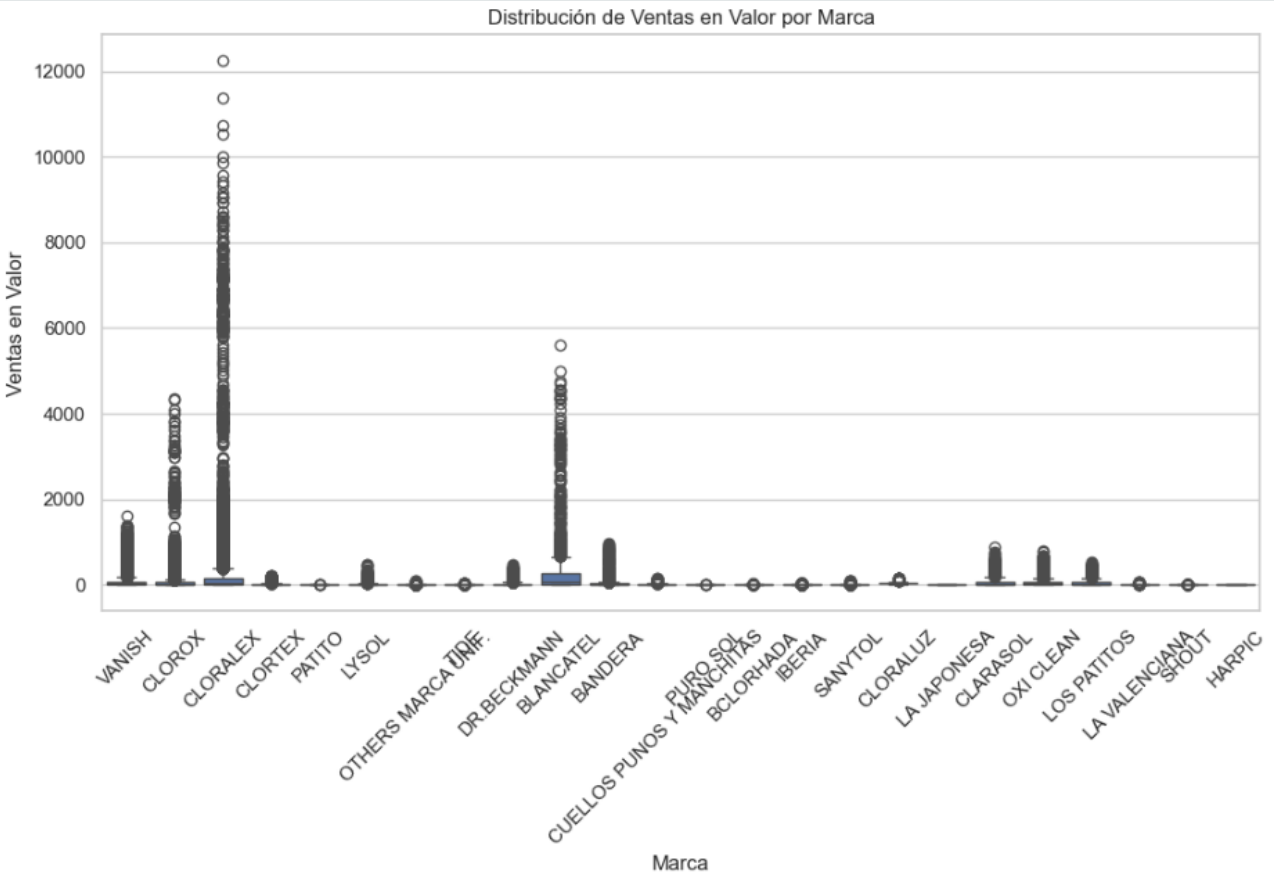
# Boxplot de ventas en valor segmentado por marca
plt.figure(figsize=(12,6))
sns.boxplot(x='brand', y='total_value_sales', data=df_consolidado)
plt.title('Distribución de Ventas en Valor por Marca')
plt.xlabel('Marca')
plt.ylabel('Ventas en Valor')
plt.xticks(rotation=45)
plt.savefig('3.3-Distribución de Ventas en Valor por Marca.png', dpi=300, bbox_inches='tight')
plt.show()

# Boxplot de ventas en unidades segmentado por región
plt.figure(figsize=(12,6))
sns.boxplot(x='region', y='total_unit_sales', data=df_consolidado)
plt.title('Distribución de Ventas en Unidades por Región')
plt.xlabel('Región')
plt.ylabel('Ventas en Unidades')
plt.xticks(rotation=45)
plt.savefig('3.4-Distribución de Ventas en Unidades por Región.png', dpi=300, bbox_inches='tight')
plt.show()
```

3.1 – VISUALIZACIÓN DE LA DISTRIBUCIÓN DE VENTAS



3.1 – VISUALIZACIÓN DE LA DISTRIBUCIÓN DE VENTAS



3.1 – VISUALIZACIÓN DE LA DISTRIBUCIÓN DE VENTAS

```
[75]: # Punto 7: Identificar posibles outliers con gráficos de caja (Boxplots)

import matplotlib.pyplot as plt
import seaborn as sns

# Estilo visual de seaborn
sns.set(style="whitegrid")

# Boxplot por Categoría
plt.figure(figsize=(14, 8))
sns.boxplot(
    data=df_consolidado,
    x='category_id',
    y='total_value_sales',
    palette='Set2'
)
plt.title('Distribución de Ventas en Valor por Categoría')
plt.xlabel('ID de Categoría')
plt.ylabel('Ventas en Valor')
plt.xticks(rotation=45)
plt.savefig('7.1-Distribución de Ventas en Valor por Categoría.png', dpi=300, bbox_inches='tight')
plt.show()

# Boxplot por Región
plt.figure(figsize=(14, 8))
sns.boxplot(
    data=df_consolidado,
    x='region',
    y='total_value_sales',
    palette='Set3'
)
plt.title('Distribución de Ventas en Valor por Región')
plt.xlabel('Región')
plt.ylabel('Ventas en Valor')
plt.xticks(rotation=45)
plt.savefig('7.2-Distribución de Ventas en Valor por Región.png', dpi=300, bbox_inches='tight')
plt.show()

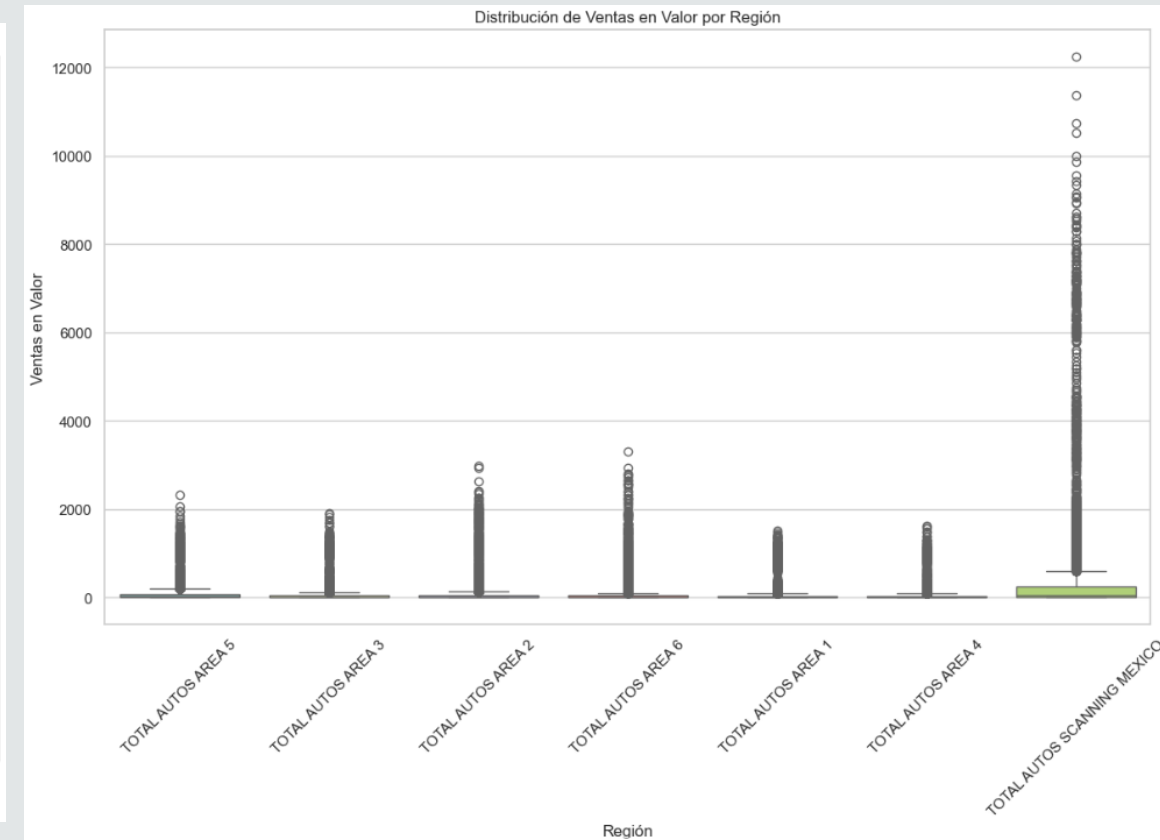
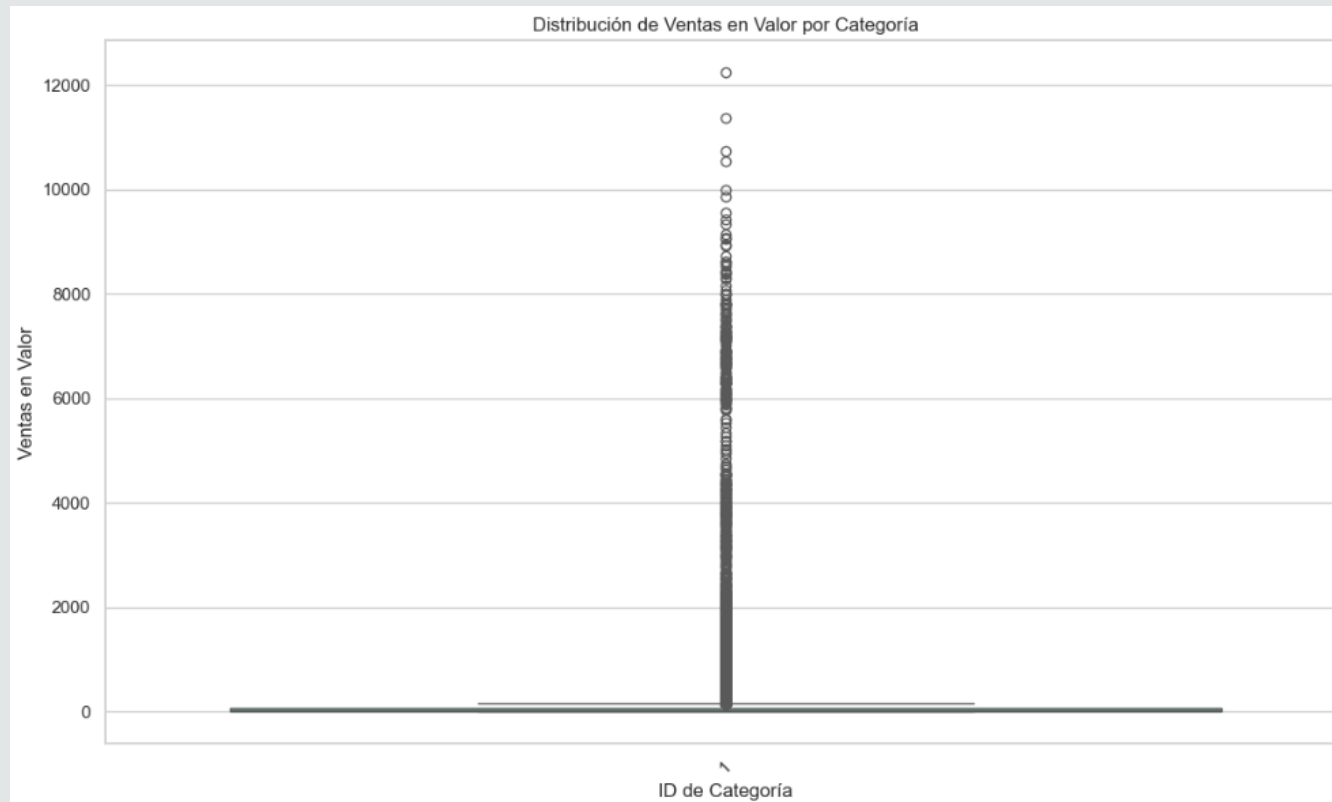
# Boxplot por Producto (para evitar saturación, mostraremos los Top 10 productos más vendidos)
# Identificamos los productos con mayores ventas en total
top_productos = df_consolidado.groupby('item_description')['total_value_sales'].sum().nlargest(10).index.tolist()

# Filtramos el dataframe solo para estos productos
df_top_productos = df_consolidado[df_consolidado['item_description'].isin(top_productos)]

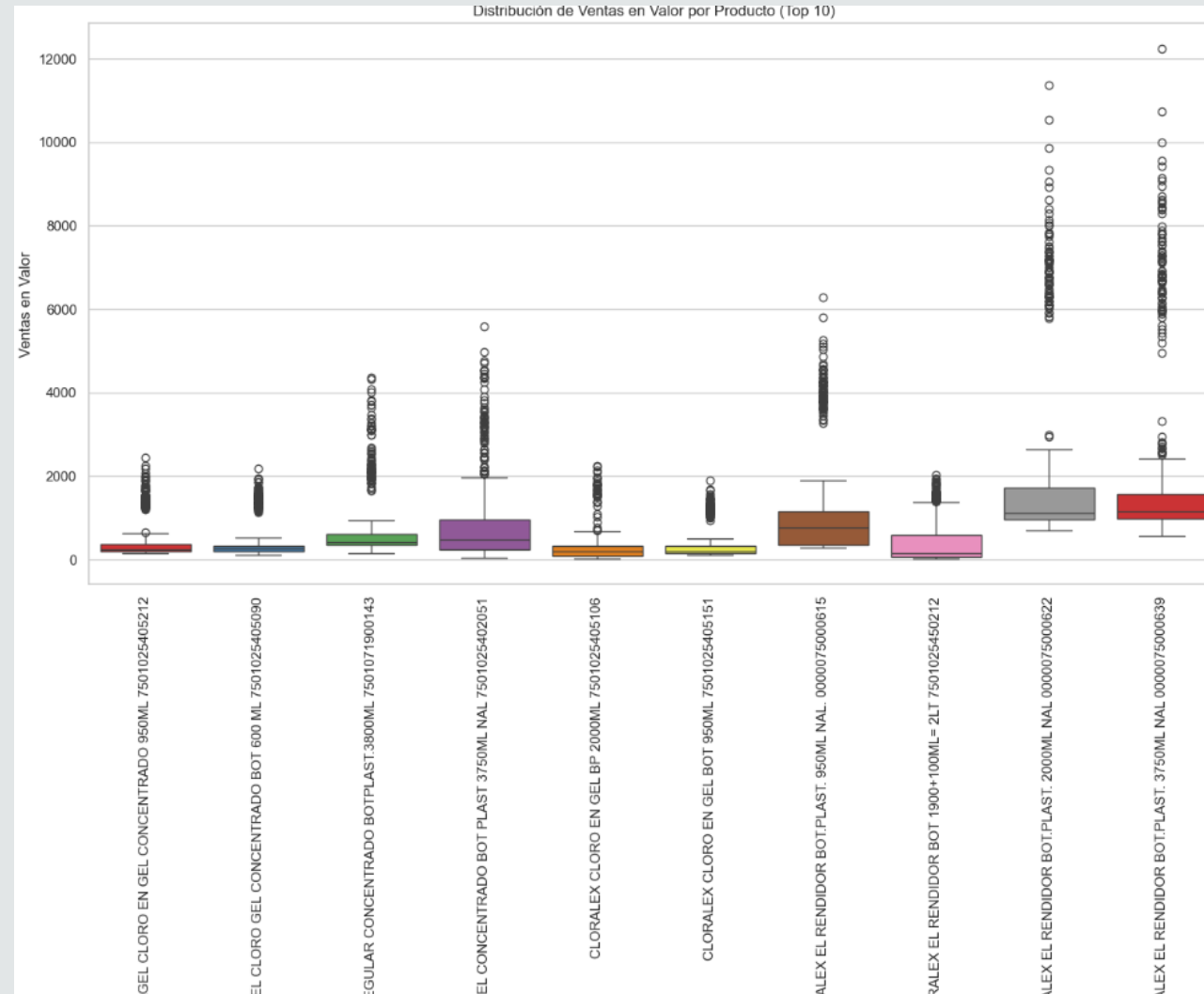
plt.figure(figsize=(16, 8))
sns.boxplot(
    data=df_top_productos,
    x='item_description',
    y='total_value_sales',
    palette='Set1'
)
plt.title('Distribución de Ventas en Valor por Producto (Top 10)')
plt.xlabel('Producto')
plt.ylabel('Ventas en Valor')
plt.xticks(rotation=90)
plt.savefig('7.3-Distribución de Ventas en Valor por Producto .png', dpi=300, bbox_inches='tight')
plt.show()
```

- Código específico de boxplots por categoría y región

3.1 – VISUALIZACIÓN DE LA DISTRIBUCIÓN DE VENTAS



3.1 – VISUALIZACIÓN DE LA DISTRIBUCIÓN DE VENTAS



3.2 – ANÁLISIS DE TENDENCIAS

- Las ventas fueron agregadas por fecha y analizadas mediante gráficos de líneas, permitiendo observar su evolución entre enero de 2022 y julio de 2023.
- La serie presenta fluctuaciones semanales y picos relevantes, pero también un incremento hacia los últimos meses del periodo analizado.
- El análisis se desagregó por región y principales marcas. TOTAL AUTOS SCANNING MEXICO concentra los mayores niveles de venta, mientras que Vanish destaca claramente entre las principales marcas.
- Adicionalmente, se calculó un promedio móvil de 4 semanas para suavizar las variaciones semanales y facilitar la identificación de la tendencia subyacente.

3.2 – ANÁLISIS DE TENDENCIAS

- Preparación temporal, agrupación por fecha y construcción de las series generales.
- Agrupación por región, selección de principales marcas y promedio móvil.

```
[67]: # Paso 4: Analizar la tendencia de ventas en el tiempo

import matplotlib.pyplot as plt
import seaborn as sns

# Aseguramos que la fecha esté en datetime
df_consolidado['date'] = pd.to_datetime(df_consolidado['date'])
df_consolidado.sort_values(by='date', inplace=True)

# Agrupación por fecha
ventas_por_fecha = df_consolidado.groupby('date').agg({
    'total_value_sales': 'sum',
    'total_unit_sales': 'sum'
}).reset_index()

# Gráfico general ventas en valor
plt.figure(figsize=(14, 6))
sns.lineplot(data=ventas_por_fecha, x='date', y='total_value_sales')
plt.title('Tendencia de Ventas en Valor a lo largo del tiempo')
plt.xlabel('Fecha')
plt.ylabel('Ventas en Valor')
plt.xticks(rotation=45)
plt.savefig('4.1-Tendencia de Ventas en Valor a lo largo del tiempo.png', dpi=300, bbox_inches='tight')
plt.show()

# Gráfico general ventas en unidades
plt.figure(figsize=(14, 6))
sns.lineplot(data=ventas_por_fecha, x='date', y='total_unit_sales')
plt.title('Tendencia de Ventas en Unidades a lo largo del tiempo')
plt.xlabel('Fecha')
plt.ylabel('Ventas en Unidades')
plt.xticks(rotation=45)
plt.savefig('4.2-Tendencia de Ventas en Unidades a lo largo del tiempo.png', dpi=300, bbox_inches='tight')
plt.show()
```

```
# Tendencia por región
ventas_por_fecha_region = df_consolidado.groupby(['date', 'region']).agg({
    'total_value_sales': 'sum'
}).reset_index()

plt.figure(figsize=(14, 6))
sns.lineplot(data=ventas_por_fecha_region, x='date', y='total_value_sales', hue='region')
plt.title('Tendencia de Ventas en Valor por Región a lo largo del tiempo')
plt.xlabel('Fecha')
plt.ylabel('Ventas en Valor')
plt.legend(title='Región')
plt.xticks(rotation=45)
plt.savefig('4.3-Tendencia de Ventas en Valor por Región a lo largo del tiempo.png', dpi=300, bbox_inches='tight')
plt.show()

# Tendencia por marca
ventas_por_fecha_marca = df_consolidado.groupby(['date', 'brand']).agg({
    'total_value_sales': 'sum'
}).reset_index()

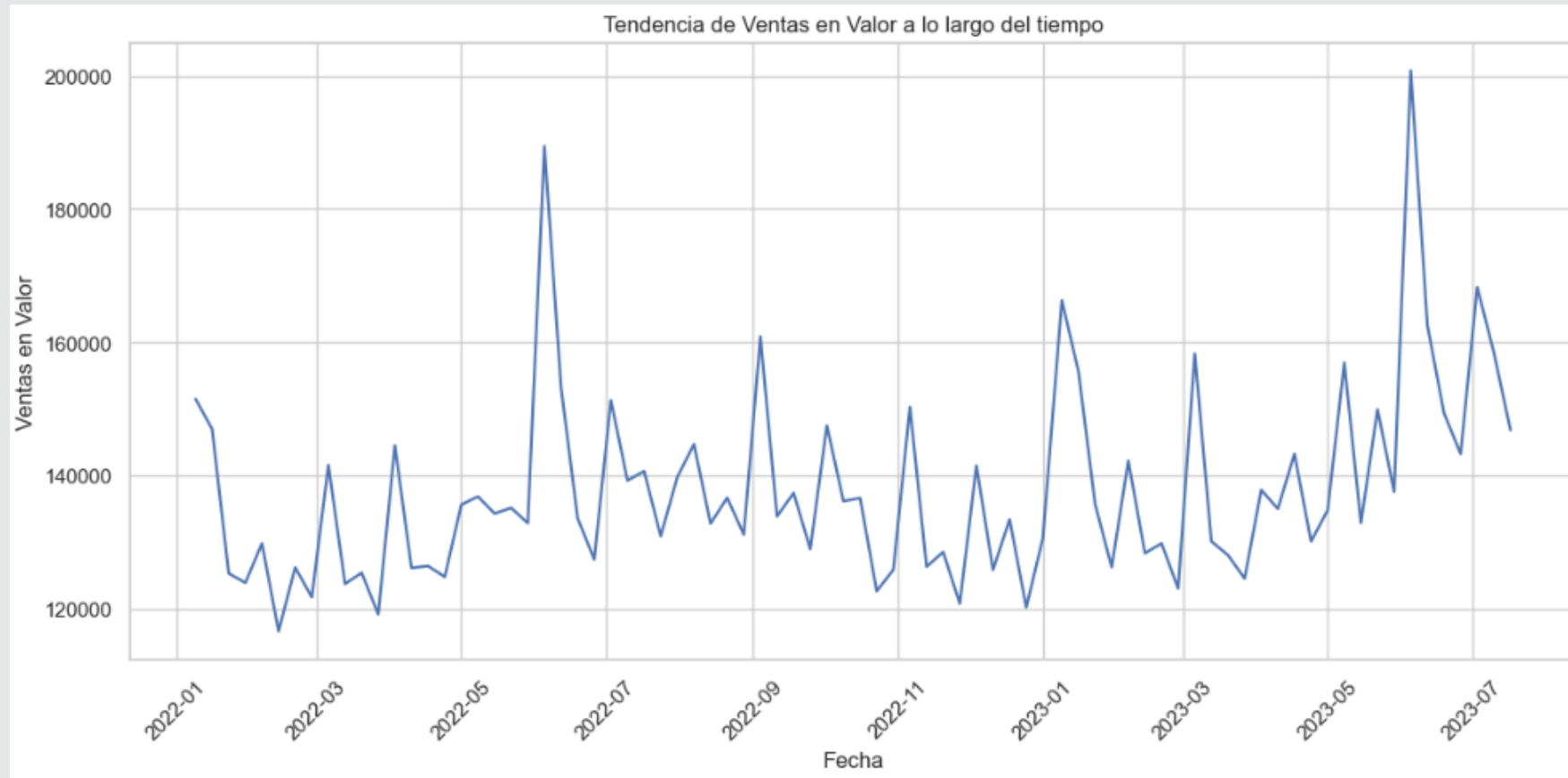
marcas_principales = ventas_por_fecha_marca['brand'].value_counts().head(5).index.tolist()
ventas_marcas_filtradas = ventas_por_fecha_marca[ventas_por_fecha_marca['brand'].isin(marcas_principales)]

plt.figure(figsize=(14, 6))
sns.lineplot(data=ventas_marcas_filtradas, x='date', y='total_value_sales', hue='brand')
plt.title('Tendencia de Ventas en Valor por Marca a lo largo del tiempo (Top 5 Marcas)')
plt.xlabel('Fecha')
plt.ylabel('Ventas en Valor')
plt.legend(title='Marca')
plt.xticks(rotation=45)
plt.savefig('4.4-Tendencia de Ventas en Valor por Marca a lo largo del tiempo.png', dpi=300, bbox_inches='tight')
plt.show()

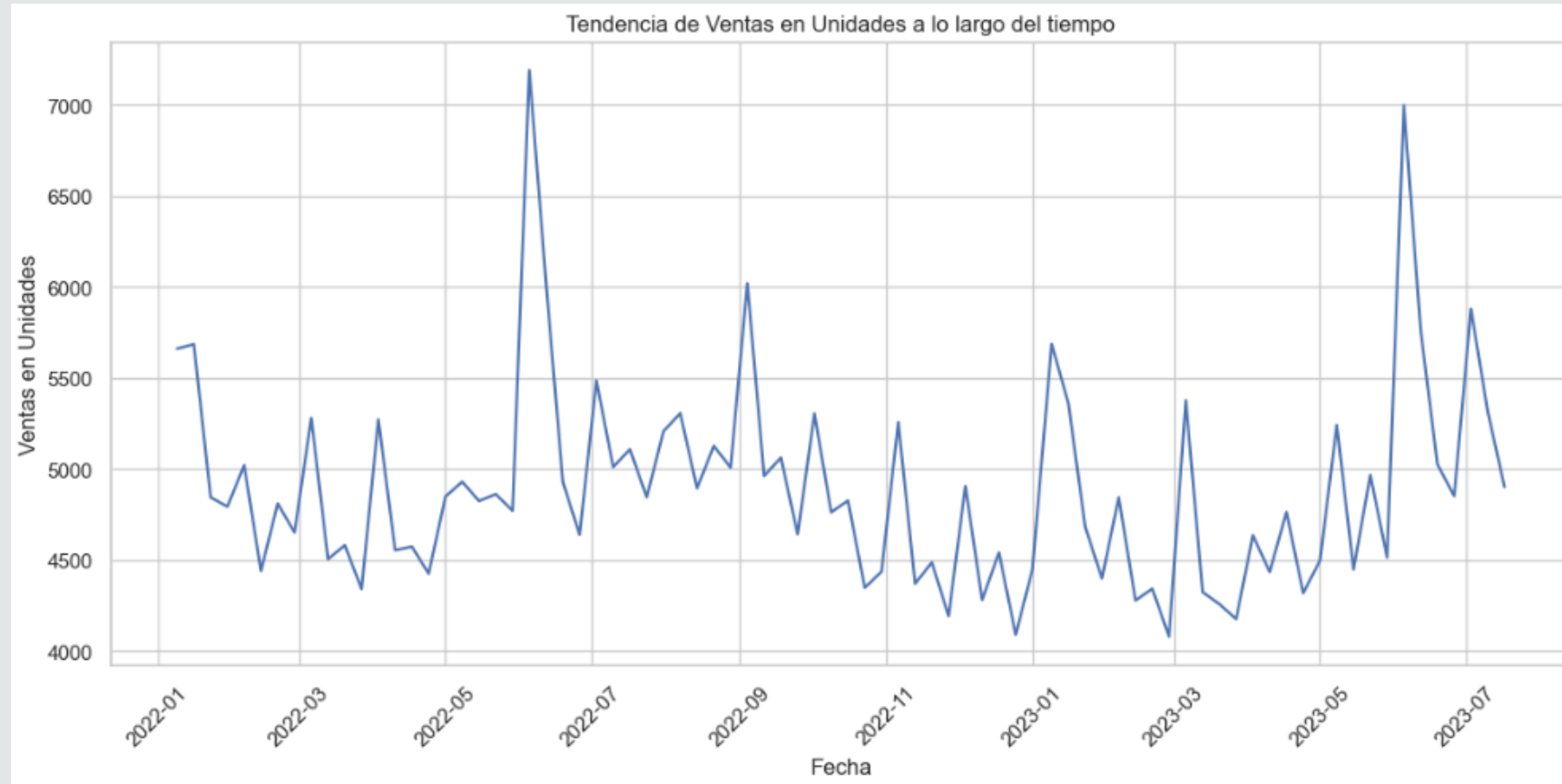
# Promedio móvil opcional
ventas_por_fecha['rolling_value_sales'] = ventas_por_fecha['total_value_sales'].rolling(window=4).mean()

plt.figure(figsize=(14, 6))
sns.lineplot(data=ventas_por_fecha, x='date', y='rolling_value_sales')
plt.title('Tendencia de Ventas en Valor con Promedio Móvil (4 semanas)')
plt.xlabel('Fecha')
plt.ylabel('Promedio Móvil de Ventas en Valor')
plt.savefig('4.5-Tendencia de Ventas en Valor con Promedio Móvil.png', dpi=300, bbox_inches='tight')
plt.xticks(rotation=45)
plt.show()
```

3.2 – ANÁLISIS DE TENDENCIAS

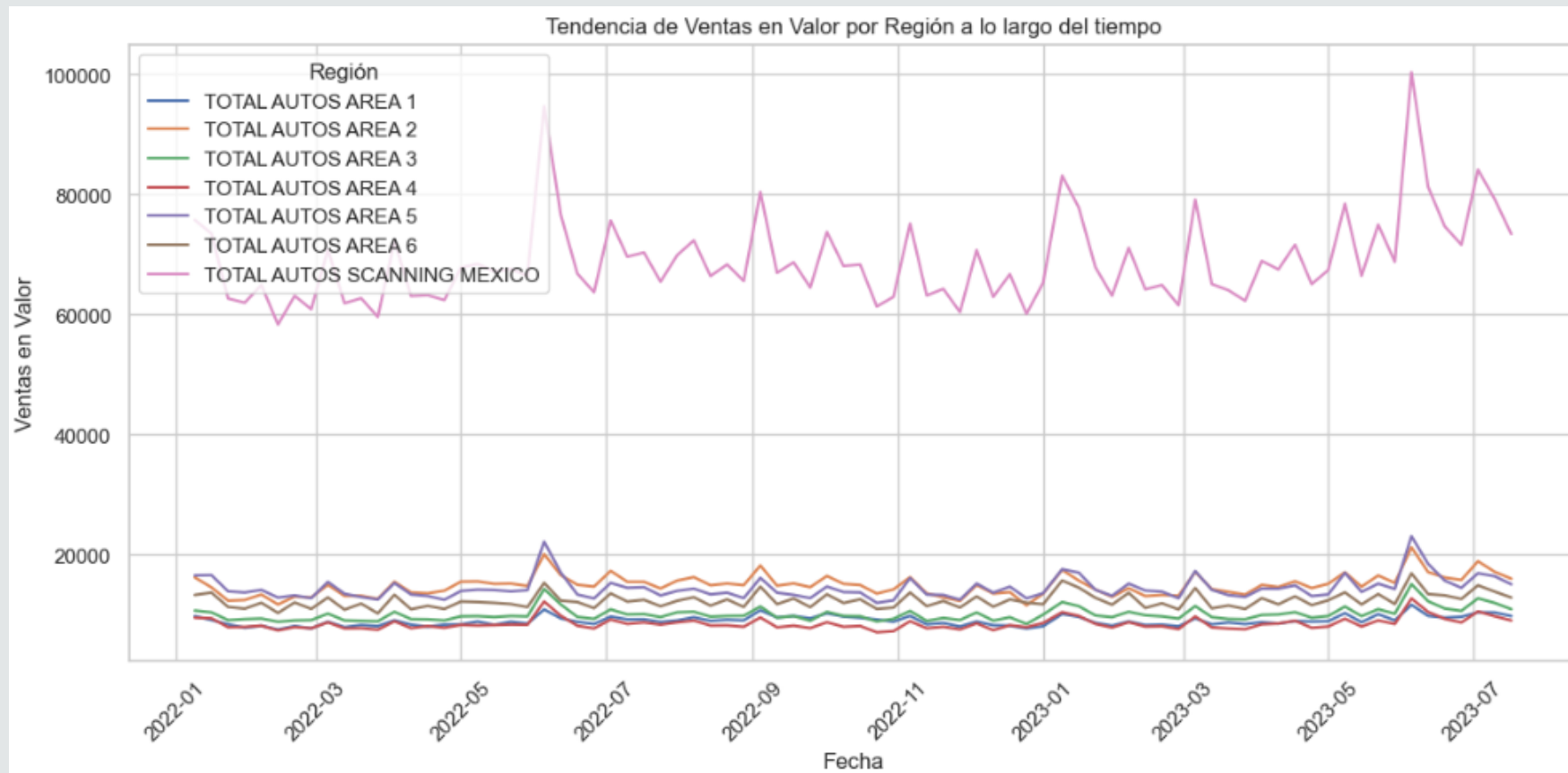


3.2 – ANÁLISIS DE TENDENCIAS



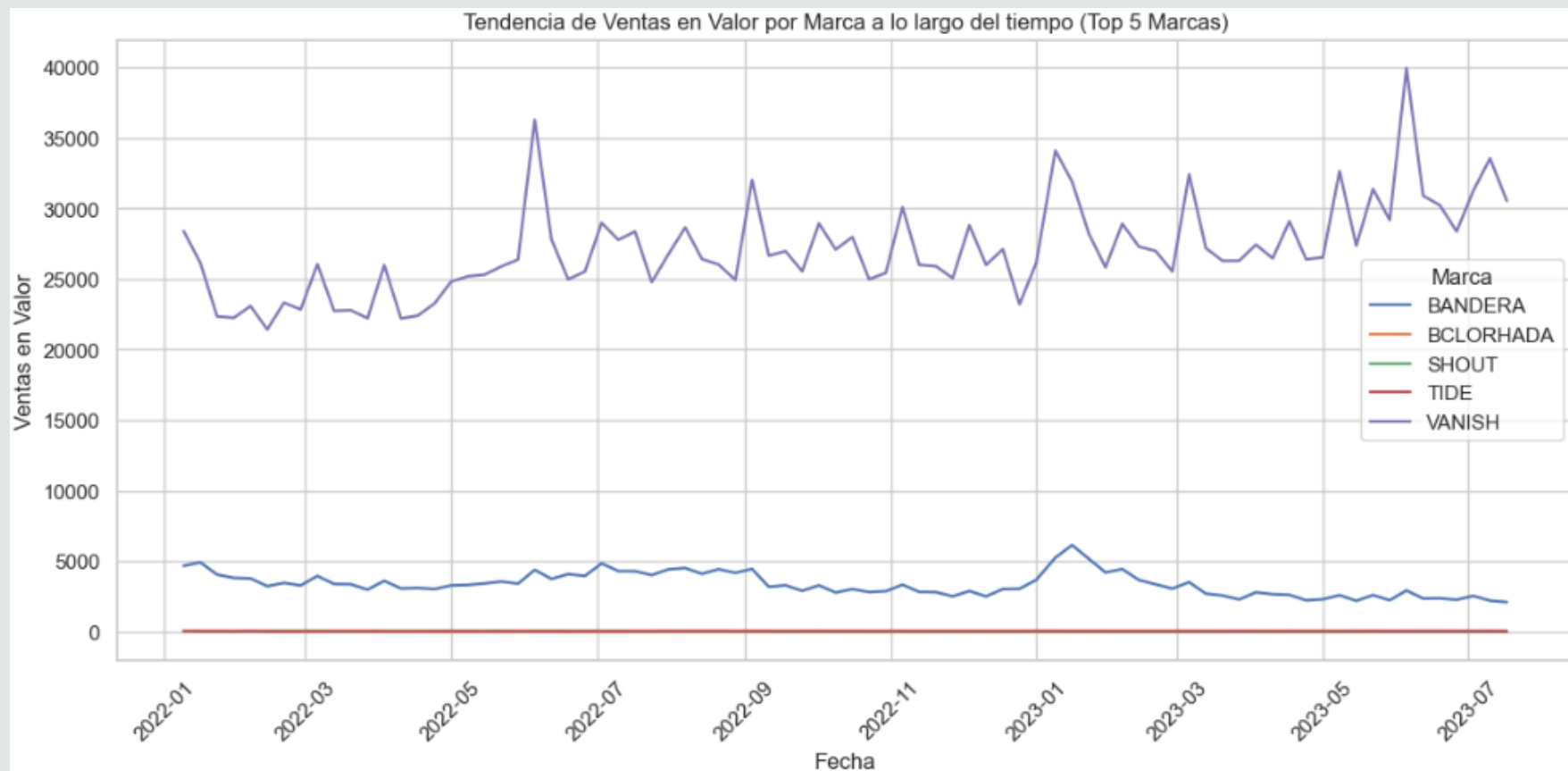
3.2 – ANÁLISIS DE TENDENCIAS

- Esta grafica permite mostrar que TOTAL AUTOS SCANNING MEXICO permanece muy por encima de las demás regiones durante prácticamente todo el periodo. Eso también explica buena parte de los valores extremos que observamos posteriormente.

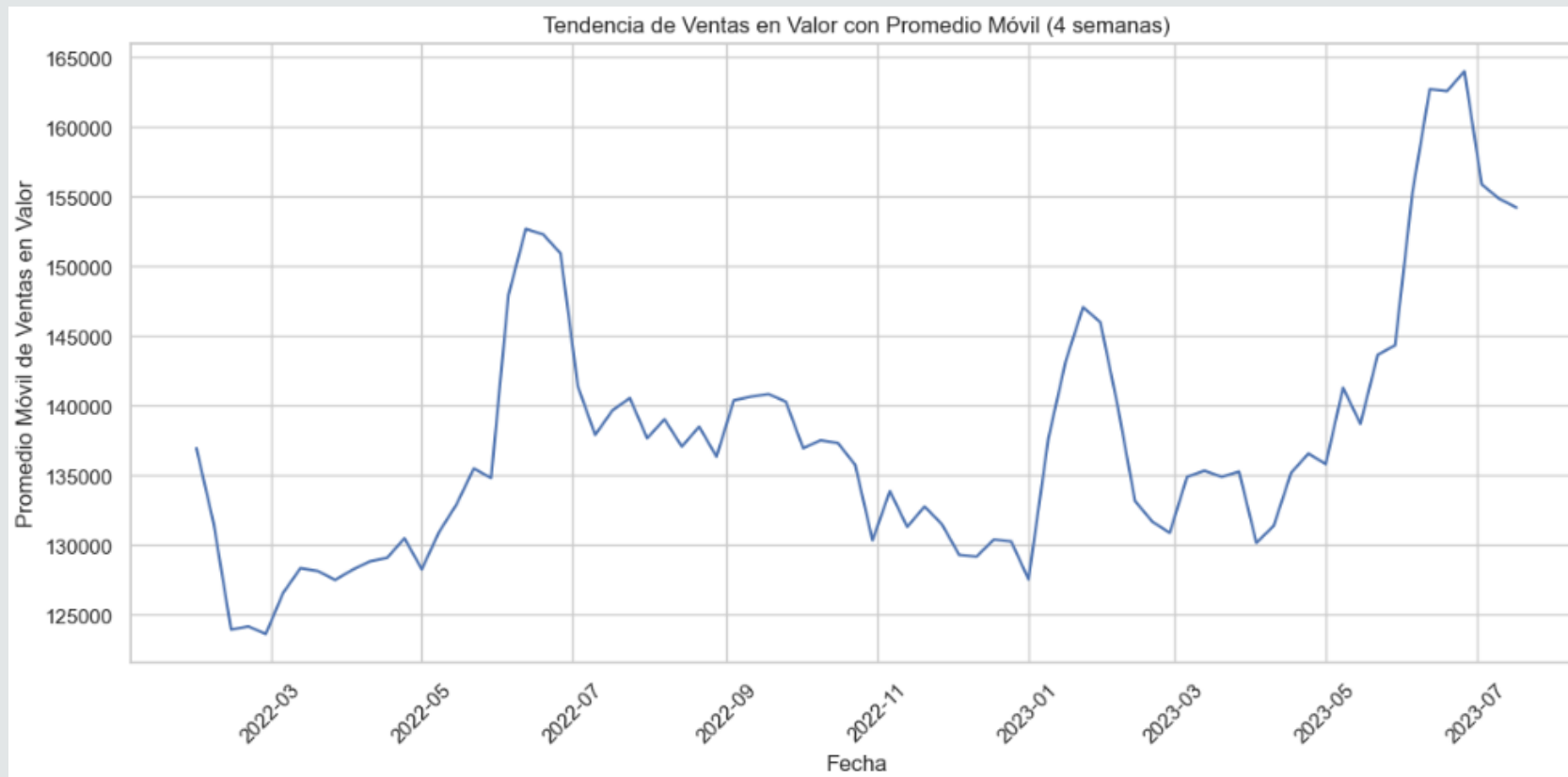


3.2 – ANÁLISIS DE TENDENCIAS

- Esta imagen muestra otra conclusión interesante: Vanish domina ampliamente las ventas entre las marcas representadas, mientras que el resto permanece en niveles mucho menores.



3.2 – ANÁLISIS DE TENDENCIAS



3.3 – IDENTIFICACIÓN DE OUTLIERS

- Los histogramas, gráficos de dispersión y boxplots evidenciaron la presencia de numerosas observaciones alejadas de la distribución central.
- Se analizaron las relaciones entre ventas en unidades y ventas en valor, complementándolas con comparaciones por región y producto.
- Los valores extremos se concentran especialmente en determinadas regiones y productos, por lo que no fueron considerados automáticamente como errores: pueden representar operaciones reales de alto volumen.
- Se utilizaron filtros visuales para estudiar con mayor claridad la distribución principal sin modificar el dataset original.

3.3 – IDENTIFICACIÓN DE OUTLIERS

- Código de gráficos de dispersión y análisis de relaciones entre variables.
- Código del filtrado visual utilizado para reducir el efecto de observaciones extremas.

```
[69]: # Paso 5: Explorar la relación entre diferentes variables.

import matplotlib.pyplot as plt
import seaborn as sns

# Scatter Plot 1: Ventas en unidades vs. Ventas en valor
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_consolidado, x='total_unit_sales', y='total_value_sales', alpha=0.5)
plt.title('Relación entre Ventas en Unidades y Ventas en Valor')
plt.xlabel('Ventas en Unidades')
plt.ylabel('Ventas en Valor')
plt.savefig('5.1-Relación entre Ventas en Unidades y Ventas en Valor.png', dpi=300, bbox_inches='tight')
plt.show()

# Scatter Plot 2: Precio promedio vs. Ventas en unidades
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_consolidado, x='avg_price_per_unit', y='total_unit_sales', alpha=0.5)
plt.title('Relación entre Precio Promedio y Ventas en Unidades')
plt.xlabel('Precio Promedio por Unidad')
plt.ylabel('Ventas en Unidades')
plt.savefig('5.2-Relación entre Precio Promedio y Ventas en Unidades.png', dpi=300, bbox_inches='tight')
plt.show()

# Scatter Plot 3: Margen de beneficio vs. Ventas en valor
plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_consolidado, x='profit_margin', y='total_value_sales', alpha=0.5)
plt.title('Relación entre Margen de Beneficio y Ventas en Valor')
plt.xlabel('Margen de Beneficio')
plt.ylabel('Ventas en Valor')
plt.savefig('5.3-Relación entre Margen de Beneficio y Ventas en Valor.png', dpi=300, bbox_inches='tight')
plt.show()

# Mapa de Calor de Correlaciones
plt.figure(figsize=(10, 8))
correlaciones = df_consolidado[['total_unit_sales', 'total_value_sales', 'avg_price_per_unit', 'profit_margin', 'total_unit_avg_weekly_sales']].corr()

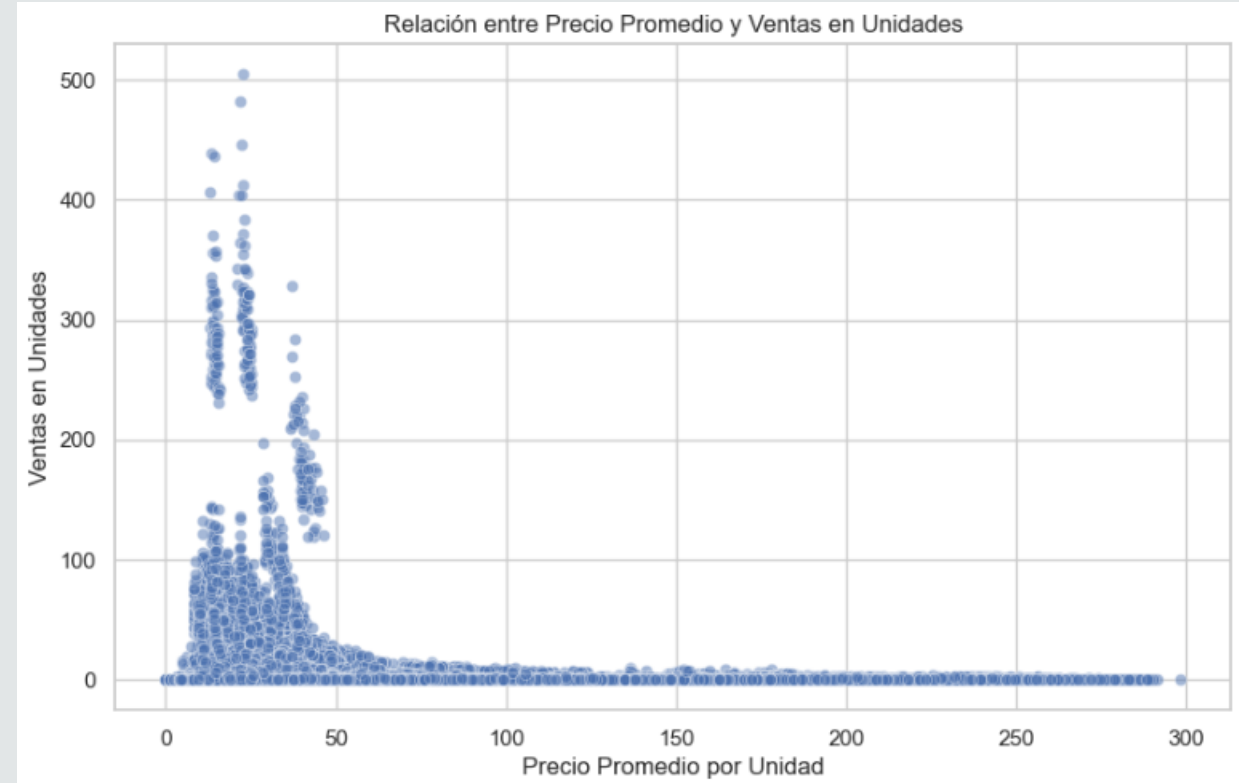
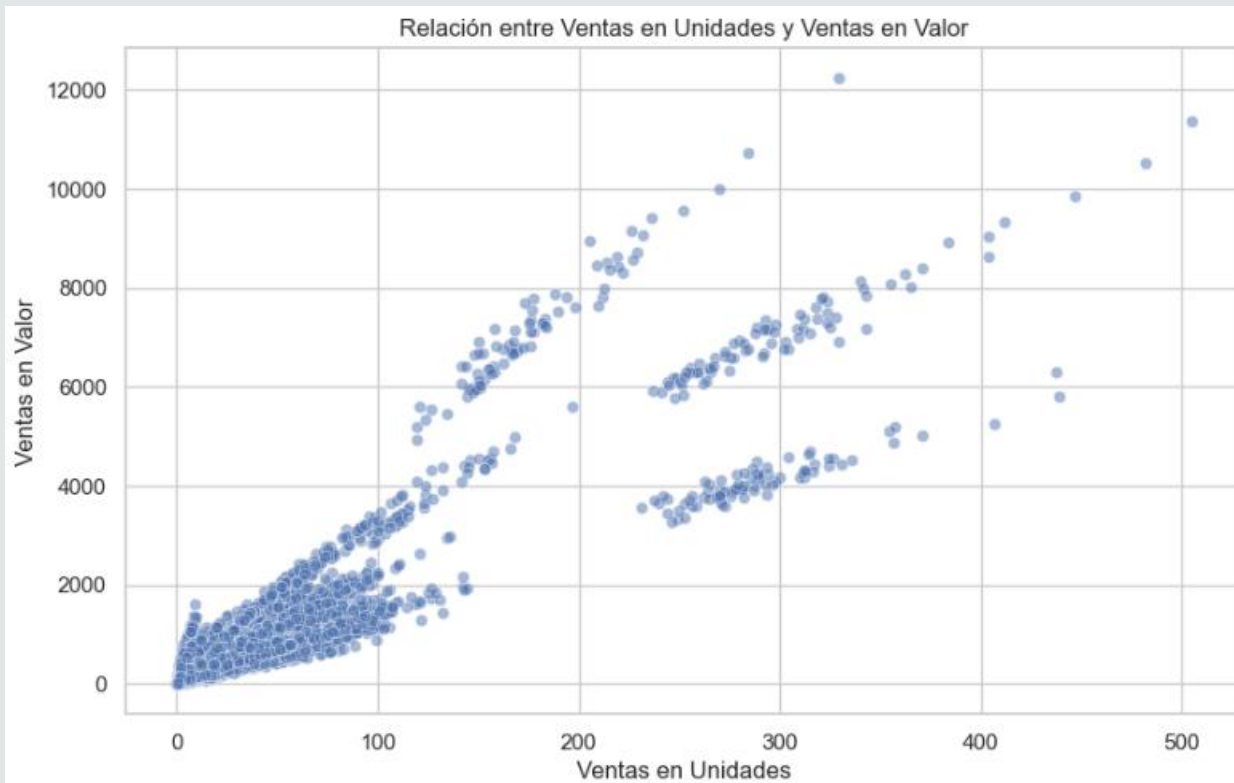
sns.heatmap(correlaciones, annot=True, cmap='coolwarm', fmt='.2f')
plt.title('Mapa de Correlaciones entre Variables Numéricas')
plt.savefig('5.4-Mapa de Correlaciones entre Variables Numéricas.png', dpi=300, bbox_inches='tight')
plt.show()

# Filtro opcional para eliminar outliers extremos en scatter plot
df_filtrado = df_consolidado[(df_consolidado['total_unit_sales'] < 200) & (df_consolidado['total_value_sales'] < 2000)]

plt.figure(figsize=(10, 6))
sns.scatterplot(data=df_filtrado, x='total_unit_sales', y='total_value_sales', alpha=0.5)
plt.title('Relación Filtrada entre Ventas en Unidades y Ventas en Valor (<200 unidades y <2000 valor)')
plt.xlabel('Ventas en Unidades')
plt.ylabel('Ventas en Valor')
plt.savefig('5.5-Relación Filtrada entre Ventas en Unidades y Ventas en Valor.png', dpi=300, bbox_inches='tight')
plt.show()
```

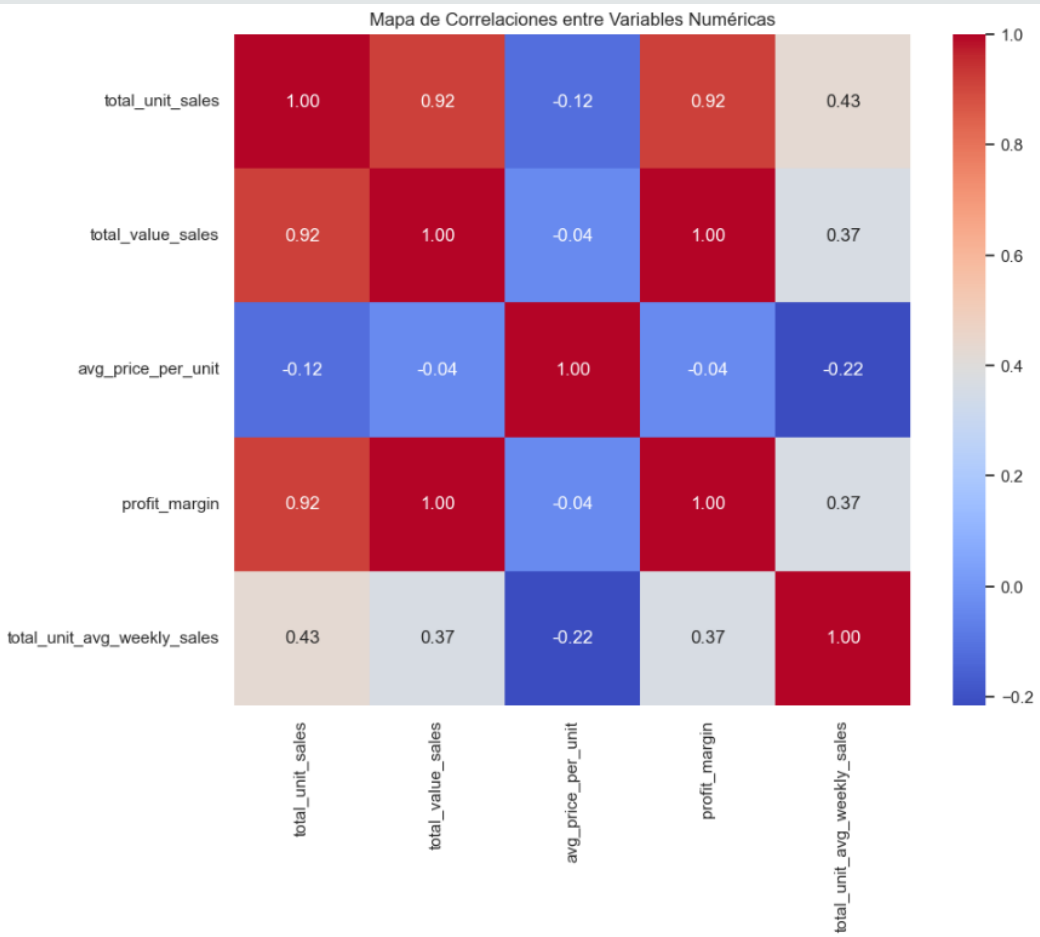
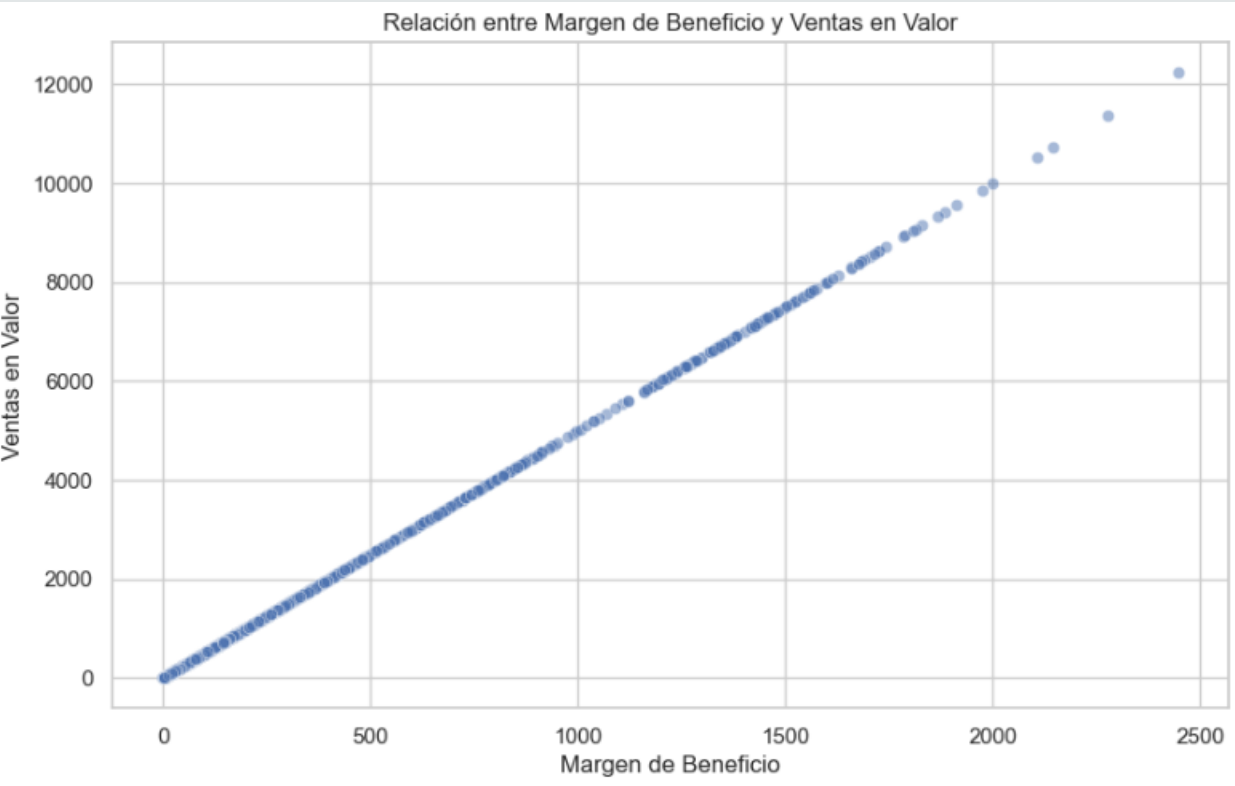
3.3 – IDENTIFICACIÓN DE OUTLIERS

- Se observa una relación positiva entre unidades y valor, pero algunos registros llegan hasta aproximadamente 500 unidades y más de 12,000 en valor, comprimiendo visualmente la mayor parte de las observaciones.

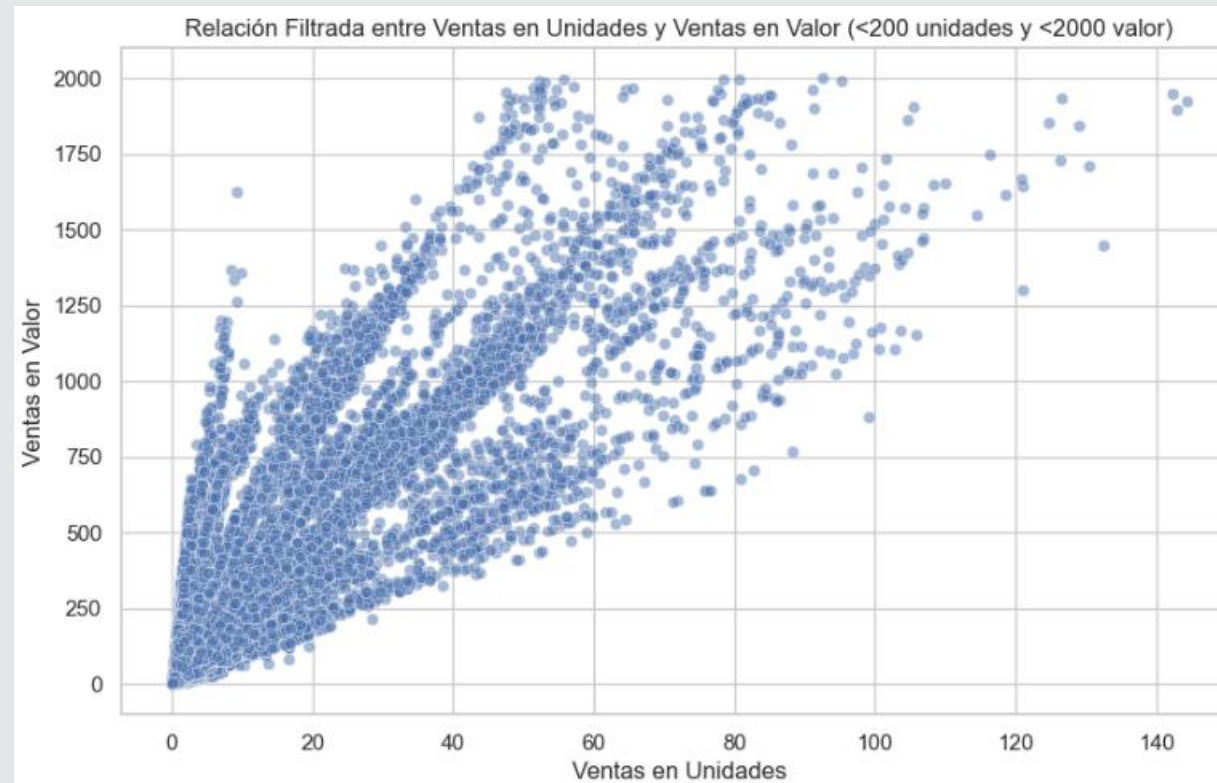


3.3 – IDENTIFICACIÓN DE OUTLIERS

- Al limitar únicamente la visualización, se aprecia mucho mejor la estructura de la mayoría de los datos y las distintas relaciones entre unidades y valor. Eso demuestra una decisión analítica razonable: los extremos se investigaron en lugar de eliminarlos automáticamente.



3.3 – IDENTIFICACIÓN DE OUTLIERS



4 – SEGMENTACIÓN DE PRODUCTOS O REGIONES CON CLUSTERING



4.1 – APLICACIÓN DE CLUSTERING CON K-MEANS

- Las 122,002 observaciones de ventas se agruparon a nivel producto, obteniendo 350 productos únicos para realizar una segmentación consistente. Se utilizaron características de:
 - Desempeño: ventas en unidades, valor y promedio semanal.
 - Producto: categoría, formato, atributos y segmento.
 - Geografía: participación de ventas en las 7 regiones.
- Las variables numéricas se estandarizaron con StandardScaler y las categóricas mediante One-Hot Encoding, generando 32 variables para el modelo. Finalmente, se aplicó K-Means para identificar productos con patrones comerciales similares.

4.1 – APLICACIÓN DE CLUSTERING CON K-MEANS

18. Seleccionar y preparar las características para el clustering de productos

Variables descriptivas que identifican y caracterizan a cada producto

```
columnas_producto = [  
    'ItemCode',  
    'ItemDescription',  
    'Manufacturer',  
    'Brand',  
    'CategoryName',  
    'Format',  
    'Attr1',  
    'Attr2',  
    'Attr3',  
    'Segment'  
]
```

1. Agregar las ventas para obtener una fila por producto

```
df_productos_cluster = (  
    df_consolidado  
        .groupby(columnas_producto, dropna=False)  
        .agg(  
            TotalUnitSales=('TotalUnitSales', 'sum'),  
            TotalValueSales=('TotalValueSales', 'sum'),  
            AvgWeeklySales=('TotalUnitAvgWeeklySales', 'mean')  
        )  
        .reset_index()  
)
```

2. Incorporar la información geográfica:

ventas en valor de cada producto por región

```
ventas_por_region = (  
    df_consolidado  
        .pivot_table(  
            index='ItemCode',  
            columns='Region',  
            values='TotalValueSales',  
            aggfunc='sum',  
            fill_value=0  
        )  
        .reset_index()  
)
```

Renombrar las columnas regionales para identificarlas fácilmente

```
ventas_por_region.columns = [  
    'ItemCode' if col == 'ItemCode'  
    else f'Region_{col}'  
    for col in ventas_por_region.columns  
]
```

3. Unir las características del producto con su comportamiento regional

```
df_productos_cluster = df_productos_cluster.merge(  
    ventas_por_region,  
    on='ItemCode',  
    how='left',  
    validate='one_to_one'  
)
```

4. Verificación del resultado

```
print('Dimensiones del dataset original:', df_consolidado.shape)  
print('Productos únicos en el consolidado:', df_consolidado['ItemCode'].nunique())  
print('Dimensiones del dataset para clustering:', df_productos_cluster.shape)
```

print('\nColumnas seleccionadas:')

```
print(df_productos_cluster.columns.tolist())
```

```
df_productos_cluster.head()
```

Dimensiones del dataset original: (122002, 20)

Productos únicos en el consolidado: 350

Dimensiones del dataset para clustering: (350, 20)

Columnas seleccionadas:

```
['ItemCode', 'ItemDescription', 'Manufacturer', 'Brand', 'CategoryName', 'Format', 'Attr1', 'Attr2', 'Attr3', 'Segment', 'TotalUnitSales', 'TotalValueSales', 'AvgWeeklySales', 'Region_TOTAL AUTOS AREA 1', 'Region_TOTAL AUTOS AREA 2', 'Region_TOTAL AUTOS AREA 3', 'Region_TOTAL AUTOS AREA 4', 'Region_TOTAL AUTOS AREA 5', 'Region_TOTAL AUTOS AREA 6', 'Region_TOTAL AUTOS SCANNING MEXICO']
```

	ItemCode	ItemDescription	Manufacturer	Brand	CategoryName	Format	Attr1	Attr2	Attr3	Segment	TotalUnitSales	TotalValueSales	AvgWeeklySales
0	0000075000592	CLORALEX EL RENDIDOR BOT.PLAST. 250ML NAL 000...	INDS. ALEN	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	COLORO	COLORO	NO DEFINIDO	BLEACH	6.042	24.663	4.8
1	0000075000608	CLORALEX EL RENDIDOR BOT.PLAST. 500ML NAL 000...	INDS. ALEN	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	COLORO	COLORO	NO DEFINIDO	BLEACH	1423.786	12423.156	35.1
2	0000075000615	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL 000...	INDS. ALEN	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	COLORO	COLORO	NO DEFINIDO	BLEACH	46461.839	658584.568	92.5
3	0000075000622	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL 000...	INDS. ALEN	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	COLORO	COLORO	NO DEFINIDO	BLEACH	48445.151	1139233.787	112.0
4	0000075000639	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000...	INDS. ALEN	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	COLORO	COLORO	NO DEFINIDO	BLEACH	28334.417	1146849.632	60.2

4.1 – APLICACIÓN DE CLUSTERING CON K-MEANS

```
# 19. Preparar y estandarizar las características para K-Means

import numpy as np
import pandas as pd

from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer

# Creamos una copia para no modificar el DataFrame preparado anteriormente
df_modelo = df_productos_cluster.copy()

# -----
# 1. Limpiar textos de las variables categóricas
# -----

columnas_categoricas = [
    'CategoryName',
    'Format',
    'Attr1',
    'Attr2',
    'Attr3',
    'Segment'
]

for col in columnas_categoricas:
    df_modelo[col] = (
        df_modelo[col]
        .astype(str)
        .str.replace(r'[\r\n]+' , ' ', regex=True)
        .str.replace(r'\\s+' , ' ', regex=True)
        .str.strip()
    )

# -----
# 2. Identificar las columnas correspondientes a regiones
# -----

columnas_region = [
    col for col in df_modelo.columns
    if col.startswith('Region_')
]

print('Columnas regionales encontradas:')
print(columnas_region)
```

```
# -----
# 3. Convertir ventas regionales absolutas en proporciones
# -----

for col in columnas_region:
    nueva_columna = col.replace('Region_', 'Share_')

    df_modelo[nueva_columna] = (
        df_modelo[col] /
        df_modelo['TotalValueSales'].replace(0, np.nan)
    ).fillna(0)

columnas_share_region = [
    col for col in df_modelo.columns
    if col.startswith('Share_')
]

# -----
# 4. Seleccionar variables numéricas para el clustering
# -----

columnas_numericas = [
    'TotalUnitSales',
    'TotalValueSales',
    'AvgWeeklySales'
] + columnas_share_region

# -----
# 5. Crear el preprocesador
# -----

preprocesador = ColumnTransformer(
    transformers=[
        (
            'numericas',
            StandardScaler(),
            columnas_numericas
        ),
        (
            'categoricas',
            OneHotEncoder(
                drop='first',
                handle_unknown='ignore',
                sparse_output=False
            ),
            columnas_categoricas
        )
    ],
    remainder='drop'
)
```

4.1 – APLICACIÓN DE CLUSTERING CON K-MEANS

```
# -----  
# 6. Aplicar la transformación  
# -----  
  
X_cluster = preprocesador.fit_transform(df_modelo)  
  
# -----  
# 7. Obtener los nombres de las variables transformadas  
# -----  
  
nombres_variables = preprocesador.get_feature_names_out()  
  
df_cluster_preparado = pd.DataFrame(  
    X_cluster,  
    columns=nombres_variables,  
    index=df_modelo.index  
)  
  
# -----  
# 8. Verificar el resultado  
# -----  
  
print('\nDimensiones antes del preprocesamiento:',  
      df_modelo.shape)  
  
print('Dimensiones después del preprocesamiento:',  
      df_cluster_preparado.shape)  
  
print('\nValores nulos después del preprocesamiento:',  
      df_cluster_preparado.isnull().sum().sum())  
  
print('\nPrimeras variables generadas:')  
print(df_cluster_preparado.columns.tolist()[:20])  
  
df_cluster_preparado.head()
```

```
# 21. Aplicar el modelo K-Means definitivo con k = 2  
  
from sklearn.cluster import KMeans  
from sklearn.metrics import silhouette_score  
  
# -----  
# 1. Crear y entrenar el modelo definitivo  
# -----  
  
k_optimo = 2  
  
kmeans_final = KMeans(  
    n_clusters=k_optimo,  
    random_state=42,  
    n_init=20  
)  
  
clusters = kmeans_final.fit_predict(df_cluster_preparado)  
  
# -----  
# 2. Incorporar el cluster al dataset original de productos  
# -----  
  
df_resultados = df_productos_cluster.copy()  
  
df_resultados['Cluster'] = clusters  
  
# -----  
# 3. Verificar la asignación  
# -----  
  
print('Número de productos por cluster:')  
print(df_resultados['Cluster'].value_counts().sort_index())  
  
print('\nPorcentaje de productos por cluster:')  
print(  
    df_resultados['Cluster']  
    .value_counts(normalize=True)  
    .sort_index()  
    .mul(100)  
    .round(2)  
)
```

4.1 – APLICACIÓN DE CLUSTERING CON K-MEANS

```
# -----  
# 4. Verificar calidad final del clustering  
# -----  
  
silhouette_final = silhouette_score(  
    df_cluster_preparado,  
    clusters  
)  
  
print(f'\nSilhouette Score final: {silhouette_final:.4f}')
```

```
# -----  
# 5. Mostrar una vista de los productos con su cluster  
# -----  
  
columnas_revision = [  
    'ItemCode',  
    'ItemDescription',  
    'Brand',  
    'CategoryName',  
    'Format',  
    'Segment',  
    'TotalUnitSales',  
    'TotalValueSales',  
    'AvgWeeklySales',  
    'Cluster'  
]  
  
df_resultados[columnas_revision].head(10)
```

Número de productos por cluster:

```
Cluster  
0    347  
1      3
```

Name: count, dtype: int64

Porcentaje de productos por cluster:

```
Cluster  
0    99.14  
1     0.86
```

Name: proportion, dtype: float64

Silhouette Score final: 0.7069

	ItemCode	ItemDescription	Brand	CategoryName	Format	Segment	TotalUnitSales	TotalValueSales	AvgWeeklySales	Cluster
0	0000075000592	CLORALEX EL RENDIDOR BOT.PLAST. 250ML NAL 000...	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	BLEACH	6.042	24.663	4.860489	0
1	0000075000608	CLORALEX EL RENDIDOR BOT.PLAST. 500ML NAL 000...	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	BLEACH	1423.786	12423.156	35.166597	0
2	0000075000615	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL 000...	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	BLEACH	46461.839	658584.568	92.527904	1
3	0000075000622	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL 000...	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	BLEACH	48445.151	1139233.787	112.098525	1
4	0000075000639	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000...	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	BLEACH	28334.417	1146849.632	60.226977	1
5	0000075000677	CLORALEX FRESCURA CITRICA BOT PLAST 950ML NAL...	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	BLEACH	0.002	0.004	1.000000	0
6	0000075000684	CLORALEX FRESCURA CITRICA BOT PLAST 2000ML NAL...	CLORALEX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	BLEACH	0.004	0.020	1.000000	0
7	0019200958714	LYSOL SANITIZANTE DE ROPA CRISP LINEN BOT 1.2 ...	LYSOL	FABRIC TREATMENT and SANITVr\n	LIQUIDO	SANITIZER	0.004	0.474	1.000000	0
8	0037000018704	9TIDE LIPIADOR DE ROPA QUITAMANCHAS PLUMON 10 ...	TIDE	FABRIC TREATMENT and SANITVr\n	LIQUIDO	PRETREAT	38.428	3519.870	2.646341	0
9	0044600305974	CLOROX 2 STAIN FIGTHER REMOVER PEN FOR COLORS ...	CLOROX	FABRIC TREATMENT and SANITVr\n	LIQUIDO	PRETREAT	0.038	1.812	1.947368	0

4.2 – NÚMERO ÓPTIMO DE CLUSTERS

- Se evaluaron soluciones entre $k = 2$ y $k = 10$ mediante:
 - Método del Codo: mostró una disminución progresiva de la inercia, sin un quiebre completamente definido.
 - Silhouette Score: permitió complementar la decisión midiendo cohesión y separación entre clusters.
- La solución $k = 2$ obtuvo el mayor Silhouette Score: 0.7069 muy superior a las alternativas evaluadas (~ 0.19 – 0.30). Por ello, se seleccionaron 2 clusters como solución final.

4.2 – NÚMERO ÓPTIMO DE CLUSTERS

```
# 20. Determinar el número adecuado de clusters
# mediante el Método del Codo y Silhouette Score

import pandas as pd
import matplotlib.pyplot as plt

from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# -----
# 1. Definir el rango de clusters a evaluar
# -----

rango_k = range(2, 11)

inercias = []
silhouettes = []

# -----
# 2. Probar K-Means para diferentes valores de k
# -----

for k in rango_k:

    modelo_kmeans = KMeans(
        n_clusters=k,
        random_state=42,
        n_init=20
    )

    etiquetas = modelo_kmeans.fit_predict(df_cluster_preparado)

    # Inercia: suma de distancias cuadráticas
    # de cada observación a su centroide
    inercias.append(modelo_kmeans.inertia_)

    # Silhouette Score:
    # mide cohesión interna y separación entre clusters
    silhouettes.append(
        silhouette_score(
            df_cluster_preparado,
            etiquetas
        )
    )
```

```
# -----
# 3. Crear tabla resumen
# -----

evaluacion_clusters = pd.DataFrame({
    'k': list(rango_k),
    'Inercia': inercias,
    'Silhouette': silhouettes
})

print('Evaluación de diferentes números de clusters:')
display(evaluacion_clusters)

# -----
# 4. Método del Codo
# -----

plt.figure(figsize=(9, 5))

plt.plot(
    evaluacion_clusters['k'],
    evaluacion_clusters['Inercia'],
    marker='o'
)

plt.title('Método del Codo para determinar el número de clusters')
plt.xlabel('Número de clusters (k)')
plt.ylabel('Inercia')
plt.xticks(list(rango_k))
plt.grid(True)

plt.show()

# -----
# 5. Silhouette Score
# -----

plt.figure(figsize=(9, 5))

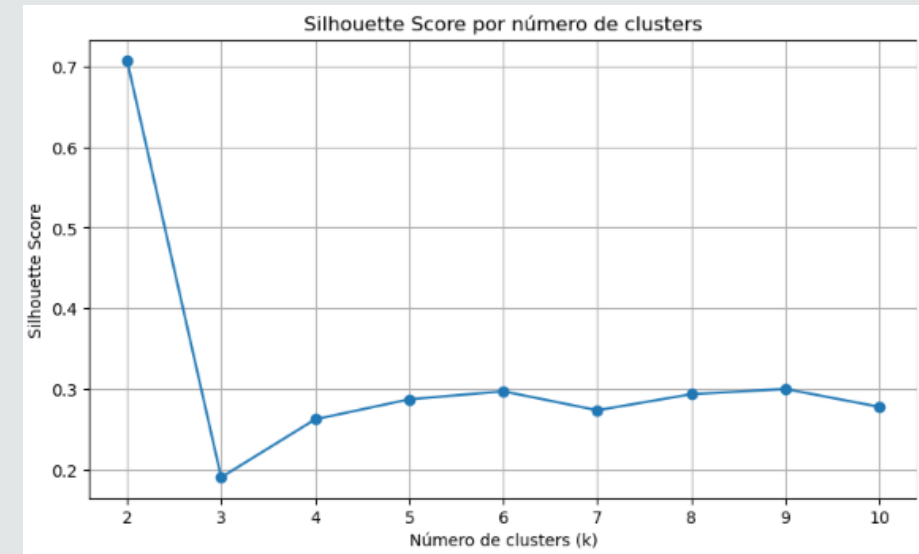
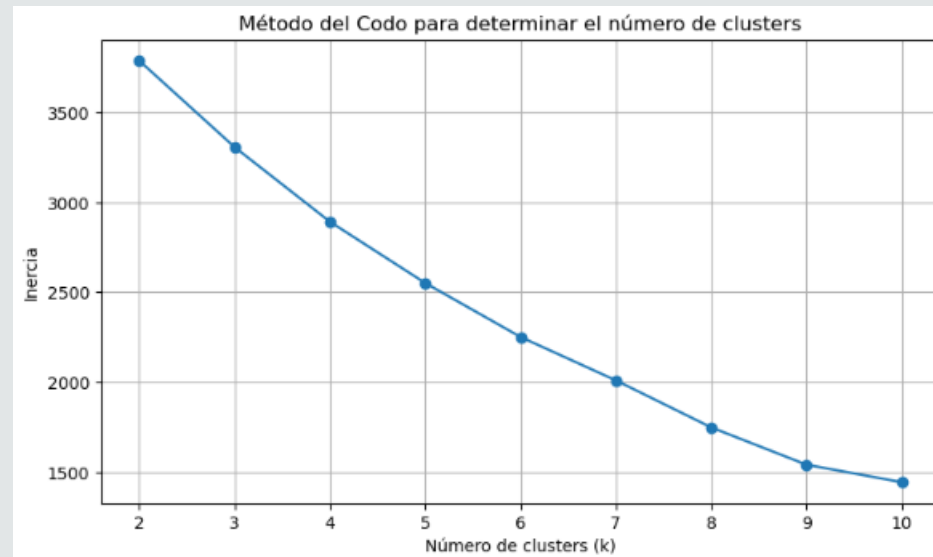
plt.plot(
    evaluacion_clusters['k'],
    evaluacion_clusters['Silhouette'],
    marker='o'
)

plt.title('Silhouette Score por número de clusters')
plt.xlabel('Número de clusters (k)')
plt.ylabel('Silhouette Score')
plt.xticks(list(rango_k))
plt.grid(True)

plt.show()
```

4.2 – NÚMERO ÓPTIMO DE CLUSTERS

	k	Inercia	Silhouette
0	2	3786.870221	0.706892
1	3	3306.199173	0.190513
2	4	2891.291442	0.262455
3	5	2550.261790	0.287265
4	6	2250.054972	0.297184
5	7	2008.649796	0.273683
6	8	1747.156731	0.293610
7	9	1539.506945	0.300057
8	10	1442.067737	0.277894



4.3 – INTERPRETACIÓN DE LOS CLUSTERS

- El modelo identificó una segmentación fuertemente diferenciada por desempeño comercial.
 - Cluster 0 — Portafolio general concentra prácticamente todo el portafolio y presenta un comportamiento mucho más heterogéneo, con ventas bajas y medias.
 - Cluster 1 — Productos de desempeño excepcional sólo contiene 3 productos, todos CLORALEX, líquidos y del segmento BLEACH, pero concentran aproximadamente 26.67% de las ventas totales en valor.
- Insight comercial: El clustering identificó un pequeño grupo de productos estrella cuya escala de ventas es significativamente superior al resto del portafolio. Vanish: presenta una presencia importante dentro del Cluster 0, pero ningún producto. Vanish alcanzó el Cluster 1 de desempeño excepcional.

4.3 – INTERPRETACIÓN DE LOS CLUSTERS

4.3 – INTERPRETACIÓN DE LOS CLUSTERS

```
# 22. Evaluar y caracterizar los resultados del clustering
```

```
import pandas as pd
import numpy as np
```

```
# -----
# 1. Funciones auxiliares para variables categóricas
# -----
```

```
def moda_segura(serie):
    modas = serie.dropna().mode()

    if len(modas) == 0:
        return np.nan

    return modas.iloc[0]

def porcentaje_moda(serie):
    serie = serie.dropna()

    if len(serie) == 0:
        return np.nan

    moda = moda_segura(serie)

    return (serie == moda).mean() * 100
```

```
# -----
# 2. Resumen numérico por cluster
# -----

resumen_cluster = (
    df_resultados
    .groupby('Cluster')
    .agg(
        Productos=('ItemCode', 'nunique'),

        UnitSales_Media=('TotalUnitSales', 'mean'),
        UnitSales_Mediana=('TotalUnitSales', 'median'),

        ValueSales_Media=('TotalValueSales', 'mean'),
        ValueSales_Mediana=('TotalValueSales', 'median'),

        AvgWeeklySales_Media=('AvgWeeklySales', 'mean'),
        AvgWeeklySales_Mediana=('AvgWeeklySales', 'median'),

        VentasValorTotales=('TotalValueSales', 'sum')
    )
    .reset_index()
)

# Participación de productos dentro de cada cluster
resumen_cluster['PorcentajeProductos'] = (
    resumen_cluster['Productos'] /
    resumen_cluster['Productos'].sum()
    * 100
)

# Participación de cada cluster en las ventas totales en valor
resumen_cluster['ParticipacionVentasValor'] = (
    resumen_cluster['VentasValorTotales'] /
    resumen_cluster['VentasValorTotales'].sum()
    * 100
)

print('RESUMEN NUMÉRICO POR CLUSTER')
display(resumen_cluster.round(2))
```

4.3 – INTERPRETACIÓN DE LOS CLUSTERS

```
# -----  
# 3. Perfil categórico dominante de cada cluster  
# -----  
  
variables_categoricas = [  
    'Manufacturer',  
    'Brand',  
    'CategoryName',  
    'Format',  
    'Attr1',  
    'Attr2',  
    'Attr3',  
    'Segment'  
]  
  
perfil_categorico = []  
  
for cluster, grupo in df_resultados.groupby('Cluster'):  
    for variable in variables_categoricas:  
        perfil_categorico.append(  
            'Cluster': cluster,  
            'Variable': variable,  
            'Valor_Dominante': moda_segura(grupo[variable]),  
            'Porcentaje_Dominante': porcentaje_moda(grupo[variable]),  
            'Valores_Unicos': grupo[variable].nunique()  
        )  
  
perfil_categorico = pd.DataFrame(perfil_categorico)  
  
print('\nPERFIL CATEGÓRICO POR CLUSTER')  
display(perfil_categorico.round(2))
```

```
# -----  
# 4. Perfil geográfico de cada cluster  
# -----  
  
columnas_region = [  
    col for col in df_resultados.columns  
    if col.startswith('Region_')  
]  
  
ventas_region_cluster = (  
    df_resultados  
    .groupby('Cluster')[columnas_region]  
    .sum()  
)  
  
# Convertimos las ventas regionales en porcentaje  
perfil_region_cluster = (  
    ventas_region_cluster  
    .div(  
        ventas_region_cluster.sum(axis=1),  
        axis=0  
    )  
    * 100  
)  
  
# Limpiar nombres de columnas  
perfil_region_cluster.columns = [  
    col.replace('Region_', '')  
    for col in perfil_region_cluster.columns  
]  
  
print('\nDISTRIBUCIÓN REGIONAL DE LAS VENTAS POR CLUSTER (%)')  
display(perfil_region_cluster.round(2))
```

4.3 – INTERPRETACIÓN DE LOS CLUSTERS

```
# -----  
# 5. Revisar los productos de mayor venta de cada cluster  
# -----  
  
columnas_productos = [  
    'ItemCode',  
    'ItemDescription',  
    'Manufacturer',  
    'Brand',  
    'Format',  
    'Segment',  
    'TotalUnitSales',  
    'TotalValueSales',  
    'AvgWeeklySales',  
    'Cluster'  
]  
  
productos_destacados = (  
    df_resultados  
        .sort_values(  
            ['Cluster', 'TotalValueSales'],  
            ascending=[True, False]  
        )  
        .groupby('Cluster')  
        .head(10)  
)  
  
print('\nPRODUCTOS CON MAYORES VENTAS EN CADA CLUSTER')  
display(  
    productos_destacados[columnas_productos]  
)
```

```
# -----  
# 6. Mostrar todos los productos del cluster minoritario  
# -----  
  
cluster_minoritario = (  
    df_resultados['Cluster']  
        .value_counts()  
        .idxmin()  
)  
  
print(  
    f'\nPRODUCTOS DEL CLUSTER MINORITARIO '  
    f'(Cluster {cluster_minoritario})'  
)  
  
display(  
    df_resultados.loc[  
        df_resultados['Cluster'] == cluster_minoritario,  
        columnas_productos  
    ]  
        .sort_values(  
            'TotalValueSales',  
            ascending=False  
        )  
)
```

4.3 – INTERPRETACIÓN DE LOS CLUSTERS

RESUMEN NUMÉRICO POR CLUSTER									
Cluster	Productos	UnitSales_Media	UnitSales_Mediana	ValueSales_Media	ValueSales_Mediana	AvgWeeklySales_Media	AvgWeeklySales_Mediana	VentasValorTotales	
0	0	347	773.83	85.40	23337.73	3436.40	7.66	3.16	8098191.91
1	1	3	41080.47	46461.84	981556.00	1139233.79	88.28	92.53	2944667.96
PERFIL CATEGÓRICO POR CLUSTER									
Cluster	Variable		Valor_Dominante	Porcentaje_Dominante	Valores_Unicos				
0	0	Manufacturer	RECKITT	24.78	17				
1	0	Brand	VANISH	22.19	24				
2	0	CategoryName	FABRIC TREATMENT and SANITVr\n	100.00	1				
3	0	Format	LIQUIDO	61.67	5				
4	0	Attr1	CORO	49.86	4				
5	0	Attr2	CORO	49.86	3				
6	0	Attr3	NO DEFINIDO	34.58	8				
7	0	Segment	BLEACH	49.86	7				
8	1	Manufacturer	INDS. ALEN	100.00	1				
9	1	Brand	CLORALEX	100.00	1				
10	1	CategoryName	FABRIC TREATMENT and SANITVr\n	100.00	1				
11	1	Format	LIQUIDO	100.00	1				
12	1	Attr1	CORO	100.00	1				
13	1	Attr2	CORO	100.00	1				
14	1	Attr3	NO DEFINIDO	100.00	1				
15	1	Segment	BLEACH	100.00	1				

DISTRIBUCIÓN REGIONAL DE LAS VENTAS POR CLUSTER (%)										
	TOTAL AUTOS AREA 1	TOTAL AUTOS AREA 2	TOTAL AUTOS AREA 3	TOTAL AUTOS AREA 4	TOTAL AUTOS AREA 5	TOTAL AUTOS AREA 6	TOTAL AUTOS SCANNING MEXICO			
Cluster										
0	6.14	10.45	7.30	6.05	11.53	8.53	50.0			
1	7.36	11.63	7.21	6.38	7.46	9.96	50.0			
PRODUCTOS CON MAYORES VENTAS EN CADA CLUSTER										
	ItemCode	ItemDescription	Manufacturer	Brand	Format	Segment	TotalUnitSales	TotalValueSales	AvgWeeklySales	Cluster
92	7501025402051	BLANCATEL CONCENTRADO BOT PLAST 3750ML NAL 750...	INDS. ALEN	BLANCATEL	LIQUIDO	BLEACH	15613.210	491075.232	43.184630	0
232	7501071900143	CLOROX REGULAR CONCENTRADO BOTPLAST.3800ML 750...	CLOROX	CLOROX	LIQUIDO	BLEACH	11129.230	399994.779	26.395200	0
104	7501025405212	CLORALEX ULTRAGEL CLORO EN GEL CONCENTRADO 950...	INDS. ALEN	CLORALEX	GEL	BLEACH	9667.082	242181.140	26.767171	0
134	7501025450212	CLORALEX EL RENDIDOR BOT 1900+100ML= 2LT 75010...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	10079.695	241498.884	127.815700	0
100	7501025405090	CLORALEX ULTRAGEL CLORO GEL CONCENTRADO BOT 60...	INDS. ALEN	CLORALEX	GEL	BLEACH	14031.408	235784.387	32.389982	0
102	7501025405151	CLORALEX CLORO EN GEL BOT 950ML 7501025405151	INDS. ALEN	CLORALEX	GEL	BLEACH	10968.694	199811.454	24.407734	0
101	7501025405106	CLORALEX CLORO EN GEL BP 2000ML 7501025405106	INDS. ALEN	CLORALEX	GEL	BLEACH	5624.462	175110.993	18.464189	0
333	7503002319529	VANISH LIQUIDO BOTELLA 925ML NAL 7503002319529	RECKITT	VANISH	LIQUIDO	LIQUID & GEL	2937.696	162102.968	7.405821	0
175	7501058717023	VANISH ROSA QUITAMANCHAS GEL MULTUSOS POUCH 4...	RECKITT	VANISH	GEL	LIQUID & GEL	7024.094	158725.720	31.705505	0
121	7501025412005	CLORALEX MASCOTAS BOT PLAST 3750ML NAL 7501025...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	1676.297	132197.343	7.562221	0
4	0000075000639	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	28334.417	1146849.632	60.226977	1
3	0000075000622	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL 000...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	48445.151	1139233.787	112.098525	1
2	0000075000615	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL 000...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	46461.839	658584.568	92.527904	1
PRODUCTOS DEL CLUSTER MINORITARIO (Cluster 1)										
	ItemCode	ItemDescription	Manufacturer	Brand	Format	Segment	TotalUnitSales	TotalValueSales	AvgWeeklySales	Cluster
4	0000075000639	CLORALEX EL RENDIDOR BOT.PLAST. 3750ML NAL 000...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	28334.417	1146849.632	60.226977	1
3	0000075000622	CLORALEX EL RENDIDOR BOT.PLAST. 2000ML NAL 000...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	48445.151	1139233.787	112.098525	1
2	0000075000615	CLORALEX EL RENDIDOR BOT.PLAST. 950ML NAL 000...	INDS. ALEN	CLORALEX	LIQUIDO	BLEACH	46461.839	658584.568	92.527904	1

4.4 – VISUALIZACIÓN DE CLUSTERS

- Para representar gráficamente las 32 variables utilizadas por K-Means, se aplicó PCA (Principal Component Analysis) y se redujo el espacio a dos componentes principales.
- Los dos componentes explican conjuntamente 33.42% de la variabilidad. La visualización muestra que los 3 productos del Cluster 1 se encuentran claramente separados del resto del portafolio, consistente con el alto Silhouette Score obtenido. Las gráficas complementarias de barras y boxplots confirman las fuertes diferencias en volumen, valor y ventas promedio semanales.

4.4 – VISUALIZACIÓN DE CLUSTERS

```
# 23. Visualizar Los clusters en dos dimensiones mediante PCA

import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.decomposition import PCA

# -----
# 1. Reducir las variables del clustering a 2 componentes
# -----

pca = PCA(
    n_components=2,
    random_state=42
)

componentes_pca = pca.fit_transform(
    df_cluster_preparado
)

# -----
# 2. Crear DataFrame para visualización
# -----

df_pca = pd.DataFrame([
    'PCA1': componentes_pca[:, 0],
    'PCA2': componentes_pca[:, 1],
    'Cluster': clusters
])

# Incorporar información descriptiva
df_pca['ItemCode'] = df_resultados['ItemCode'].values
df_pca['Brand'] = df_resultados['Brand'].values
df_pca['ItemDescription'] = df_resultados['ItemDescription'].values
```

```
# -----
# 3. Mostrar varianza explicada
# -----

varianza_explicada = (
    pca.explained_variance_ratio_ * 100
)

print(
    f'Varianza explicada por PCA1: '
    f'{varianza_explicada[0]:.2f}%'
)

print(
    f'Varianza explicada por PCA2: '
    f'{varianza_explicada[1]:.2f}%'
)

print(
    f'Varianza explicada acumulada: '
    f'{varianza_explicada.sum():.2f}%'
)

# -----
# 4. Gráfico de dispersión por cluster
# -----

plt.figure(figsize=(10, 7))

sns.scatterplot(
    data=df_pca,
    x='PCA1',
    y='PCA2',
    hue='Cluster',
    palette='Set1',
    s=70,
    alpha=0.75
)

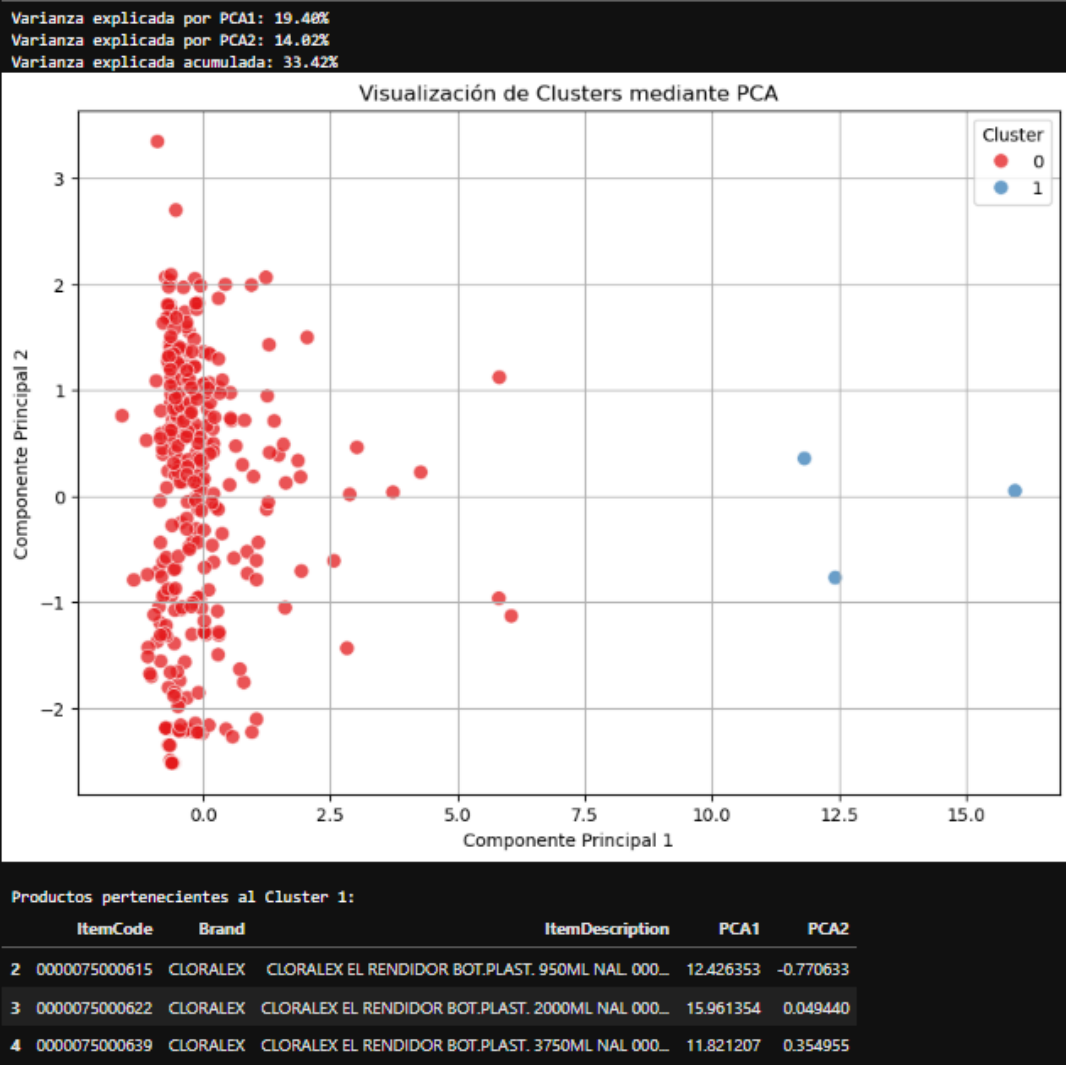
plt.title('Visualización de Clusters mediante PCA')
plt.xlabel('Componente Principal 1')
plt.ylabel('Componente Principal 2')

plt.grid(True)
plt.legend(title='Cluster')

plt.show()
```

4.4 – VISUALIZACIÓN DE CLUSTERS

```
# -----  
# 5. Mostrar productos del cluster minoritario  
# -----  
  
print('\nProductos pertenecientes al Cluster 1:')  
  
display(  
    df_pca.loc[  
        df_pca['Cluster'] == 1,  
        [  
            'ItemCode',  
            'Brand',  
            'ItemDescription',  
            'PCA1',  
            'PCA2'  
        ]  
    ]  
)
```



4.4 – VISUALIZACIÓN DE CLUSTERS

```
# 24. Generar visualizaciones adicionales para analizar los clusters
```

```
import matplotlib.pyplot as plt
import seaborn as sns
```

```
# -----
# 1. Número de productos por Cluster
# -----
```

```
plt.figure(figsize=(7, 5))
```

```
sns.countplot(
    data=df_resultados,
    x='Cluster'
)
```

```
plt.title('Número de productos por Cluster')
plt.xlabel('Cluster')
plt.ylabel('Número de productos')
plt.grid(axis='y')
```

```
plt.show()
```

```
# -----
# 2. Promedio de ventas en valor por Cluster
# -----
```

```
plt.figure(figsize=(7, 5))
```

```
sns.barplot(
    data=df_resultados,
    x='Cluster',
    y='TotalValueSales',
    estimator='mean',
    errorbar=None
)
```

```
plt.title('Promedio de ventas en valor por Cluster')
plt.xlabel('Cluster')
plt.ylabel('Ventas promedio en valor')
plt.grid(axis='y')
```

```
plt.show()
```

```
# -----
# 3. Promedio de unidades vendidas por Cluster
# -----
```

```
plt.figure(figsize=(7, 5))
```

```
sns.barplot(
    data=df_resultados,
    x='Cluster',
    y='TotalUnitSales',
    estimator='mean',
    errorbar=None
)
```

```
plt.title('Promedio de unidades vendidas por Cluster')
plt.xlabel('Cluster')
plt.ylabel('Unidades vendidas promedio')
plt.grid(axis='y')
```

```
plt.show()
```

```
# -----
# 4. Distribución de las ventas promedio semanales
# -----
```

```
plt.figure(figsize=(8, 5))
```

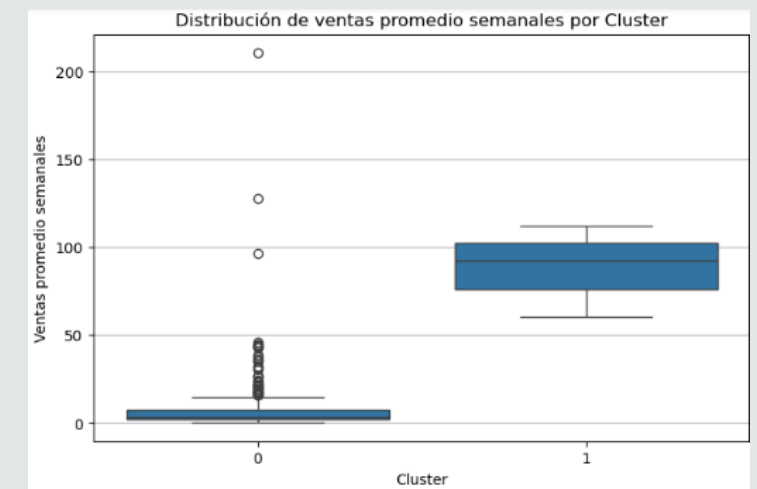
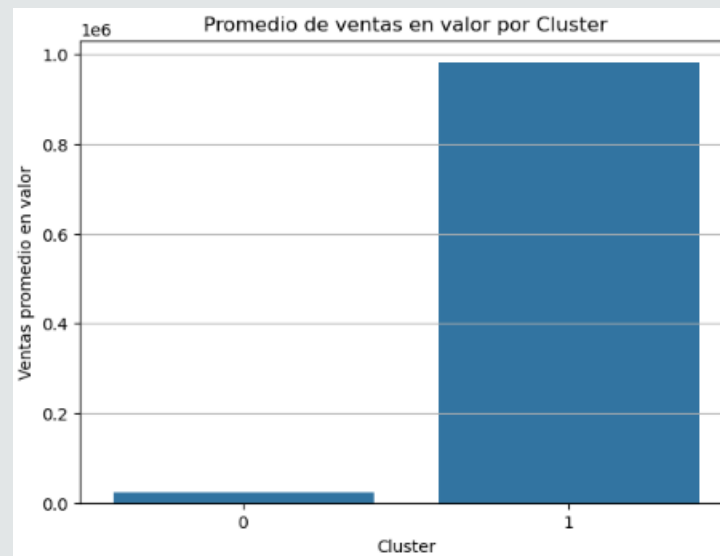
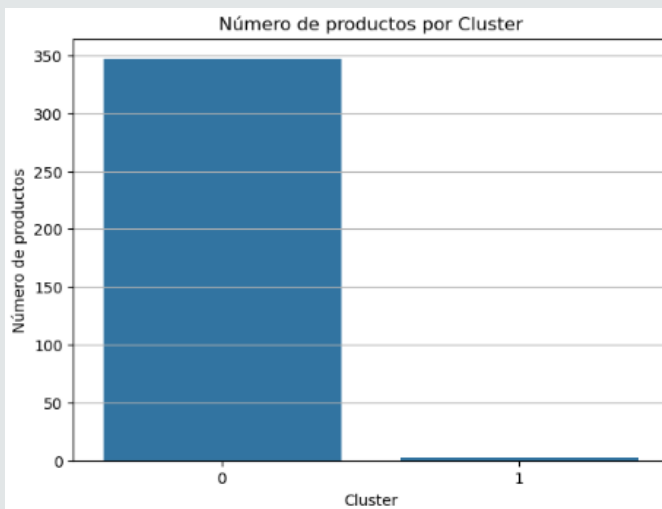
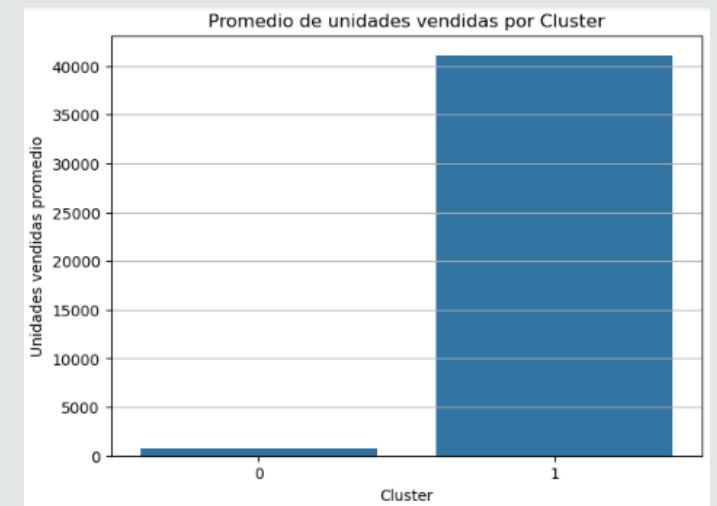
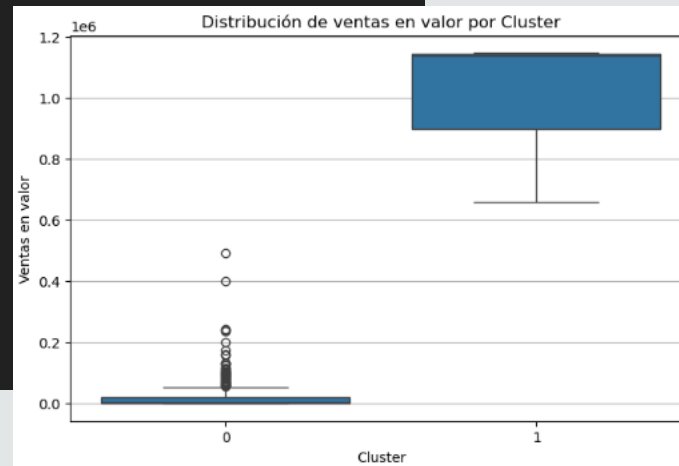
```
sns.boxplot(
    data=df_resultados,
    x='Cluster',
    y='AvgWeeklySales'
)
```

```
plt.title('Distribución de ventas promedio semanales por Cluster')
plt.xlabel('Cluster')
plt.ylabel('Ventas promedio semanales')
plt.grid(axis='y')
```

```
plt.show()
```

4.4 – VISUALIZACIÓN DE CLUSTERS

```
# -----  
# 5. Distribución de las ventas en valor  
# -----  
  
plt.figure(figsize=(8, 5))  
  
sns.boxplot(  
    data=df_resultados,  
    x='Cluster',  
    y='TotalValueSales'  
)  
  
plt.title('Distribución de ventas en valor por Cluster')  
plt.xlabel('Cluster')  
plt.ylabel('Ventas en valor')  
plt.grid(axis='y')  
plt.show()
```



5 – ANÁLISIS DE DATOS CON SQL



5.1 CREACIÓN DE LA BASE DE DATOS

- Crear una nueva base de datos en SSMS para almacenar todas las tablas de datos proporcionadas
- Definir la estructura de las tablas necesarias para los conjuntos de datos (DIM_CATEGORY, DIM_PRODUCT, DIM_SEGMENT, DIM_CALENDAR, FACT_SALES)

```
1 ----- a) Base de datos del proyecto -----
2 IF DB_ID('EmpresaAliadaDW') IS NULL
3     CREATE DATABASE EmpresaAliadaDW;
4 GO
5 USE EmpresaAliadaDW;
6 GO
7
8 IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='dim') EXEC('CREATE SCHEMA dim');
9 IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='fact') EXEC('CREATE SCHEMA fact');
10 GO
11
```

```
13 ----- b) Estructura de tablas -----
14 /* 1) DIM_CATEGORY */
15 IF OBJECT_ID('dim.DIM_CATEGORY') IS NULL
16 CREATE TABLE dim.DIM_CATEGORY(
17     CategoryKey INT IDENTITY(1,1) PRIMARY KEY,
18     CategoryID INT NOT NULL,
19     CategoryName NVARCHAR(150) NOT NULL,
20     IsActive BIT NOT NULL DEFAULT(1),
21     LoadDateUtc DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
22     CONSTRAINT UQ_DIM_CATEGORY_Business UNIQUE (CategoryID)
23 );
24 GO
```

```
26 /* 2) DIM_SEGMENT (p.ej.: Consumer / Corporate ... y opcionalmente Región) */
27 IF OBJECT_ID('dim.DIM_SEGMENT') IS NULL
28 CREATE TABLE dim.DIM_SEGMENT(
29     SegmentKey INT IDENTITY(1,1) PRIMARY KEY,
30     CategoryID INT NOT NULL,
31     Attr1 NVARCHAR(100) NULL,
32     Attr2 NVARCHAR(100) NULL,
33     Attr3 NVARCHAR(100) NULL,
34     [Format] NVARCHAR(50) NULL,
35     SegmentName NVARCHAR(150) NOT NULL,
36     IsActive BIT NOT NULL DEFAULT(1),
37     LoadDateUtc DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
38     CONSTRAINT FK_Segment_CategoryID
39         FOREIGN KEY (CategoryID) REFERENCES dim.DIM_CATEGORY(CategoryID),
40     -- Evitar duplicados del mismo "cubo" de atributos:
41     CONSTRAINT UQ_DIM_SEGMENT_Grain UNIQUE (CategoryID, Attr1, Attr2, Attr3, [Format])
42 );
43 GO
```

5.1 CREACIÓN DE LA BASE DE DATOS

```
46  /* 3) DIM_PRODUCT */
47  IF OBJECT_ID('dim.DIM_PRODUCT') IS NULL
48  CREATE TABLE dim.DIM_PRODUCT(
49      ProductKey      INT IDENTITY(1,1) PRIMARY KEY,
50      ItemCode         NVARCHAR(50) NOT NULL,
51      ItemDescription  NVARCHAR(250) NOT NULL,
52      Manufacturer     NVARCHAR(120) NULL,
53      Brand            NVARCHAR(120) NULL,
54      CategoryID       INT NOT NULL,
55      [Format]         NVARCHAR(50) NULL,
56      Attr1            NVARCHAR(100) NULL,
57      Attr2            NVARCHAR(100) NULL,
58      Attr3            NVARCHAR(100) NULL,
59
60      IsActive         BIT NOT NULL DEFAULT(1),
61      LoadDateUtc      DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
62      CONSTRAINT UQ_DIM_PRODUCT_Item UNIQUE (ItemCode),
63      CONSTRAINT FK_Product_CategoryID
64          FOREIGN KEY (CategoryID) REFERENCES dim.DIM_CATEGORY(CategoryID)
65  );
66  GO
67
```

```
81  /* 4) DIM_CALENDAR (clave YYYYMMDD) */
82  CREATE TABLE dim.DIM_CALENDAR(
83      DateKey          INT NOT NULL PRIMARY KEY,
84      FullDate         DATE NOT NULL,
85      WeekCode         NVARCHAR(10) NOT NULL,
86      [Year]           SMALLINT NOT NULL,
87      [Month]          TINYINT NOT NULL,
88      WeekNumber       TINYINT NOT NULL,
89      MonthName        NVARCHAR(20) NULL,
90      DayOfWeek        TINYINT NULL,
91      LoadDateUtc     DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
92      CONSTRAINT UQ_DIM_CALENDAR_Week UNIQUE (WeekCode)
93  );
94  GO
95
```

```
97  /* 5) FACT_SALES */
98  CREATE TABLE fact.FACT_SALES(
99      SalesKey          BIGINT IDENTITY(1,1) PRIMARY KEY,
100     ProductKey         INT NOT NULL,
101     WeekCode           NVARCHAR(10) NOT NULL,
102     Region             NVARCHAR(120) NOT NULL,
103     TotalUnitSales     DECIMAL(18,4) NULL,
104     TotalValueSales    DECIMAL(18,4) NULL,
105     TotalUnitAvgWeeklySales DECIMAL(18,4) NULL,
106     LoadDateUtc       DATETIME2(3) NOT NULL DEFAULT SYSUTCDATETIME(),
107     CONSTRAINT FK_FACT_Product FOREIGN KEY (ProductKey) REFERENCES dim.DIM_PRODUCT(ProductKey),
108     CONSTRAINT FK_FACT_Week   FOREIGN KEY (WeekCode) REFERENCES dim.DIM_CALENDAR(WeekCode)
109 );
110 GO
111
112 CREATE INDEX IX_FACT_SALES_Week ON fact.FACT_SALES(WeekCode);
113 CREATE INDEX IX_FACT_SALES_Product ON fact.FACT_SALES(ProductKey);
114 CREATE INDEX IX_FACT_SALES_Region ON fact.FACT_SALES(Region);
115
```

5.1 CREACIÓN DE LA BASE DE DATOS

- Importar los datos en cada tabla utilizando las funciones de importación

```
120 ----- c) Importación + Carga -----
121 USE EmpresaAliadaDW;
122 SET NOCOUNT ON;
123
124 IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='stg') EXEC('CREATE SCHEMA stg');
125
126 IF OBJECT_ID('stg.DIM_CATEGORY') IS NOT NULL DROP TABLE stg.DIM_CATEGORY;
127 IF OBJECT_ID('stg.DIM_PRODUCT') IS NOT NULL DROP TABLE stg.DIM_PRODUCT;
128 IF OBJECT_ID('stg.DIM_SEGMENT') IS NOT NULL DROP TABLE stg.DIM_SEGMENT;
129 IF OBJECT_ID('stg.DIM_CALENDAR') IS NOT NULL DROP TABLE stg.DIM_CALENDAR;
130 IF OBJECT_ID('stg.FACT_SALES') IS NOT NULL DROP TABLE stg.FACT_SALES;
131
```

```
132 -- 1) Tablas STAGING (tipos flexibles)
133 CREATE TABLE stg.DIM_CATEGORY(
134     ID_CATEGORY NVARCHAR(50) NULL,
135     CATEGORY NVARCHAR(200) NULL
136 );
137
138 CREATE TABLE stg.DIM_PRODUCT(
139     MANUFACTURER NVARCHAR(200) NULL,
140     BRAND NVARCHAR(200) NULL,
141     ITEM NVARCHAR(100) NULL,
142     ITEM_DESCRIPTION NVARCHAR(300) NULL,
143     CATEGORY NVARCHAR(50) NULL,
144     FORMAT NVARCHAR(100) NULL,
145     ATTR1 NVARCHAR(100) NULL,
146     ATTR2 NVARCHAR(100) NULL,
147     ATTR3 NVARCHAR(100) NULL
148 );
149
150 CREATE TABLE stg.DIM_SEGMENT(
151     CATEGORY NVARCHAR(50) NULL,
152     ATTR1 NVARCHAR(100) NULL,
153     ATTR2 NVARCHAR(100) NULL,
154     ATTR3 NVARCHAR(100) NULL,
155     FORMAT NVARCHAR(100) NULL,
156     SEGMENT NVARCHAR(150) NULL
157 );
158
159 CREATE TABLE stg.DIM_CALENDAR(
160     [WEEK] NVARCHAR(20) NULL,
161     [YEAR] NVARCHAR(10) NULL,
162     [MONTH] NVARCHAR(10) NULL,
163     [WEEK_NUMBER] NVARCHAR(10) NULL,
164     [DATE] NVARCHAR(50) NULL
165 );
166
167
168 CREATE TABLE stg.FACT_SALES(
169     [WEEK] NVARCHAR(20) NULL,
170     [ITEM_CODE] NVARCHAR(100) NULL,
171     [TOTAL_UNIT_SALES] NVARCHAR(100) NULL,
172     [TOTAL_VALUE_SALES] NVARCHAR(100) NULL,
173     [TOTAL_UNIT_AVG_WEEKLY_SALES] NVARCHAR(100) NULL,
174     [REGION] NVARCHAR(200) NULL
175 );
```

5.1 CREACIÓN DE LA BASE DE DATOS

- Los archivos Excel se hacen CSV

```
204  /* 3) CARGA a DIMENSIONES */
205  -- DIM_CATEGORY
206  ;WITH C AS (
207      SELECT DISTINCT
208          CategoryID = TRY_CONVERT(INT, LTRIM(RTRIM(ID_CATEGORY))),
209          CategoryName = LTRIM(RTRIM(CATEGORY))
210      FROM stg.DIM_CATEGORY
211  )
212  INSERT INTO dim.DIM_CATEGORY (CategoryID, CategoryName)
213  SELECT C.CategoryID, C.CategoryName
214  FROM C
215  WHERE C.CategoryID IS NOT NULL
216  AND NOT EXISTS (
217      SELECT 1
218      FROM dim.DIM_CATEGORY D
219      WHERE D.CategoryID = C.CategoryID
220  );
```

```
178  -- 2) CSV
179  -- DIM_CATEGORY.csv
180  BULK INSERT stg.DIM_CATEGORY
181  FROM 'C:\Data\EmpresaAliada\DIM_CATEGORY.csv'
182  WITH (FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0d0a', CODEPAGE='65001', FIELDQUOTE='"', TABLOCK);
183
184  -- FACT_SALES.csv
185  BULK INSERT stg.FACT_SALES
186  FROM 'C:\Data\EmpresaAliada\FACT_SALES.csv'
187  WITH (FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0d0a', CODEPAGE='65001', FIELDQUOTE='"', TABLOCK);
188
189  -- DIM_PRODUCT.csv
190  BULK INSERT stg.DIM_PRODUCT
191  FROM 'C:\Data\EmpresaAliada\Copia de DIM_PRODUCT.csv'
192  WITH (FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0d0a', CODEPAGE='65001', FIELDQUOTE='"', TABLOCK);
193
194  -- DIM_SEGMENT.csv
195  BULK INSERT stg.DIM_SEGMENT
196  FROM 'C:\Data\EmpresaAliada\Copia de DIM_SEGMENT.csv'
197  WITH (FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0d0a', CODEPAGE='65001', FIELDQUOTE='"', TABLOCK);
198
199  -- DIM_CALENDAR.csv
200  BULK INSERT stg.DIM_CALENDAR
201  FROM 'C:\Data\EmpresaAliada\Copia de DIM_CALENDAR.csv'
202  WITH (FIRSTROW=2, FIELDTERMINATOR=',', ROWTERMINATOR='0x0d0a', CODEPAGE='65001', FIELDQUOTE='"', TABLOCK);
203
```

5.1 CREACIÓN DE LA BASE DE DATOS

```
221 -- DIM_SEGMENT
222
223 ;WITH S AS (
224     SELECT
225         CategoryID = TRY_CONVERT(INT, LTRIM(RTRIM(S.CATEGORY))),
226         Attr1      = NULLIF(LTRIM(RTRIM(S.ATTR1)), N''),
227         Attr2      = NULLIF(LTRIM(RTRIM(S.ATTR2)), N''),
228         Attr3      = NULLIF(LTRIM(RTRIM(S.ATTR3)), N''),
229         [Format]   = NULLIF(LTRIM(RTRIM(S.FORMAT)), N''),
230         Segment    = LTRIM(RTRIM(S.SEGMENT))
231     FROM stg.DIM_SEGMENT S
232 ),
233
234 G AS (
235     SELECT
236         CategoryID, Attr1, Attr2, Attr3, [Format],
237         SegmentName = MIN(Segment)
238     FROM S
239     WHERE CategoryID IS NOT NULL
240     GROUP BY CategoryID, Attr1, Attr2, Attr3, [Format]
241 )
242 INSERT INTO dim.DIM_SEGMENT (CategoryID, Attr1, Attr2, Attr3, [Format], SegmentName)
243 SELECT g.CategoryID, g.Attr1, g.Attr2, g.Attr3, g.[Format], g.SegmentName
244 FROM G g
245 WHERE NOT EXISTS (
246     SELECT 1
247     FROM dim.DIM_SEGMENT d
248     WHERE d.CategoryID = g.CategoryID
249         AND ISNULL(d.Attr1, '') = ISNULL(g.Attr1, '')
250         AND ISNULL(d.Attr2, '') = ISNULL(g.Attr2, '')
251         AND ISNULL(d.Attr3, '') = ISNULL(g.Attr3, '')
252         AND ISNULL(d.[Format], '') = ISNULL(g.[Format], '')
253 );
254
255
```

```
-- DIM_PRODUCT
;WITH P0 AS (
    SELECT DISTINCT
        ItemCode      = NULLIF(LTRIM(RTRIM(ITEM)), N''),
        ItemDescription= LTRIM(RTRIM(ITEM_DESCRIPTION)),
        Manufacturer   = NULLIF(LTRIM(RTRIM(MANUFACTURER)), N''),
        Brand          = NULLIF(LTRIM(RTRIM(BRAND)), N''),
        CategoryID     = TRY_CONVERT(INT, LTRIM(RTRIM(CATEGORY))),
        [Format]       = NULLIF(LTRIM(RTRIM([FORMAT])), N''),
        Attr1          = NULLIF(LTRIM(RTRIM(ATTR1)), N''),
        Attr2          = NULLIF(LTRIM(RTRIM(ATTR2)), N''),
        Attr3          = NULLIF(LTRIM(RTRIM(ATTR3)), N'')
    FROM stg.DIM_PRODUCT
),
P1 AS ( -- descarta códigos inválidos
    SELECT *
    FROM P0
    WHERE ItemCode IS NOT NULL
        AND UPPER(ItemCode) NOT IN (N'N/A', N'NA', N'NO DEFINIDO')
        AND CategoryID IS NOT NULL
),
P2 AS ( -- una fila por ItemCode
    SELECT *,
        ROW_NUMBER() OVER (PARTITION BY ItemCode ORDER BY ItemDescription) AS rn
    FROM P1
)
INSERT INTO dim.DIM_PRODUCT
(ItemCode, ItemDescription, Manufacturer, Brand, CategoryID, [Format], Attr1, Attr2, Attr3)
SELECT
    p.ItemCode, p.ItemDescription, p.Manufacturer, p.Brand, p.CategoryID, p.[Format], p.Attr1, p.Attr2, p.Attr3
FROM P2 p
LEFT JOIN dim.DIM_PRODUCT d
    ON d.ItemCode = p.ItemCode
WHERE p.rn = 1
    AND d.ProductKey IS NULL;
```

5.1 CREACIÓN DE LA BASE DE DATOS

```
-- DIM_CALENDAR
DELETE d
FROM dim.DIM_CALENDAR d
JOIN (SELECT DISTINCT REPLACE(REPLACE(REPLACE(LTRIM(RTRIM([WEEK])), ' ', ''), CHAR(9), ''), CHAR(160), '' ) AS WeekClean
      FROM stg.DIM_CALENDAR) s
ON s.WeekClean = d.WeekCode;

UPDATE s
SET [WEEK] = REPLACE(REPLACE(REPLACE(LTRIM(RTRIM([WEEK])), ' ', ''), CHAR(9), ''), CHAR(160), '' )
FROM stg.FACT_SALES s;

;WITH X AS (
    SELECT
        WeekClean = REPLACE(REPLACE(REPLACE(LTRIM(RTRIM([WEEK])), ' ', ''), CHAR(9), ''), CHAR(160), '' ),
        [Year] = TRY_CONVERT(SMALLINT, [YEAR]),
        [Month] = TRY_CONVERT(TINYINT, [MONTH]),
        WeekNumber = TRY_CONVERT(TINYINT, [WEEK_NUMBER]),
        FullDate = CASE
            WHEN TRY_CONVERT(FLOAT, [DATE]) IS NOT NULL
            THEN DATEADD(DAY,
                CONVERT(INT, FLOOR(CONVERT(FLOAT, [DATE]))),
                DATEFROMPARTS(1899, 12, 30))
            ELSE COALESCE(
                TRY_CONVERT(date, [DATE], 103),
                TRY_CONVERT(date, [DATE], 101),
                TRY_CONVERT(date, [DATE])
            )
        END
    FROM stg.DIM_CALENDAR
),
R AS (
    SELECT
        DateKey = TRY_CONVERT(INT, FORMAT(FullDate, 'yyyyMMdd')),
        FullDate,
        WeekCode = WeekClean,
        [Year], [Month], WeekNumber
    FROM X
    WHERE FullDate IS NOT NULL
    AND WeekClean IS NOT NULL AND WeekClean <> N''
),
G AS ( -- dedup por DateKey
```

```
SELECT
    DateKey,
    FullDate = MIN(FullDate),
    WeekCode = MIN(WeekCode),
    [Year] = MIN([Year]),
    [Month] = MIN([Month]),
    WeekNumber = MIN(WeekNumber)
FROM R
GROUP BY DateKey
)
INSERT INTO dim.DIM_CALENDAR
(DateKey, FullDate, WeekCode, [Year], [Month], WeekNumber, MonthName, DayOfWeek)
SELECT
    g.DateKey, g.FullDate, g.WeekCode, g.[Year], g.[Month], g.WeekNumber,
    DATENAME(MONTH, g.FullDate), DATEPART(WEEKDAY, g.FullDate)
FROM G g
WHERE NOT EXISTS (SELECT 1 FROM dim.DIM_CALENDAR d WHERE d.DateKey = g.DateKey);
```

```
357 /* 4) CARGA a FACT */
358 TRUNCATE TABLE fact.FACT_SALES;
359 GO
360
361 ;WITH P AS ( -- ItemDigits desde DIM_PRODUCT
362     SELECT
363         d.ProductKey,
364         ItemDigits =
365         (
366             SELECT ' ' + SUBSTRING(d.ItemCode, v.number, 1)
367             FROM master..spt_values v
368             WHERE v.type = 'P'
369             AND v.number BETWEEN 1 AND LEN(d.ItemCode)
370             AND SUBSTRING(d.ItemCode, v.number, 1) LIKE '[0-9]'
371             ORDER BY v.number
372             FOR XML PATH(''), TYPE
373         ).value(' ','nvarchar(100)')
374     FROM dim.DIM_PRODUCT d
375     WHERE d.ItemCode IS NOT NULL
376 ),
377 F0 AS ( -- limpia WEEK / ITEM_CODE / REGION y prepara textos numéricos
378     SELECT
379         WeekClean = REPLACE(REPLACE(REPLACE(LTRIM(RTRIM(s.[WEEK])), ' ', ''), CHAR(9), ''), CHAR(160), '' ),
380         ItemClean = REPLACE(REPLACE(REPLACE(LTRIM(RTRIM(s.[ITEM_CODE])), ' ', ''), CHAR(9), ''), CHAR(160), '' ),
381         Region = LTRIM(RTRIM(s.[REGION])),
382
383         Raw_TUS = s.[TOTAL_UNIT_SALES],
384         Raw_TVS = s.[TOTAL_VALUE_SALES],
385         Raw_TUAW = s.[TOTAL_UNIT_AVG_WEEKLY_SALES]
386     FROM stg.FACT_SALES s
387 ),
388 F0C AS ( -- quita comillas/espacios, coma->punto
389     SELECT
390         WeekClean, ItemClean, Region,
391         Clean_TUS = NULLIF(REPLACE(REPLACE(REPLACE(REPLACE(Raw_TUS, ' ', ''), ' ', ''), CHAR(160), ''), ','), ''),
392         Clean_TVS = NULLIF(REPLACE(REPLACE(REPLACE(REPLACE(Raw_TVS, ' ', ''), ' ', ''), CHAR(160), ''), ','), ''),
393         Clean_TUAW = NULLIF(REPLACE(REPLACE(REPLACE(REPLACE(Raw_TUAW, ' ', ''), ' ', ''), CHAR(160), ''), ','), ''),
394     FROM F0
395 )
```

5.1 CREACIÓN DE LA BASE DE DATOS

```
396 F1 AS ( -- convierte a DECIMAL y extrae dígitos de Item
397     SELECT
398         WeekClean,
399         Region,
400         TotalUnitSales = TRY_CONVERT(DECIMAL(18,4), Clean_TUS),
401         TotalValueSales = TRY_CONVERT(DECIMAL(18,4), Clean_TVS),
402         TotalUnitAvgWeeklySales = TRY_CONVERT(DECIMAL(18,4), Clean_TUAW),
403         ItemDigits =
404         (
405             SELECT '' + SUBSTRING(f.ItemClean, v.number, 1)
406             FROM master..spt_values v
407             WHERE v.type = 'P'
408                   AND v.number BETWEEN 1 AND LEN(f.ItemClean)
409                   AND SUBSTRING(f.ItemClean, v.number, 1) LIKE '[0-9]'
410             ORDER BY v.number
411             FOR XML PATH(''), TYPE
412             ).value('.', 'nvarchar(100)')
413     FROM FOC f
414     WHERE f.ItemClean IS NOT NULL
415           AND f.WeekClean IS NOT NULL
416           AND f.Region IS NOT NULL
417 ),
418 F2 AS ( -- resuelve surrogate keys
419     SELECT
420         p.ProductKey,
421         c.WeekCode,
422         f.Region,
423         f.TotalUnitSales,
424         f.TotalValueSales,
425         f.TotalUnitAvgWeeklySales
426     FROM F1 f
427     INNER JOIN P          ON p.ItemDigits = f.ItemDigits
428     INNER JOIN dim.DIM_CALENDAR c ON c.WeekCode = f.WeekClean
429 )
430 INSERT INTO fact.FACT_SALES
431 (ProductKey, WeekCode, Region, TotalUnitSales, TotalValueSales, TotalUnitAvgWeeklySales)
432 SELECT
433     t.ProductKey, t.WeekCode, t.Region,
434     t.TotalUnitSales, t.TotalValueSales, t.TotalUnitAvgWeeklySales
435 FROM F2 t;
```

5.2 – CONSULTAS CLAVE

- Realizar consultas básicas para verificar que los datos se han cargado correctamente

```
437 ----- d) Validaciones Rápidas -----
438 SET NOCOUNT ON;
439
440 -- Conteos por tabla
441 SELECT 'DIM_CATEGORY' AS Tabla, COUNT(*) AS Regs FROM dim.DIM_CATEGORY
442 UNION ALL SELECT 'DIM_PRODUCT', COUNT(*) FROM dim.DIM_PRODUCT
443 UNION ALL SELECT 'DIM_SEGMENT', COUNT(*) FROM dim.DIM_SEGMENT
444 UNION ALL SELECT 'DIM_CALENDAR', COUNT(*) FROM dim.DIM_CALENDAR
445 UNION ALL SELECT 'FACT_SALES', COUNT(*) FROM fact.FACT_SALES;
```

	Tabla	Regs
1	DIM_CATEGORY	4
2	DIM_PRODUCT	503
3	DIM_SEGMENT	53
4	DIM_CALENDAR	156
5	FACT_SALES	113479

```
-- Muestras de cada tabla (para screenshots)
SELECT TOP (10) * FROM dim.DIM_CATEGORY ORDER BY CategoryID;
SELECT TOP (10) * FROM dim.DIM_PRODUCT ORDER BY ProductKey;
SELECT TOP (10) * FROM dim.DIM_SEGMENT ORDER BY SegmentKey;
SELECT TOP (10) * FROM dim.DIM_CALENDAR ORDER BY FullDate;
SELECT TOP (10) * FROM fact.FACT_SALES ORDER BY SalesKey;
```

CategoryKey	CategoryID	CategoryName	IsActive	LoadDateUtc
1	1	FABRIC TREATMENT and SANIT	1	2025-10-07 04:03:06.568
2	3	LAVAJAILLAS	1	2025-10-07 04:03:06.568
3	4	MEGA SUPERFICIES	1	2025-10-07 04:03:06.568
4	5	LAVATORY CARE & BRC	1	2025-10-07 04:03:06.568

ProductKey	ItemCode	ItemDescription	Manufacturer	Brand	CategoryID	Format	Atr1	Atr2	Atr3	IsActive	LoadDateUtc
211	0757037514016BP1	90XI CLEAN ELIMINADOR DIMANCHAS BOTE PLAST. 1580 ...	CHURCH & DWIGHT	OXI CLEAN	1	POLVO	SAFE BLEACH	FABRIC TREATMENT	QUITAMANCHAS	1	2025-10-08 02:11:13.442
212	123456808218	CLORALEX REGULAR BOT 950 ML NAL 0123456808218	INDS. ALEN	CLORALEX	1	LIQUIDO	COLORO	COLORO	NO DEFINIDO	1	2025-10-08 02:11:13.442
213	19200958714	LYSOL SANITIZANTE DE ROPA CRISP LINEN BOT 1.2 LT 001...	RECKITT	LYSOL	1	LIQUIDO	SANITIZANTE	SANITIZANTE	SANITIZANTE	1	2025-10-08 02:11:13.442
214	3045200760009	SANYTOL DESINFECTANTE DE ROPA BOT 500 ML 3045200...	GRUPO AC MARCAS	SANYTOL	1	LIQUIDO	SANITIZANTE	SANITIZANTE	SANITIZANTE	1	2025-10-08 02:11:13.442
215	3045206360609	SANYTOL DESINFECTANTE DE ROPA BOT 1 LT 3045206360...	GRUPO AC MARCAS	SANYTOL	1	LIQUIDO	SANITIZANTE	SANITIZANTE	SANITIZANTE	1	2025-10-08 02:11:13.442
216	3045206382007	SANYTOL QUITAMANCHAS DESINFECTANTE POLVO BOT 45...	GRUPO AC MARCAS	SANYTOL	1	POLVO	SAFE BLEACH	FABRIC TREATMENT	PRE LAVADOR	1	2025-10-08 02:11:13.442
217	37000018704	9TIDE LUPADOR DE ROPA QUITAMANCHAS PLUMON 10 ML...	P&G	TIDE	1	LIQUIDO	SAFE BLEACH	FABRIC TREATMENT	PRE LAVADOR	1	2025-10-08 02:11:13.442
218	37000213215	9TIDE ADMITO QUITAMANCHAS 220 GR 0037000213215	P&G	TIDE	1	POLVO	SAFE BLEACH	FABRIC TREATMENT	OTR. TIPOS	1	2025-10-08 02:11:13.442

SegmentKey	CategoryID	Atr1	Atr2	Atr3	Format	SegmentName	IsActive	LoadDateUtc
1	1	COLORO	COLORO	NULL	LIQUIDO	BLEACH	1	2025-10-07 04:03:06.570
2	1	COLORO	COLORO	BAMBINO	LIQUIDO	BLEACH	1	2025-10-07 04:03:06.570
3	1	COLORO	COLORO	GERMIL	LIQUIDO	BLEACH	1	2025-10-07 04:03:06.570
4	1	COLORO	COLORO	MASCO	LIQUIDO	BLEACH	1	2025-10-07 04:03:06.570
5	1	COLORO	COLORO	MULTI...	GEL	BLEACH	1	2025-10-07 04:03:06.570
6	1	COLORO	COLORO	MULTI...	LIQUIDO	BLEACH	1	2025-10-07 04:03:06.570
7	1	COLORO	COLORO	NO DEF...	LIQUIDO	BLEACH	1	2025-10-07 04:03:06.570
8	1	COLORO	COLORO	NO DEF...	POLVO	BLEACH	1	2025-10-07 04:03:06.570

DateKey	FullDate	WeekCode	Year	Month	WeekNumber	MonthName	DayOfWeek	LoadDateUtc
20210110	2021-01-10	01-21	2021	1	1	Enero	7	2025-10-14 02:00:28.515
20210117	2021-01-17	02-21	2021	1	2	Enero	7	2025-10-14 02:00:28.515
20210124	2021-01-24	03-21	2021	1	3	Enero	7	2025-10-14 02:00:28.515
20210131	2021-01-31	04-21	2021	1	4	Enero	7	2025-10-14 02:00:28.515
20210207	2021-02-07	05-21	2021	2	5	Febrero	7	2025-10-14 02:00:28.515
20210214	2021-02-14	06-21	2021	2	6	Febrero	7	2025-10-14 02:00:28.515
20210221	2021-02-21	07-21	2021	2	7	Febrero	7	2025-10-14 02:00:28.515
20210228	2021-02-28	08-21	2021	2	8	Febrero	7	2025-10-14 02:00:28.515

SalesKey	ProductKey	WeekCode	Region	TotalUnitSales	TotalValueSales	TotalUnitAvgWeeklySales	LoadDateUtc
1	214	39-22	TOTAL AUTOS AREA 4	0.1710	10.1950	3.0000	2025-10-14 03:38:42.217
2	214	28-22	TOTAL AUTOS SCANNING MEXICO	1.2700	67.7350	3.3870	2025-10-14 03:38:42.217
3	214	40-22	TOTAL AUTOS SCANNING MEXICO	0.8590	51.3820	2.5800	2025-10-14 03:38:42.217
4	214	13-22	TOTAL AUTOS AREA 2	0.0800	3.8940	2.2860	2025-10-14 03:38:42.217
5	214	15-22	TOTAL AUTOS AREA 5	0.6480	27.0570	5.0630	2025-10-14 03:38:42.217
6	214	04-22	TOTAL AUTOS SCANNING MEXICO	1.7730	87.4720	4.1910	2025-10-14 03:38:42.217
7	214	27-23	TOTAL AUTOS AREA 1	0.0560	3.1480	2.5450	2025-10-14 03:38:42.217
8	214	30-22	TOTAL AUTOS AREA 5	0.5700	30.4890	4.6720	2025-10-14 03:38:42.217
9	214	27-23	TOTAL AUTOS AREA 2	0.0570	2.8250	1.8390	2025-10-14 03:38:42.217
10	214	19-23	TOTAL AUTOS AREA 1	0.0410	2.2620	1.8640	2025-10-14 03:38:42.217

5.2 – CONSULTAS CLAVE

```
-- Integridad referencial efectiva (huérfanos = 0)
SELECT
    SUM(CASE WHEN p.ProductKey IS NULL THEN 1 ELSE 0 END) AS Orphans_Product,
    SUM(CASE WHEN c.WeekCode IS NULL THEN 1 ELSE 0 END) AS Orphans_WeekCode
FROM fact.FACT_SALES f
LEFT JOIN dim.DIM_PRODUCT p ON p.ProductKey = f.ProductKey
LEFT JOIN dim.DIM_CALENDAR c ON c.WeekCode = f.WeekCode;

-- Nulidades en medidas (deben ser 0)
SELECT
    SUM(CASE WHEN TotalUnitSales IS NULL THEN 1 ELSE 0 END) AS Null_TUS,
    SUM(CASE WHEN TotalValueSales IS NULL THEN 1 ELSE 0 END) AS Null_TVS,
    SUM(CASE WHEN TotalUnitAvgWeeklySales IS NULL THEN 1 ELSE 0 END) AS Null_TUAW
FROM fact.FACT_SALES;
```

```
-- Rango de fechas y semanas (calendario y fact)
SELECT MIN(FullDate) AS MinDate, MAX(FullDate) AS MaxDate, COUNT(*) AS Weeks
FROM dim.DIM_CALENDAR;

SELECT MIN(WeekCode) AS MinWeekFact, MAX(WeekCode) AS MaxWeekFact, COUNT(DISTINCT WeekCode) AS WeeksInFact
FROM fact.FACT_SALES;
```

100 % No se encontraron problemas.

	Orphans_Product	Orphans_WeekCode
1	0	0

	Null_TUS	Null_TVS	Null_TUAW
1	0	0	0

	MinDate	MaxDate	Weeks
1	2021-01-10	2024-01-01	156

	MinWeekFact	MaxWeekFact	WeeksInFact
1	01-22	52-22	80

```
-- Perfil rápido (distribución por semana y región)
SELECT TOP (10) WeekCode, COUNT(*) AS Rows,
    SUM(TotalUnitSales) AS SumTUS,
    SUM(TotalValueSales) AS SumTVS
FROM fact.FACT_SALES
GROUP BY WeekCode
ORDER BY WeekCode;

SELECT TOP (10) Region, COUNT(*) AS Rows,
    SUM(TotalUnitSales) AS SumTUS,
    SUM(TotalValueSales) AS SumTVS
FROM fact.FACT_SALES
GROUP BY Region
ORDER BY Region;
```

	WeekCode	Rows	SumTUS	SumTVS
1	01-22	1476	3762.5900	108938.5400
2	01-23	1361	3626.7320	118071.8990
3	02-22	1489	3777.5140	105724.1780
4	02-23	1367	3469.7380	111680.0920
5	03-22	1475	3177.5820	89708.0100
6	03-23	1358	3040.1000	97816.3250
7	04-22	1474	3144.3680	88814.8700
8	04-23	1380	2811.6210	90379.9530

	Region	Rows	SumTUS	SumTVS
1	TOTAL AUTOS AREA 1	14751	16226.8190	493982.1980
2	TOTAL AUTOS AREA 2	17267	29345.4400	840022.1500
3	TOTAL AUTOS AREA 3	15381	17830.9720	588760.4940
4	TOTAL AUTOS AREA 4	16196	15338.6810	469544.5380
5	TOTAL AUTOS AREA 5	15253	27880.8050	929389.2560
6	TOTAL AUTOS AREA 6	15603	21855.8820	640983.6450
7	TOTAL AUTOS SCAN...	19028	128478.5...	3962681.1...

5.2 – CONSULTAS CLAVE

- Ejecutar uniones entre las tablas para combinar la información y obtener insights clave sobre las ventas por categoría, región y periodo de tiempo

```
-- e) Uniones entre tablas
-- Vista de trabajo (join una sola vez)
USE EmpresaAliadaDW;
GO

IF OBJECT_ID('dbo.vw_Sales', 'V') IS NOT NULL
    DROP VIEW dbo.vw_Sales;
GO

CREATE VIEW dbo.vw_Sales
AS
SELECT
    c.[Year],
    c.WeekCode,
    c.FullDate,
    f.Region,
    p.ProductKey,
    p.ItemCode,
    p.ItemDescription,
    p.Brand,
    p.Manufacturer,
    p.CategoryID,
    cat.CategoryName,
    f.TotalUnitSales,
    f.TotalValueSales,
    f.TotalUnitAvgWeeklySales
FROM fact.FACT_SALES AS f
JOIN dim.DIM_PRODUCT AS p ON p.ProductKey = f.ProductKey
JOIN dim.DIM_CATEGORY AS cat ON cat.CategoryID = p.CategoryID
JOIN dim.DIM_CALENDAR AS c ON c.WeekCode = f.WeekCode;
GO
```

```
-- Parámetros del análisis (ajústalos)
USE EmpresaAliadaDW;
SET NOCOUNT ON;

DECLARE @YearFrom smallint = 2021;
DECLARE @YearTo smallint = 2022;
DECLARE @RegionLike nvarchar(100) = NULL;

-- Visualizar el JOIN base
SELECT TOP (20)
    s.*
FROM dbo.vw_Sales AS s
ORDER BY s.FullDate, s.Region, s.ItemCode;
```

	Year	WeekCode	FullDate	Region	ProductKey	ItemCode	ItemDescription	Brand	Manufacturer	CategoryID	CategoryName	TotalUnitSales	TotalValueSales	TotalUnitAvgWeeklySales
1	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	214	3045200760009	SANYTOL DESINFECTANTE DE ROPA BOT 500 ML 3045200760009	SANYTOL	GRUPO AC MARCAS	1	"FABRIC TREATMENT and SANIT	0.1090	5.8550	2.7950
2	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	215	3045206360609	SANYTOL DESINFECTANTE DE ROPA BOT 1 LT 3045206360609	SANYTOL	GRUPO AC MARCAS	1	"FABRIC TREATMENT and SANIT	0.0170	1.5780	1.3080
3	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	216	3045206382007	SANYTOL QUITAMANCHAS DESINFECTANTE POLVO BOT 450GR	SANYTOL	GRUPO AC MARCAS	1	"FABRIC TREATMENT and SANIT	0.0300	2.1300	1.5000
4	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	221	4008455394015	DR BECKMANN EL MAGO QUITAMANCHAS 2 LIQUIDO BOT 50ML L	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0500	1.9130	1.8520
5	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	222	4008455394114	DR BECKMANN EL MAGO QUITAMANCHAS 4 LIQUIDO BOT 50ML L	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0950	3.5710	2.7140
6	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	223	4008455394213	DR BECKMANN EL MAGO QUITAMANCHAS 8 LIQUIDO BOT 50ML L	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0680	2.5770	2.1940
7	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	224	4008455394411	DR BECKMANN EL MAGO QUITAMANCHAS 3 P.TINT/CRAY/ROTUL	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0720	2.8700	2.3230
8	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	225	4008455394510	DR BECKMANN EL MAGO QUITAMANCHAS 5 LIQUIDO BOT 50ML L	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0680	2.7320	2.1940
9	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	226	4008455394619	DR BECKMANN EL MAGO QUITAMANCHAS 6 LIQUIDO BOT 50ML L	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0310	1.1950	1.7220
10	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	227	4008455423517	DR BECKMANN EL MAGO QUITAMANCHAS 4 LIQUIDO BOT 50ML L	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0460	1.7220	1.6430
11	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	228	4008455423616	DR BECKMANN EL MAGO QUITAMANCHAS 4 LIQUIDO BOT 50ML L	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0350	1.3580	1.5910
12	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	229	4008455423715	DR BECKMANN QUITAMANCHAS PRELAIVADO CICEP APLIC LIQ B	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0200	1.1810	1.3330
13	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	230	4008455425412	DR BECKMANN BLANQ INTENS SUPER BLANC PVO 6BSAS 40GR	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.3620	20.8840	5.0990
14	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	232	4008455438016	DR BECKMANN LAPIZ MAGICO QUITAMANCHAS 9ML 4008455438	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0720	4.0600	2.1180
15	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	233	4008455457512	DR BECKMANN BLANQUEADOR SIN CLORO BOL 80GR 40084554	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.4990	7.4180	8.9110
16	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	234	4008455457710	DR BECKMANN QUITAMANCHASINTENSOPABLANCAYCOLOR	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.3770	5.6370	7.6940
17	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	235	4008455458618	DR BECKMANN EL MAGO QUITAMANCHAS OXIDODESOD BOTC	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.1960	12.4980	3.4390
18	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	236	4008455458717	DR BECKMANN ORIGINAL QUITAMANCHAS PRELAIVADO BOT CA	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0340	2.5550	1.6190
19	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	237	4008455459417	DR BECKMAN SUPER BLANCO TOALLITAS ACTIVAS SICLORO CAJ	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.1510	9.6190	2.9510
20	2022	01-22	2022-01-09	TOTAL AUTOS AREA 1	238	4008455460215	DR BECKMANN BLANQUEADOR POLVO CJ C2BOLBAS 75GRDCA	DR BECKMANN	DELTA PRONATURA	1	"FABRIC TREATMENT and SANIT	0.0090	0.8150	1.5000

5.2 – CONSULTAS CLAVE

```
-- Ventas por categoría y año
SELECT
    [Year],
    CategoryID,
    CategoryName,
    SUM(TotalUnitSales) AS Units,
    SUM(TotalValueSales) AS Value,
    CAST(SUM(TotalValueSales)/NULLIF(SUM(TotalUnitSales),0) AS decimal(18,4)) AS AvgPrice
FROM dbo.vw_Sales
WHERE [Year] BETWEEN @YearFrom AND @YearTo
AND (@RegionLike IS NULL OR Region LIKE @RegionLike)
GROUP BY [Year], CategoryID, CategoryName
ORDER BY [Year], Value DESC;
```

	Year	CategoryID	CategoryName	Units	Value	AvgPrice
1	2022	1	*FABRIC TREATMENT and SANIT	169625.1170	5064630.7760	29.8578

```
-- Ventas por región y semana
SELECT
    [Year], WeekCode, Region,
    SUM(TotalUnitSales) AS Units,
    SUM(TotalValueSales) AS Value
FROM dbo.vw_Sales
WHERE [Year] BETWEEN @YearFrom AND @YearTo
AND (@RegionLike IS NULL OR Region LIKE @RegionLike)
GROUP BY [Year], WeekCode, Region
ORDER BY [Year], MIN(FullDate), Region;
```

	Year	WeekCode	Region	Units	Value
1	2022	01-22	TOTAL AUTOS AREA 1	235.8420	6700.5440
2	2022	01-22	TOTAL AUTOS AREA 2	427.5020	11412.2630
3	2022	01-22	TOTAL AUTOS AREA 3	251.7820	7822.0950
4	2022	01-22	TOTAL AUTOS AREA 4	225.0360	6509.8020
5	2022	01-22	TOTAL AUTOS AREA 5	422.9860	13159.8240
6	2022	01-22	TOTAL AUTOS AREA 6	318.1520	8864.7430
7	2022	01-22	TOTAL AUTOS SCAN...	1881.2...	54469.2690
8	2022	02-22	TOTAL AUTOS AREA 1	224.1050	6247.2170

```
-- Top 15 productos por valor
SELECT TOP (15)
    ItemCode, ItemDescription, Brand, CategoryName,
    SUM(TotalValueSales) AS Value,
    SUM(TotalUnitSales) AS Units
FROM dbo.vw_Sales
WHERE [Year] BETWEEN @YearFrom AND @YearTo
AND (@RegionLike IS NULL OR Region LIKE @RegionLike)
GROUP BY ItemCode, ItemDescription, Brand, CategoryName
ORDER BY Value DESC;
```

	ItemCode	ItemDescription	Brand	CategoryName	Value	Units
1	7501025402051	BLANCATEL CONCENTRADO BOT PLAST 3750ML NAL 7501025...	BLANCATEL	*FABRIC TREATMENT and SANIT	373008.6280	12326.9610
2	7501071900143	CLOROX REGULAR CONCENTRADO BOTPLAST 3800ML 75010...	CLOROX	*FABRIC TREATMENT and SANIT	218818.3440	5934.3890
3	7501025402012	CLORALEX EL RENDIDOR BOT 1900+100ML= 2LT 7501025450...	CLORALEX	*FABRIC TREATMENT and SANIT	159701.0570	6664.2160
4	7501025405090	CLORALEX ULTRAGEL CLORO GEL CONCENTRADO BOT 600 M...	CLORALEX	*FABRIC TREATMENT and SANIT	158175.8510	9841.5560
5	7501025405212	CLORALEX ULTRAGEL CLORO EN GEL CONCENTRADO 950ML ...	CLORALEX	*FABRIC TREATMENT and SANIT	147634.2000	6139.6570
6	7501025405151	CLORALEX CLORO EN GEL BOT 950ML 7501025405151	CLORALEX	*FABRIC TREATMENT and SANIT	122768.5530	6908.6350
7	7503002319529	VANISH LIQUIDO BOTELLA 925ML NAL 7503002319529	VANISH	*FABRIC TREATMENT and SANIT	104776.6290	1947.7090
8	7501058717023	VANISH ROSA QUITAMANCHAS GEL MULTIUSOS POUCH 450 M...	VANISH	*FABRIC TREATMENT and SANIT	101158.2330	4614.9270
9	7501025401948	CLORALEX GEL NO SALPICA DESINFECTANTE BP 1.65LT 75010...	CLORALEX	*FABRIC TREATMENT and SANIT	92258.6900	3823.6290
10	7501025401795	CLORALEX ULTRAGEL CITRICO BOTELLA 950ML 7501025401795	CLORALEX	*FABRIC TREATMENT and SANIT	87095.2380	3574.5250
11	7501025412005	CLORALEX MASCOTAS BOT PLAST 3750ML NAL 7501025412005	CLORALEX	*FABRIC TREATMENT and SANIT	84924.8890	1088.1830
12	7501025411992	CLORALEX MASCOTAS BOT PLAST 950ML NAL 7501025411992	CLORALEX	*FABRIC TREATMENT and SANIT	80928.6480	3221.8260
13	7501025405106	CLORALEX CLORO EN GEL BP 2000ML 7501025405106	CLORALEX	*FABRIC TREATMENT and SANIT	74798.1980	2420.5080
14	7501058716767	VANISH ROSA QUITAMANCHAS GEL BOT 3.8 LT PROM ESP 750...	VANISH	*FABRIC TREATMENT and SANIT	73497.3180	489.8540
15	7501058757791	VANISH BOT. 4000 ML 7501058757791	VANISH	*FABRIC TREATMENT and SANIT	72775.4500	406.8010

5.2 – CONSULTAS CLAVE

```
-- Ticket promedio (Valor/Unidad) por región y año
SELECT
    [Year],
    Region,
    SUM(TotalValueSales) AS Value,
    SUM(TotalUnitSales) AS Units,
    CAST(SUM(TotalValueSales)/NULLIF(SUM(TotalUnitSales),0) AS decimal(18,4)) AS AvgTicket
FROM dbo.vw_Sales
WHERE [Year] BETWEEN @YearFrom AND @YearTo
    AND (@RegionLike IS NULL OR Region LIKE @RegionLike)
GROUP BY [Year], Region
ORDER BY [Year], AvgTicket DESC;
```

	Year	Region	Value	Units	AvgTicket
1	2022	TOTAL AUTOS AREA 5	586556.8290	18202.5420	32.2239
2	2022	TOTAL AUTOS AREA 3	373166.6040	11727.9020	31.8187
3	2022	TOTAL AUTOS SCAN...	2532314.9...	84812.5430	29.8578
4	2022	TOTAL AUTOS AREA 4	296920.3760	10031.9460	29.5975
5	2022	TOTAL AUTOS AREA 1	318114.6890	10767.0560	29.5452
6	2022	TOTAL AUTOS AREA 6	422867.0700	14840.1800	28.4947
7	2022	TOTAL AUTOS AREA 2	534690.2570	19242.9480	27.7863

```
-- Top 15 productos por valor dentro de una región y año
SELECT TOP (15)
    ItemCode, ItemDescription, Brand, CategoryName,
    SUM(TotalValueSales) AS Value,
    SUM(TotalUnitSales) AS Units
FROM dbo.vw_Sales
WHERE [Year] BETWEEN @YearFrom AND @YearTo
    AND (@RegionLike IS NULL OR Region LIKE @RegionLike)
GROUP BY ItemCode, ItemDescription, Brand, CategoryName
ORDER BY Value DESC;
```

	ItemCode	ItemDescription	Brand	CategoryName	Value	Units
2	7501071900143	CLOROX REGULAR CONCENTRADO BOTPLAST.3800ML 75010...	CLOROX	"FABRIC TREATMENT and SANIT	218818.3440	5934.3890
3	7501025450212	CLORALEX EL RENDIDOR BOT 1900+100ML= 2LT 7501025450...	CLORALEX	"FABRIC TREATMENT and SANIT	159701.0570	6664.2160
4	7501025405090	CLORALEX ULTRAGEL CLORO GEL CONCENTRADO BOT 600 M...	CLORALEX	"FABRIC TREATMENT and SANIT	158175.8510	9841.5560
5	7501025405212	CLORALEX ULTRAGEL CLORO EN GEL CONCENTRADO 950ML ...	CLORALEX	"FABRIC TREATMENT and SANIT	147634.2000	6139.6570
6	7501025405151	CLORALEX CLORO EN GEL BOT 950ML 7501025405151	CLORALEX	"FABRIC TREATMENT and SANIT	122768.5530	6908.6350
7	7503002319529	VANISH LIQUIDO BOTELLA 925ML NAL. 7503002319529	VANISH	"FABRIC TREATMENT and SANIT	104776.6290	1947.7090
8	7501058717023	VANISH ROSA QUITAMANCHAS GEL MULTIUSOS POUCH 450 M...	VANISH	"FABRIC TREATMENT and SANIT	101158.2330	4614.9270
9	7501025401948	CLORALEX GEL NO SALPICA DESINFECTANTE BP 1.65LT 75010...	CLORALEX	"FABRIC TREATMENT and SANIT	92258.6900	3823.6290
10	7501025401795	CLORALEX ULTRAGEL CITRICO BOTELLA 950ML 7501025401795	CLORALEX	"FABRIC TREATMENT and SANIT	87095.2380	3574.5250
11	7501025412005	CLORALEX MASCOTAS BOT PLAST 3750ML NAL 7501025412005	CLORALEX	"FABRIC TREATMENT and SANIT	84924.8890	1088.1830
12	7501025411992	CLORALEX MASCOTAS BOT PLAST 950ML NAL 7501025411992	CLORALEX	"FABRIC TREATMENT and SANIT	80928.6480	3221.8260
13	7501025405106	CLORALEX CLORO EN GEL BP 2000ML 7501025405106	CLORALEX	"FABRIC TREATMENT and SANIT	74798.1980	2420.5080
14	7501058716767	VANISH ROSA QUITAMANCHAS GEL BOT 3.8 LT PROM ESP 750...	VANISH	"FABRIC TREATMENT and SANIT	73497.3180	489.8540
15	7501058757791	VANISH BOT. 4000 ML 7501058757791	VANISH	"FABRIC TREATMENT and SANIT	72775.4500	406.8010

5.2 – CONSULTAS CLAVE

```
-- Alternativa aún más robusta (opcional): un procedimiento almacenado
USE EmpresaAliadaDW;
GO
CREATE OR ALTER PROCEDURE dbo.usp_Insights
    @YearFrom    smallint,
    @YearTo      smallint,
    @RegionLike  nvarchar(100) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Ejemplo: ventas por categoría y año
    SELECT [Year], CategoryID, CategoryName,
           SUM(TotalUnitSales) AS Units,
           SUM(TotalValueSales) AS Value,
           CAST(SUM(TotalValueSales)/NULLIF(SUM(TotalUnitSales),0) AS decimal(18,4)) AS AvgPrice
    FROM dbo.vw_Sales
    WHERE [Year] BETWEEN @YearFrom AND @YearTo
           AND (@RegionLike IS NULL OR Region LIKE @RegionLike)
    GROUP BY [Year], CategoryID, CategoryName
    ORDER BY [Year], Value DESC;

    -- (añade aquí los demás SELECTs que quieras devolver)
END;
GO

-- Ejemplos de uso:
EXEC dbo.usp_Insights @YearFrom = 2021, @YearTo = 2022, @RegionLike = NULL;
EXEC dbo.usp_Insights @YearFrom = 2021, @YearTo = 2022, @RegionLike = N'%AREA 1%';
```

	Year	CategoryID	CategoryName	Units	Value	AvgPrice	
1	2022	1	*FABRIC TREATMENT and SANIT	169625.1170	5064630.7760	29.8578	

	Year	CategoryID	CategoryName	Units	Value	AvgPrice	
1	2022	1	*FABRIC TREATMENT and SANIT	10767.0560	318114.6890	29.5452	

6 – CREACIÓN DE DASHBOARDS EN POWERBI



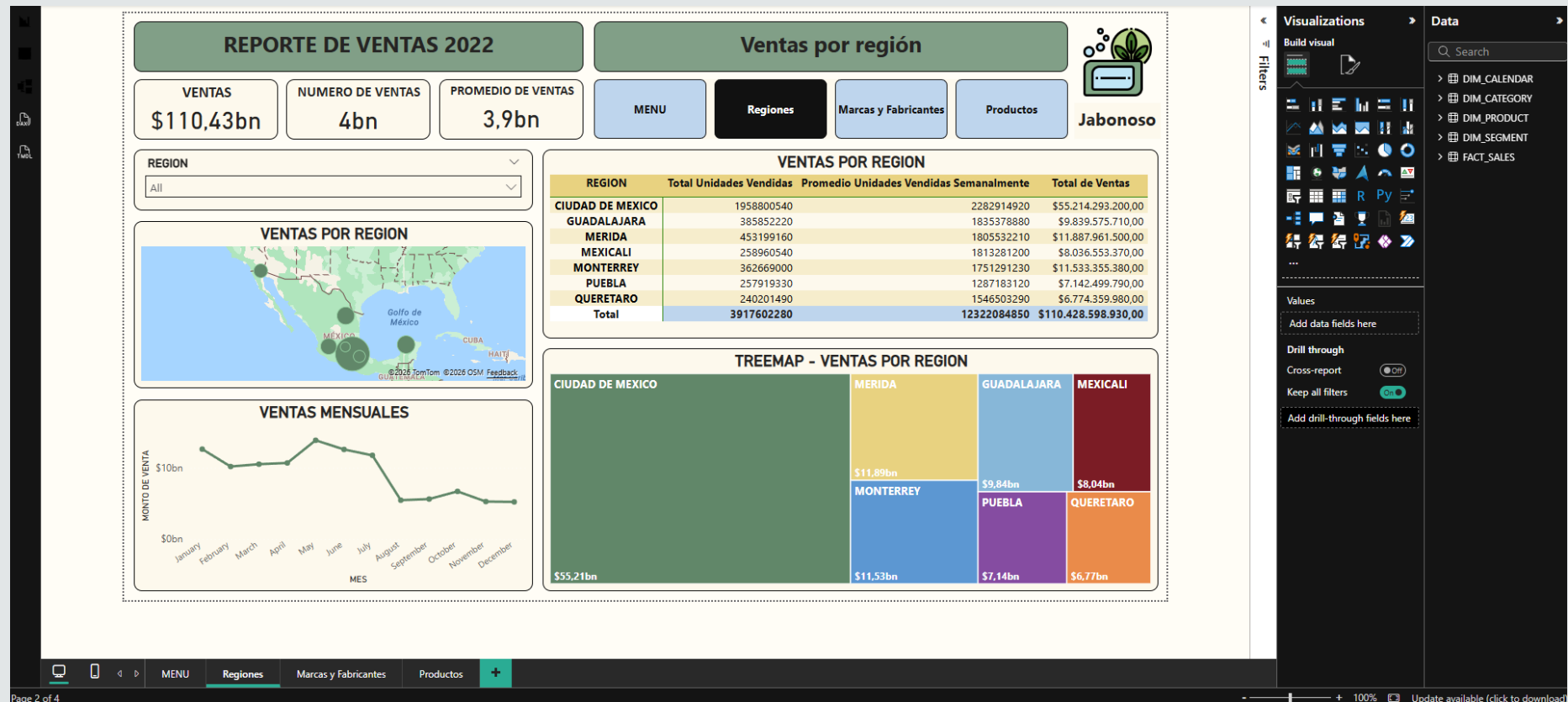
6.1 – ESTRUCTURA DE DASHBOARD

6.2 – VISUALIZACIONES INTERACTIVAS



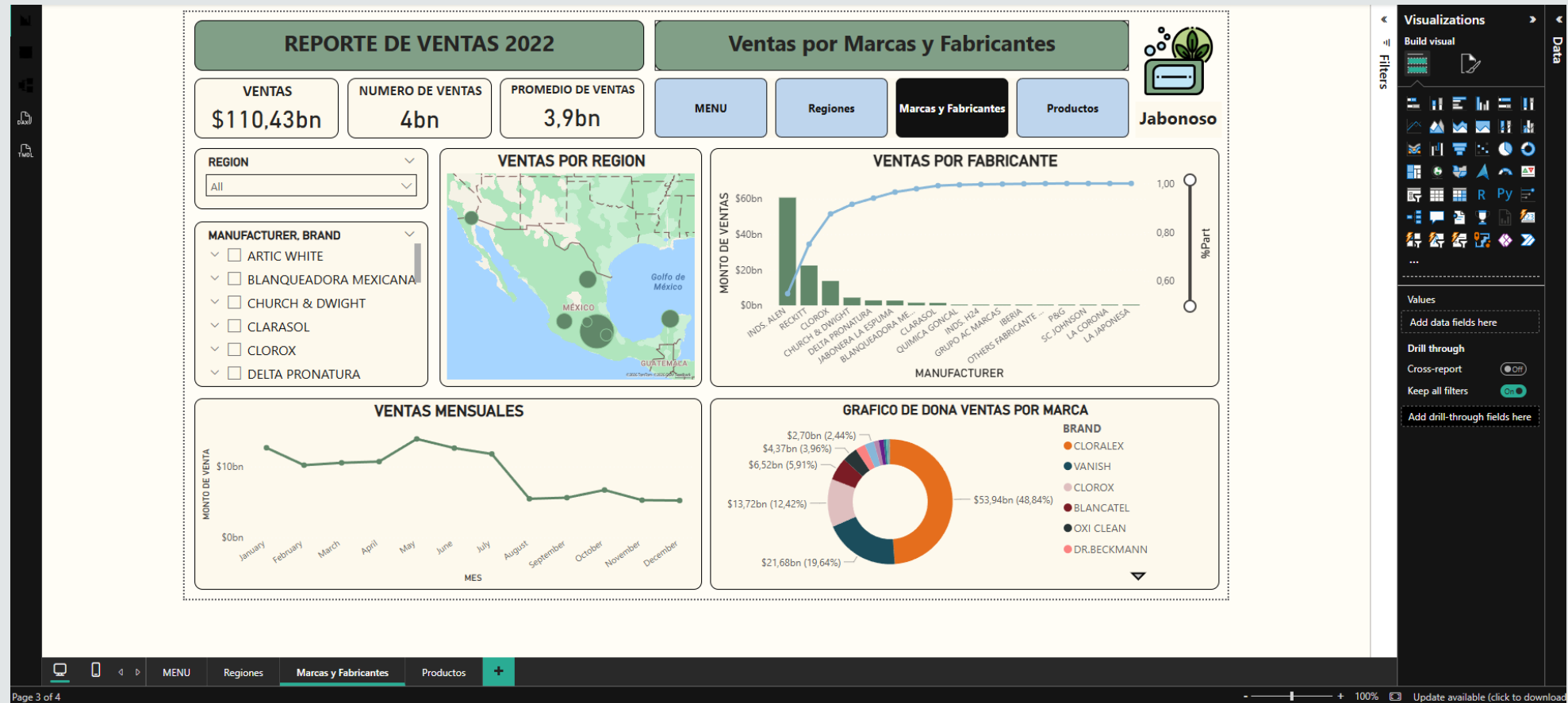
6.1 – ESTRUCTURA DE DASHBOARD

6.2 – VISUALIZACIONES INTERACTIVAS



6.1 – ESTRUCTURA DE DASHBOARD

6.2 – VISUALIZACIONES INTERACTIVAS



6.1 – ESTRUCTURA DE DASHBOARD

6.2 – VISUALIZACIONES INTERACTIVAS



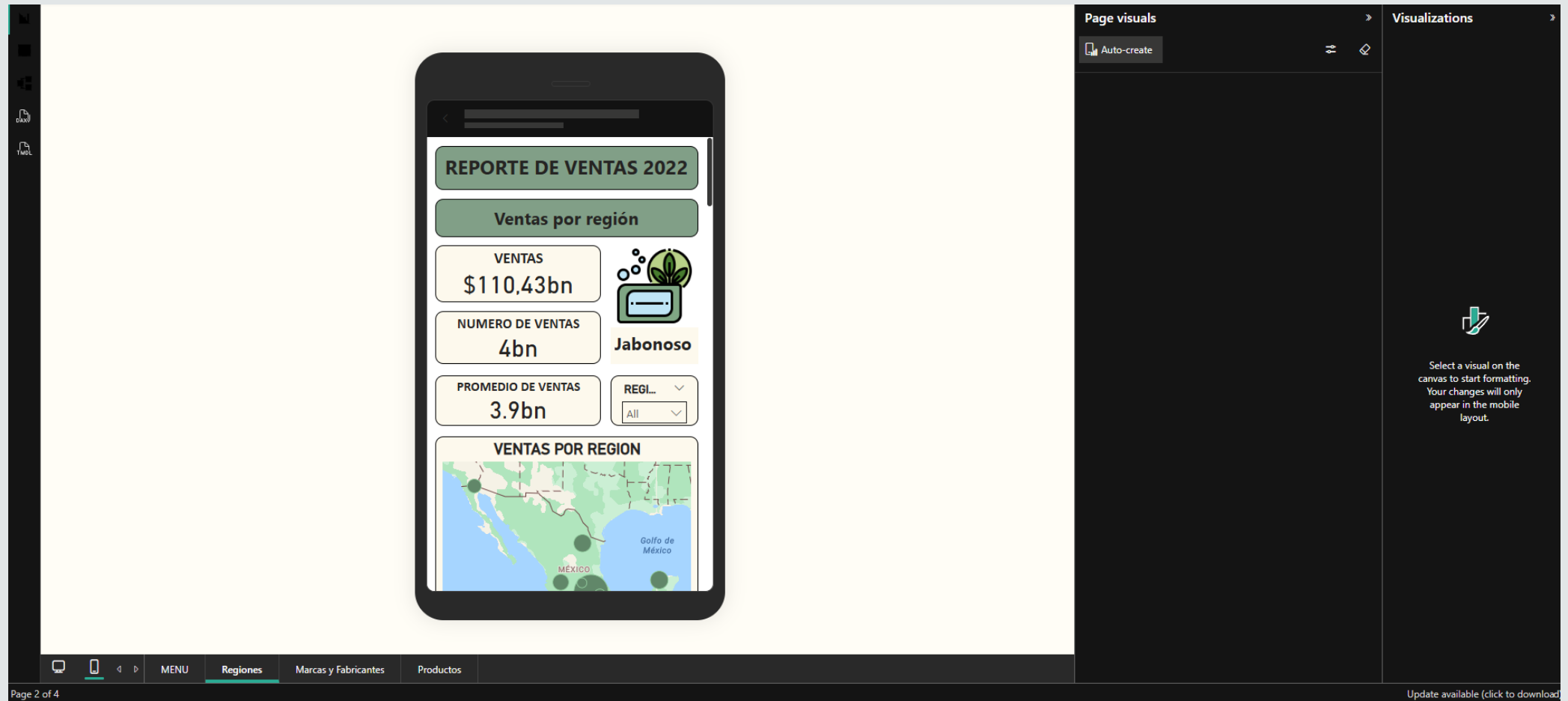
6.1 – ESTRUCTURA DE DASHBOARD

6.2 – VISUALIZACIONES INTERACTIVAS



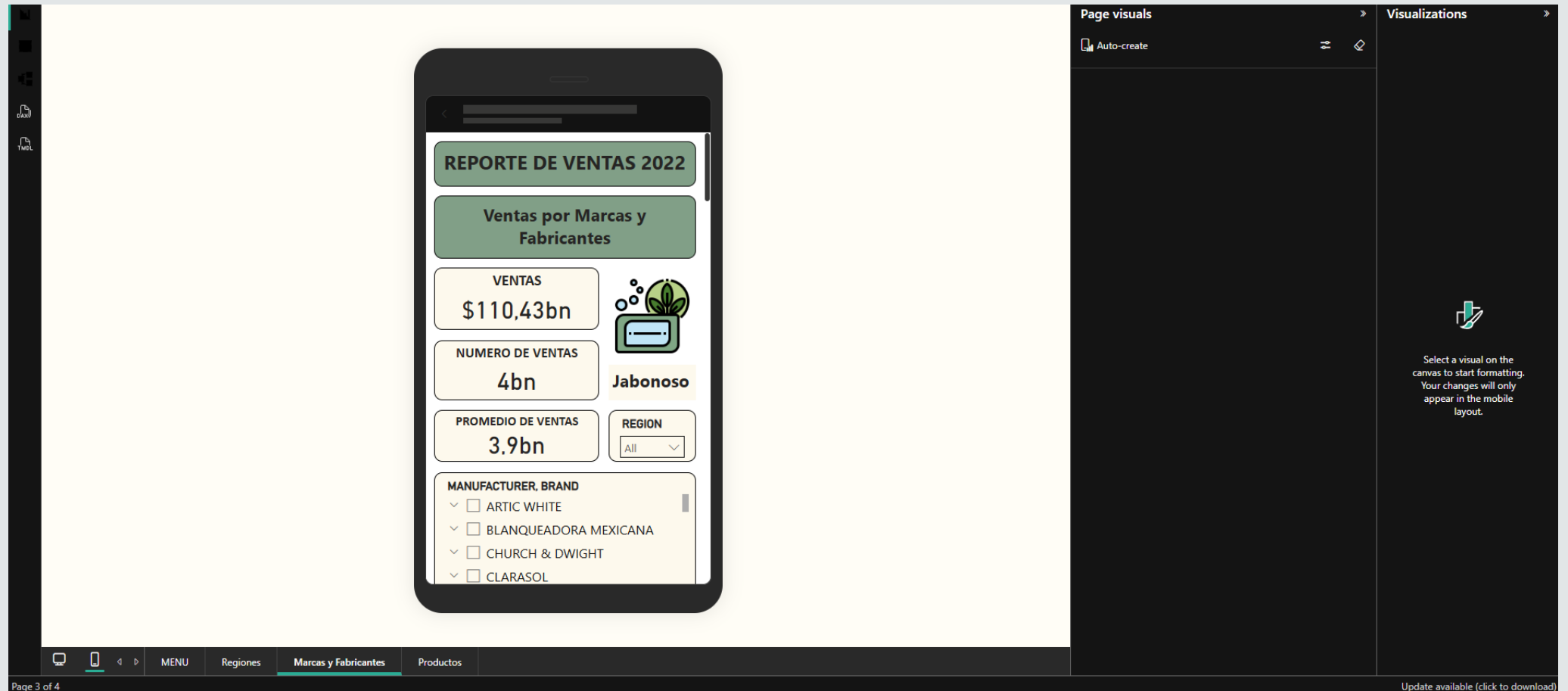
6.1 – ESTRUCTURA DE DASHBOARD

6.2 – VISUALIZACIONES INTERACTIVAS



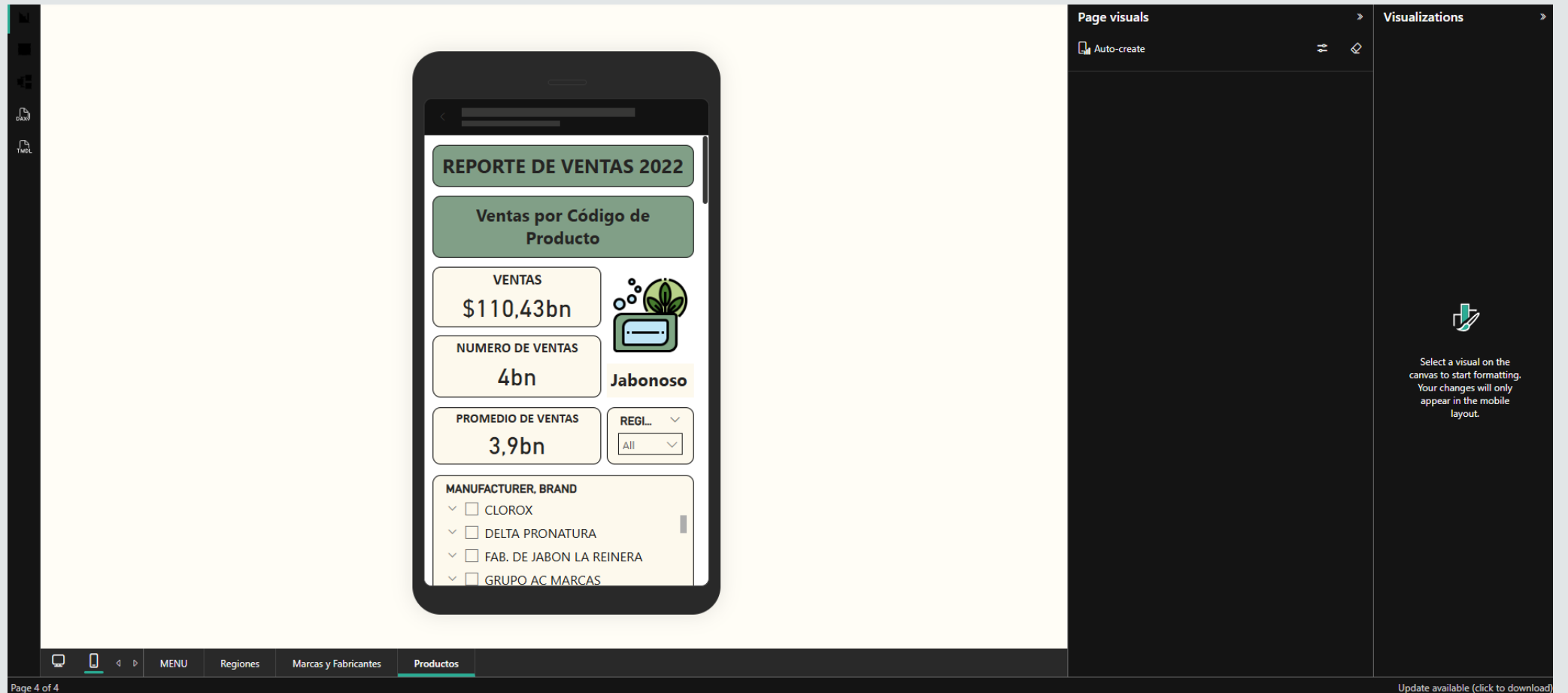
6.1 – ESTRUCTURA DE DASHBOARD

6.2 – VISUALIZACIONES INTERACTIVAS

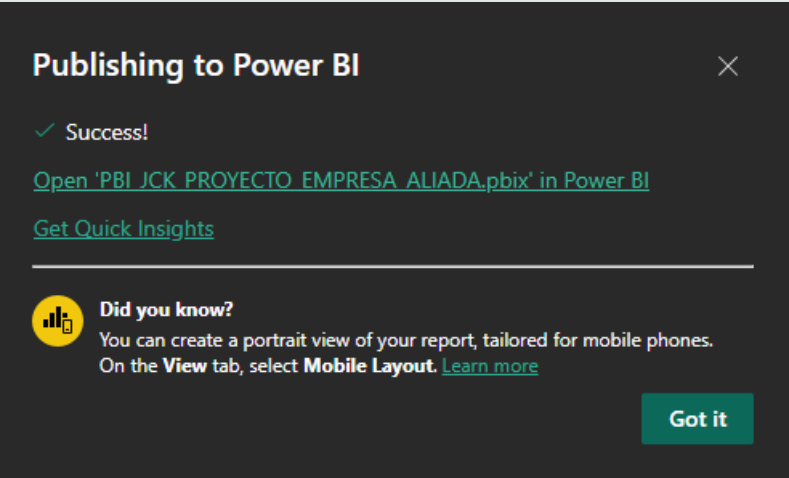


6.1 – ESTRUCTURA DE DASHBOARD

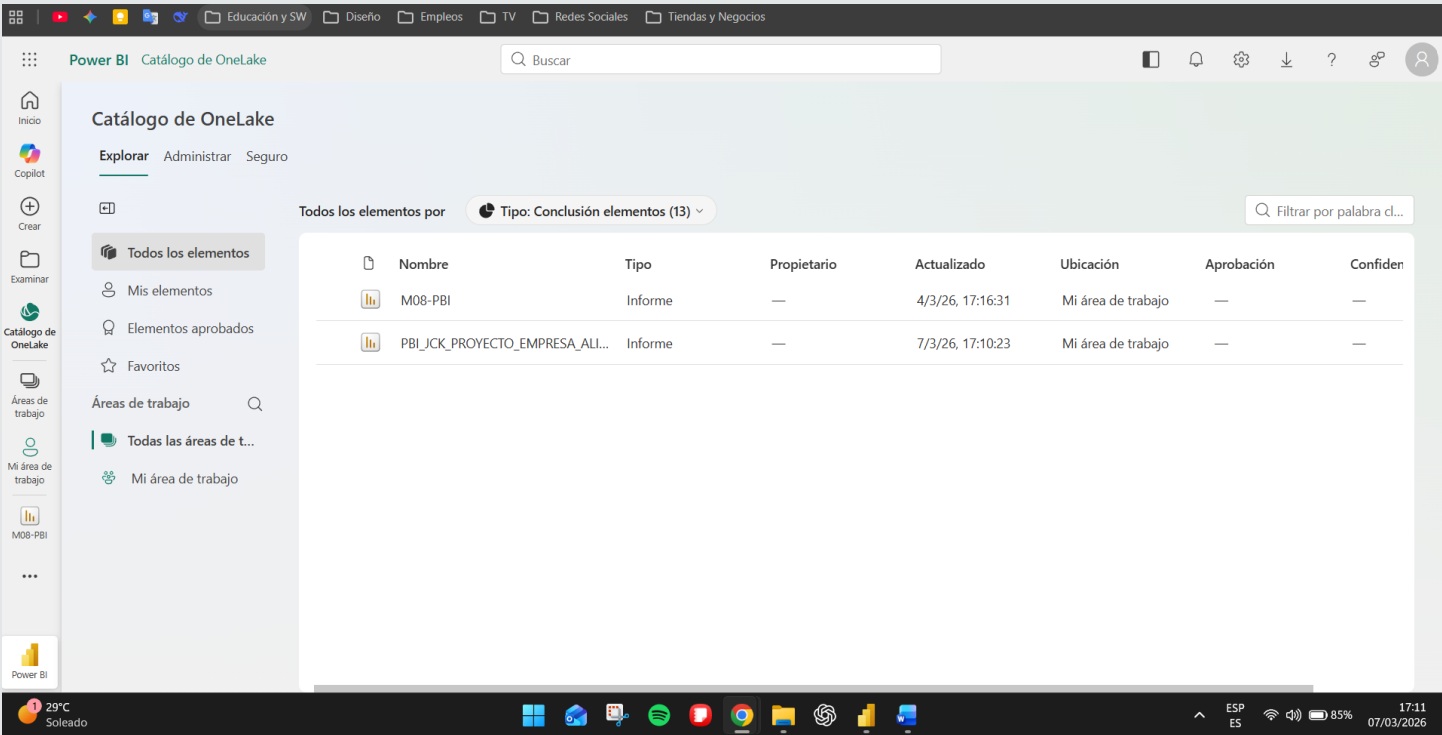
6.2 – VISUALIZACIONES INTERACTIVAS



6.3 – CAPTURAS DE PANTALLA Y ENLACE



- https://app.powerbi.com/links/HFB1ZOJVQr?ctid=f343a737-de9d-435b-8aa9-11fda05c6918&pbi_source=linkShare



7 – PREDICCIÓN DE VENTAS CON MACHINE LEARNING



7.1 – SELECCIÓN DE MODELO

- Se construyeron series de ventas semanales para Vanish y Lysol, utilizando exclusivamente el nivel nacional TOTAL AUTOS SCANNING MEXICO para evitar duplicar las ventas contenidas también en las áreas regionales.
- El conjunto analítico final contiene 80 semanas consecutivas de información para ambas marcas. Debido a que el objetivo consiste en pronosticar ventas futuras a partir de su comportamiento histórico y no se dispone de variables explicativas futuras como promociones, precios o inversión publicitaria, se seleccionó un enfoque de series de tiempo.
- El análisis gráfico mostró comportamientos diferentes entre las marcas:
 - Vanish: alta variabilidad semanal, con cambios de nivel y picos importantes.
 - Lysol: caída pronunciada durante 2022 y posterior estabilización alrededor de un nivel relativamente constante.

7.1 – SELECCIÓN DE MODELO

```
# 39. Importar la librería para visualización de datos
```

```
import matplotlib.pyplot as plt
```

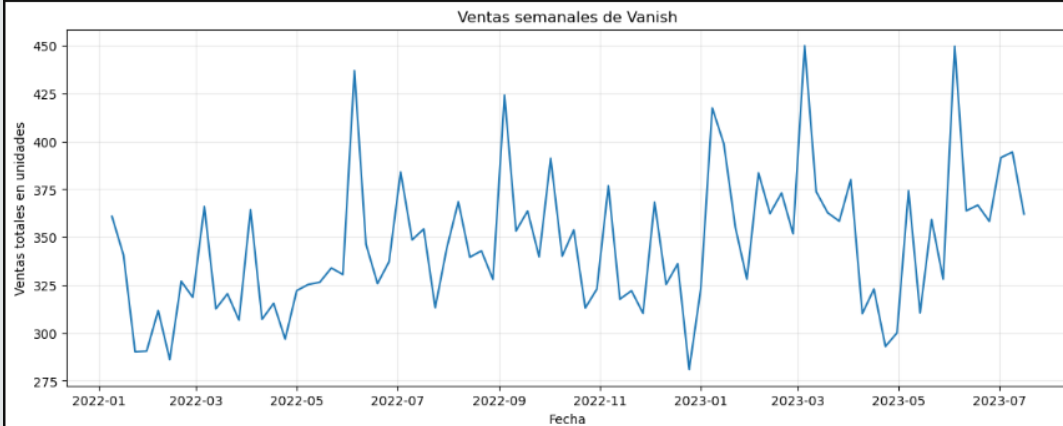
```
# 40. Visualizar la evolución semanal de las ventas de Vanish
```

```
plt.figure(figsize=(14, 5))
```

```
plt.plot(
    serie_modelo.index,
    serie_modelo['VANISH']
)
```

```
plt.title('Ventas semanales de Vanish')
plt.xlabel('Fecha')
plt.ylabel('Ventas totales en unidades')
plt.grid(alpha=0.3)
```

```
plt.show()
```



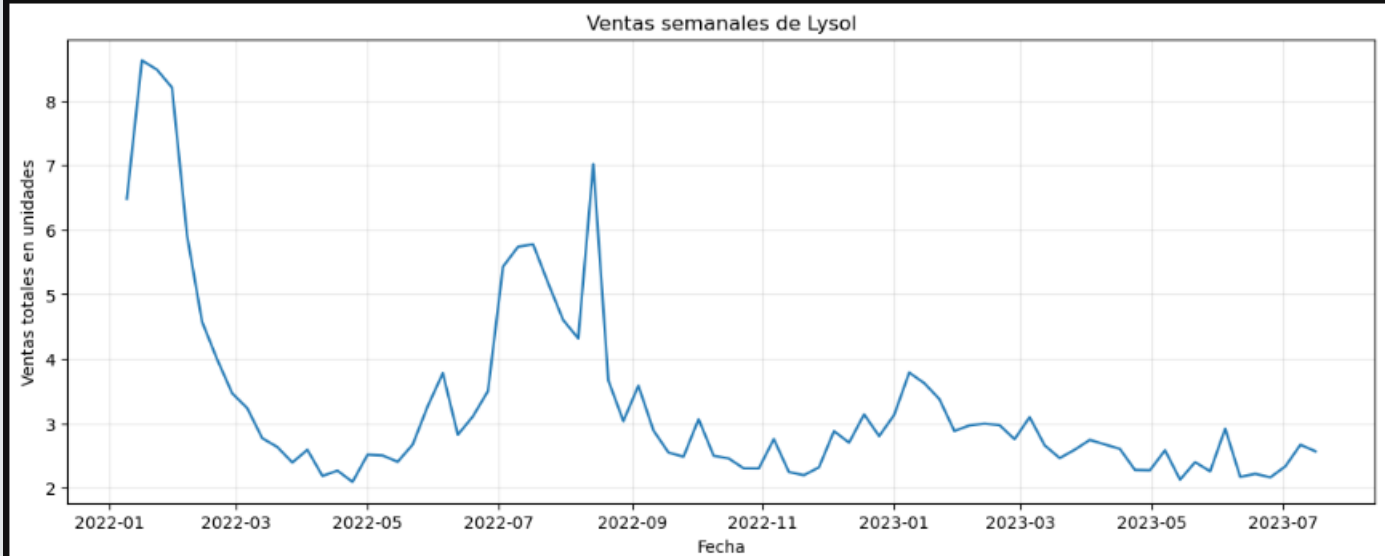
```
# 41. Visualizar la evolución semanal de las ventas de Lysol
```

```
plt.figure(figsize=(14, 5))
```

```
plt.plot(
    serie_modelo.index,
    serie_modelo['LYSOL']
)
```

```
plt.title('Ventas semanales de Lysol')
plt.xlabel('Fecha')
plt.ylabel('Ventas totales en unidades')
plt.grid(alpha=0.3)
```

```
plt.show()
```



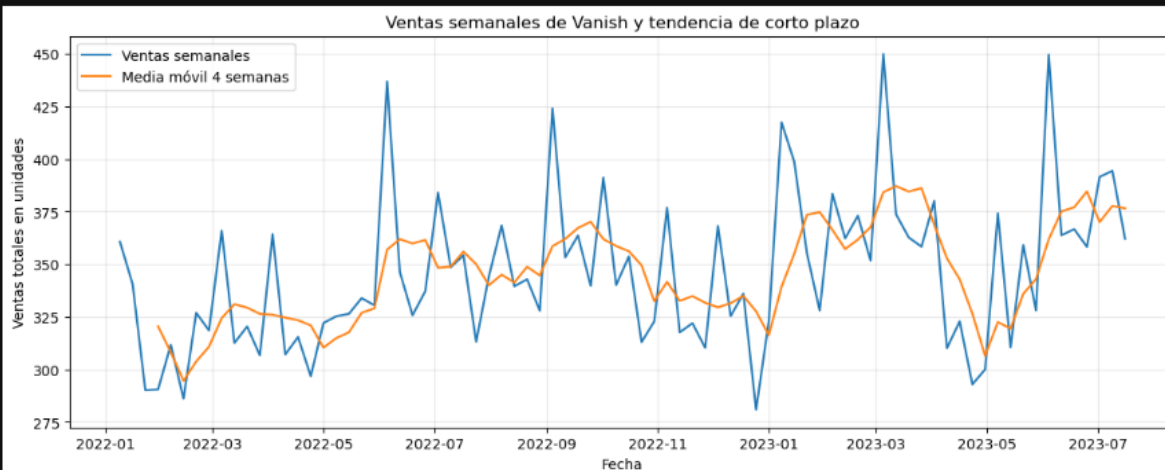
7.1 – SELECCIÓN DE MODELO

```
# 42. Calcular medias móviles de 4 semanas
```

```
serie_modelo['VANISH_MA4'] = (  
    serie_modelo['VANISH']  
    .rolling(window=4)  
    .mean()  
)  
  
serie_modelo['LYSOL_MA4'] = (  
    serie_modelo['LYSOL']  
    .rolling(window=4)  
    .mean()  
)
```

```
# 43. Comparar Las ventas de Vanish con su media móvil de 4 semanas
```

```
plt.figure(figsize=(14, 5))  
  
plt.plot(  
    serie_modelo.index,  
    serie_modelo['VANISH'],  
    label='Ventas semanales'  
)  
  
plt.plot(  
    serie_modelo.index,  
    serie_modelo['VANISH_MA4'],  
    label='Media móvil 4 semanas'  
)  
  
plt.title('Ventas semanales de Vanish y tendencia de corto plazo')  
plt.xlabel('Fecha')  
plt.ylabel('Ventas totales en unidades')  
plt.legend()  
plt.grid(alpha=0.3)  
plt.show()
```



```
# 44. Comparar Las ventas de Lysol con su media móvil de 4 semanas
```

```
plt.figure(figsize=(14, 5))  
  
plt.plot(  
    serie_modelo.index,  
    serie_modelo['LYSOL'],  
    label='Ventas semanales'  
)  
  
plt.plot(  
    serie_modelo.index,  
    serie_modelo['LYSOL_MA4'],  
    label='Media móvil 4 semanas'  
)  
  
plt.title('Ventas semanales de Lysol y tendencia de corto plazo')  
plt.xlabel('Fecha')  
plt.ylabel('Ventas totales en unidades')  
plt.legend()  
plt.grid(alpha=0.3)  
plt.show()
```



7.1 – SELECCIÓN DE MODELO

- Evaluación de estacionariedad: Antes de construir ARIMA se aplicó la prueba Augmented Dickey-Fuller (ADF) sobre las series de entrenamiento. En las series originales se obtuvieron:

Marca	p-value ADF	Resultado	p-value después de diferenciar
Vanish	0.4741	No estacionaria	0.0060
Lysol	0.0978	No estacionaria	≈ 0.0000

- Debido a estos resultados se estableció $d=1$ para los modelos ARIMA de ambas marcas

7.1 – SELECCIÓN DE MODELO

```
# 50. Importar la prueba de Dickey-Fuller aumentada para evaluar estacionariedad
```

```
from statsmodels.tsa.stattools import adfuller
```

```
# 51. Crear una función para evaluar la estacionariedad de una serie temporal
```

```
def prueba_adf(serie, nombre):  
  
    resultado = adfuller(serie.dropna())  
  
    print(f'Prueba ADF - {nombre}')  
    print('-' * 40)  
    print(f'Estadístico ADF: {resultado[0]:.4f}')  
    print(f'p-value: {resultado[1]:.6f}')  
  
    print('\nValores críticos:')  
  
    for nivel, valor in resultado[4].items():  
        print(f'{nivel}: {valor:.4f}')  
  
    if resultado[1] < 0.05:  
        print('\nConclusión: La serie es estacionaria.')  
    else:  
        print('\nConclusión: La serie NO es estacionaria.')
```

```
# 52. Evaluar la estacionariedad de las ventas de Vanish
```

```
prueba_adf(  
    train_vanish,  
    'Vanish'  
)
```

```
Prueba ADF - Vanish  
-----  
Estadístico ADF: -1.6173  
p-value: 0.474147
```

```
Valores críticos:  
1%: -3.5577  
5%: -2.9168  
10%: -2.5962
```

```
Conclusión: La serie NO es estacionaria.
```

```
# 53. Evaluar la estacionariedad de las ventas de Lysol
```

```
prueba_adf(  
    train_lysol,  
    'Lysol'  
)
```

```
Prueba ADF - Lysol  
-----  
Estadístico ADF: -2.5773  
p-value: 0.097772
```

```
Valores críticos:  
1%: -3.5387  
5%: -2.9086  
10%: -2.5919
```

```
Conclusión: La serie NO es estacionaria.
```

7.1 – SELECCIÓN DE MODELO

```
# 54. Aplicar una primera diferencia a Las series de entrenamiento

train_vanish_diff = train_vanish.diff().dropna()
train_lysol_diff = train_lysol.diff().dropna()

print('Observaciones Vanish diferenciadas:', len(train_vanish_diff))
print('Observaciones Lysol diferenciadas:', len(train_lysol_diff))

Observaciones Vanish diferenciadas: 63
Observaciones Lysol diferenciadas: 63

# 55. Visualizar la primera diferencia de las ventas de Vanish

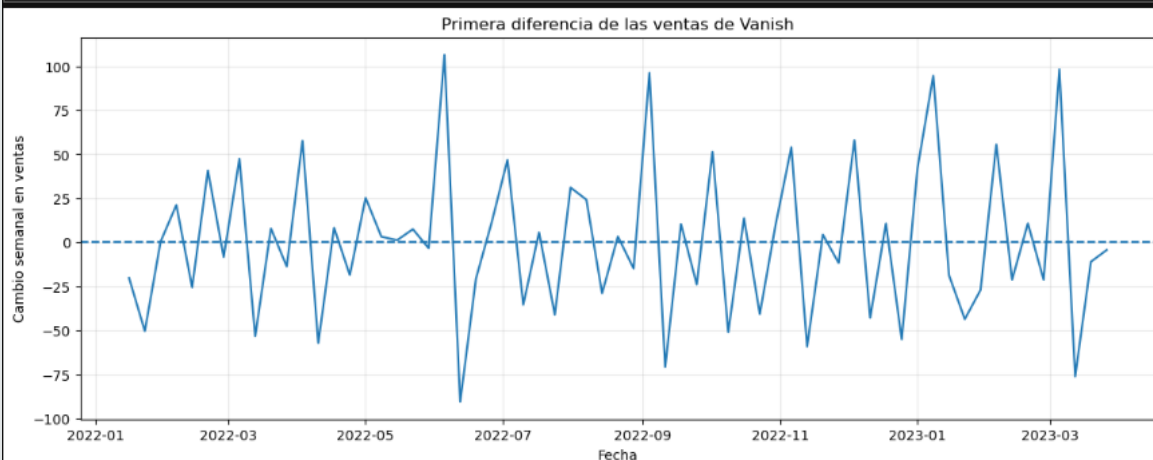
plt.figure(figsize=(14, 5))

plt.plot(
    train_vanish_diff.index,
    train_vanish_diff
)

plt.axhline(
    y=0,
    linestyle='--'
)

plt.title('Primera diferencia de las ventas de Vanish')
plt.xlabel('Fecha')
plt.ylabel('Cambio semanal en ventas')
plt.grid(alpha=0.3)

plt.show()
```



```
# 56. Visualizar la primera diferencia de las ventas de Lysol

plt.figure(figsize=(14, 5))

plt.plot(
    train_lysol_diff.index,
    train_lysol_diff
)

plt.axhline(
    y=0,
    linestyle='--'
)

plt.title('Primera diferencia de las ventas de Lysol')
plt.xlabel('Fecha')
plt.ylabel('Cambio semanal en ventas')
plt.grid(alpha=0.3)

plt.show()
```



7.1 – SELECCIÓN DE MODELO

```
# 57. Evaluar la estacionariedad después de aplicar una primera diferencia
```

```
prueba_adf(  
    train_vanish_diff,  
    'Vanish - Primera diferencia'  
)
```

```
print('\n')
```

```
prueba_adf(  
    train_lysol_diff,  
    'Lysol - Primera diferencia'  
)
```

```
Prueba ADF - Vanish - Primera diferencia  
-----  
Estadístico ADF: -3.5874  
p-value: 0.006006
```

```
Valores críticos:  
1%: -3.5656  
5%: -2.9201  
10%: -2.5980
```

```
Conclusión: Se rechaza H0. La serie puede considerarse estacionaria.
```

```
Prueba ADF - Lysol - Primera diferencia  
-----  
Estadístico ADF: -6.0571  
p-value: 0.000000
```

```
Valores críticos:  
1%: -3.5424  
5%: -2.9102  
10%: -2.5927
```

```
Conclusión: Se rechaza H0. La serie puede considerarse estacionaria.
```

7.1 – SELECCIÓN DE MODELO

Selección de parámetros p , d y q

- Después de establecer $d=1$, se analizaron las funciones ACF y PACF para identificar posibles componentes autorregresivos y de media móvil. Posteriormente se evaluaron sistemáticamente distintas configuraciones ARIMA con
- $p=0,1,2,3$
- $q=0,1,2,3$
- Manteniendo $d=1$
- Los modelos fueron comparados mediante los criterios AIC y BIC, donde valores menores representan un mejor equilibrio entre calidad de ajuste y complejidad. La selección inicial produjo:
- Vanish $\rightarrow$ ARIMA(3,1,0) AIC = 620.93 BIC = 629.50
- Lysol $\rightarrow$ ARIMA(0,1,0) AIC = 155.80 BIC = 157.94

7.1 – SELECCIÓN DE MODELO

```
# 58. Importar las funciones ACF y PACF para analizar la autocorrelación
```

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf
```

```
# 59. Analizar la autocorrelación de la serie diferenciada de Vanish
```

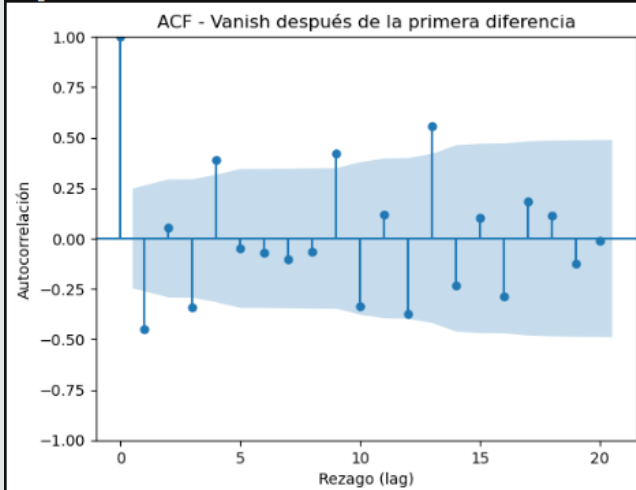
```
plt.figure(figsize=(12, 5))
```

```
plot_acf(
    train_vanish_diff,
    lags=20
)
```

```
plt.title('ACF - Vanish después de la primera diferencia')
plt.xlabel('Rezago (lag)')
plt.ylabel('Autocorrelación')
```

```
plt.show()
```

<Figure size 1200x500 with 0 Axes>



```
# 60. Analizar la autocorrelación parcial de la serie diferenciada de Vanish
```

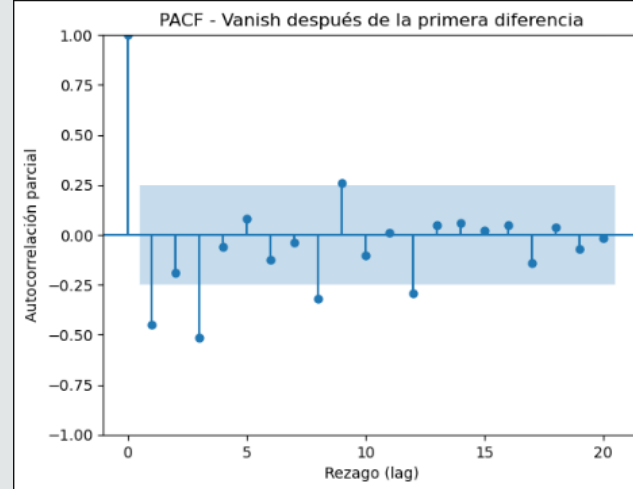
```
plt.figure(figsize=(12, 5))
```

```
plot_pacf(
    train_vanish_diff,
    lags=20,
    method='ywm'
)
```

```
plt.title('PACF - Vanish después de la primera diferencia')
plt.xlabel('Rezago (lag)')
plt.ylabel('Autocorrelación parcial')
```

```
plt.show()
```

<Figure size 1200x500 with 0 Axes>



7.1 – SELECCIÓN DE MODELO

```
# 61. Analizar la autocorrelación de la serie diferenciada de Lysol
```

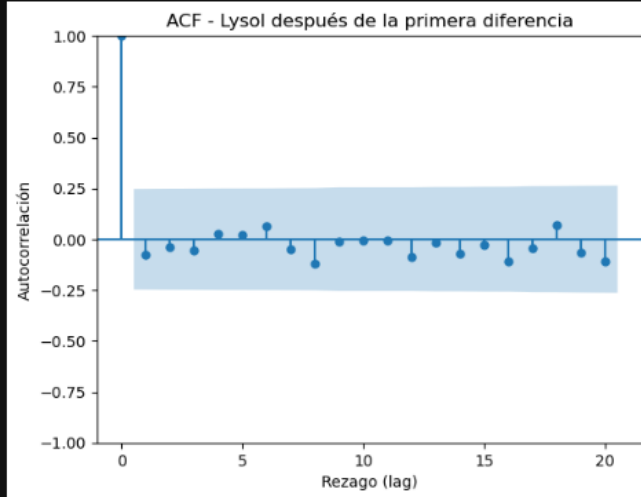
```
plt.figure(figsize=(12, 5))

plot_acf(
    train_lysol_diff,
    lags=20
)

plt.title('ACF - Lysol después de la primera diferencia')
plt.xlabel('Rezago (lag)')
plt.ylabel('Autocorrelación')

plt.show()
```

<Figure size 1200x500 with 0 Axes>



```
# 62. Analizar la autocorrelación parcial de la serie diferenciada de Lysol
```

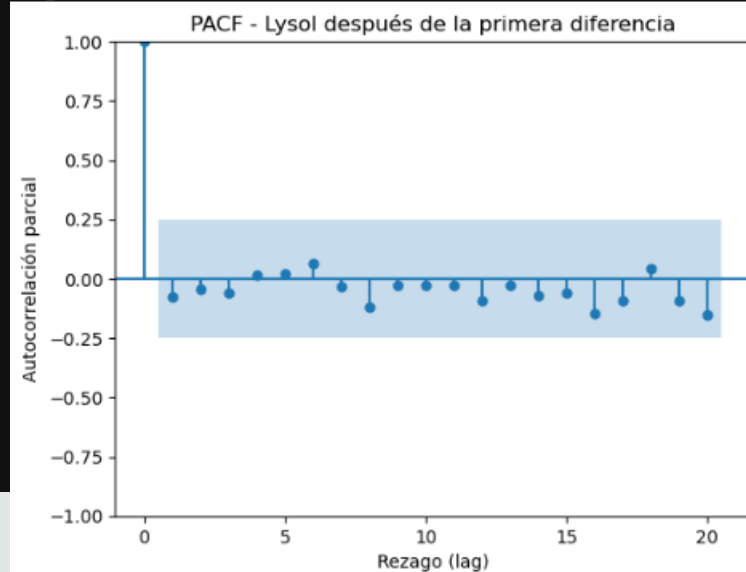
```
plt.figure(figsize=(12, 5))

plot_pacf(
    train_lysol_diff,
    lags=20,
    method='ywmm'
)

plt.title('PACF - Lysol después de la primera diferencia')
plt.xlabel('Rezago (lag)')
plt.ylabel('Autocorrelación parcial')

plt.show()
```

<Figure size 1200x500 with 0 Axes>



7.1 – SELECCIÓN DE MODELO

```
# 63. Importar el modelo ARIMA
```

```
from statsmodels.tsa.arima.model import ARIMA
import warnings
```

```
warnings.filterwarnings('ignore')
```

```
# 64. Crear una función para comparar modelos ARIMA mediante AIC y BIC
```

```
def buscar_arima(serie, nombre, max_p=3, max_q=3):
```

```
    resultados = []
```

```
    for p in range(max_p + 1):
        for q in range(max_q + 1):
```

```
        try:
```

```
            modelo = ARIMA(
                serie,
                order=(p, 1, q)
            )
```

```
            ajuste = modelo.fit()
```

```
            resultados.append({
                'Marca': nombre,
                'p': p,
                'd': 1,
                'q': q,
                'AIC': ajuste.aic,
                'BIC': ajuste.bic
            })
```

```
        except:
```

```
            pass
```

```
resultados_df = pd.DataFrame(resultados)
```

```
return resultados_df.sort_values('AIC').reset_index(drop=True)
```

```
# 65. Evaluar diferentes combinaciones ARIMA para Vanish
```

```
resultados_vanish = buscar_arima(
    train_vanish,
    'Vanish'
)
```

```
resultados_vanish.head(10)
```

	Marca	p	d	q	AIC	BIC
0	Vanish	3	1	0	620.930888	629.503426
1	Vanish	2	1	3	622.319008	635.177817
2	Vanish	3	1	1	622.418807	633.134481
3	Vanish	3	1	2	624.418670	637.277479
4	Vanish	3	1	3	625.527006	640.528949
5	Vanish	0	1	1	626.797545	631.083815
6	Vanish	1	1	3	627.155733	637.871407
7	Vanish	0	1	2	628.640831	635.070235
8	Vanish	1	1	1	628.647846	635.077250
9	Vanish	1	1	2	630.540647	639.113186

7.1 – SELECCIÓN DE MODELO

```
# 66. Evaluar diferentes combinaciones ARIMA para Lysol
```

```
resultados_lysol = buscar_arima(  
    train_lysol,  
    'Lysol'  
)
```

```
resultados_lysol.head(10)
```

	Marca	p	d	q	AIC	BIC
0	Lysol	0	1	0	155.801315	157.944450
1	Lysol	0	1	1	157.478228	161.764497
2	Lysol	1	1	0	157.504227	161.790496
3	Lysol	1	1	1	159.386770	165.816174
4	Lysol	0	1	2	159.394030	165.823434
5	Lysol	2	1	0	159.418022	165.847427
6	Lysol	2	1	2	160.344641	171.060315
7	Lysol	3	1	0	161.244675	169.817214
8	Lysol	0	1	3	161.312363	169.884902
9	Lysol	2	1	1	161.370474	169.943012

7.1 – SELECCIÓN DE MODELO

- Modelos ARIMA inicialmente seleccionados. Los modelos seleccionados mediante AIC/BIC fueron entrenados utilizando las 64 semanas del conjunto de entrenamiento.
- Vanish – ARIMA(3,1,0)
- Los tres componentes autorregresivos resultaron estadísticamente significativos.
- Lysol – ARIMA(0,1,0)
- La estructura simplificada fue consistente con la baja autocorrelación observada después de aplicar la primera diferencia. La prueba Ljung-Box aplicada a los residuos obtuvo
- Vanish $p=0.258$
- Lysol $p=0.994$
- Al ser ambos valores superiores a 0.05, no se encontró evidencia de autocorrelación residual significativa hasta el rezago evaluado.

7.1 – SELECCIÓN DE MODELO

```
# 67. Entrenar el modelo ARIMA seleccionado para Vanish
```

```
modelo_vanish = ARIMA(
    train_vanish,
    order=(3, 1, 0)
)

ajuste_vanish = modelo_vanish.fit()

print('Modelo Vanish: ARIMA(3,1,0)')
print('AIC:', ajuste_vanish.aic)
print('BIC:', ajuste_vanish.bic)
```

```
Modelo Vanish: ARIMA(3,1,0)
AIC: 620.9308875063089
BIC: 629.503426411875
```

```
# 68. Entrenar el modelo ARIMA seleccionado para Lysol
```

```
modelo_lysol = ARIMA(
    train_lysol,
    order=(0, 1, 0)
)

ajuste_lysol = modelo_lysol.fit()

print('Modelo Lysol: ARIMA(0,1,0)')
print('AIC:', ajuste_lysol.aic)
print('BIC:', ajuste_lysol.bic)
```

```
Modelo Lysol: ARIMA(0,1,0)
AIC: 155.8013147915948
BIC: 157.94444951798633
```

```
# 69. Mostrar el resumen estadístico del modelo de Vanish
```

```
print(ajuste_vanish.summary())
```

```
SARIMAX Results
=====
Dep. Variable:          VANISH    No. Observations:         64
Model:                ARIMA(3, 1, 0)  Log Likelihood        -306.465
Date:                 Sun, 27 Sep 2026  AIC                620.931
Time:                 18:43:51    BIC                629.503
Sample:               01-09-2022    HQIC               624.303
                   - 03-26-2023
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
ar.L1         -0.6669      0.145     -4.604      0.000     -0.951    -0.383
ar.L2         -0.5387      0.164     -3.291      0.001     -0.860    -0.218
ar.L3         -0.5817      0.164     -3.548      0.000     -0.903    -0.260
sigma2        960.0218    117.828      8.148      0.000     729.083    1190.960
=====
Ljung-Box (L1) (Q):                0.14    Jarque-Bera (JB):                36.01
Prob(Q):                           0.71    Prob(JB):                        0.00
Heteroskedasticity (H):              1.08    Skew:                            1.42
Prob(H) (two-sided):                0.86    Kurtosis:                        5.45
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

```
# 70. Mostrar el resumen estadístico del modelo de Lysol
```

```
print(ajuste_lysol.summary())
```

```
SARIMAX Results
=====
Dep. Variable:          LYSOL    No. Observations:         64
Model:                ARIMA(0, 1, 0)  Log Likelihood        -76.901
Date:                 Sun, 27 Sep 2026  AIC                155.801
Time:                 18:44:40    BIC                157.944
Sample:               01-09-2022    HQIC               156.644
                   - 03-26-2023
Covariance Type:      opg
=====
              coef    std err          z      P>|z|      [0.025    0.975]
-----
sigma2         0.6726      0.061     10.975      0.000      0.552      0.793
=====
Ljung-Box (L1) (Q):                0.37    Jarque-Bera (JB):                84.01
Prob(Q):                           0.54    Prob(JB):                        0.00
Heteroskedasticity (H):              0.18    Skew:                            -0.26
Prob(H) (two-sided):                0.00    Kurtosis:                        8.63
=====
```

Warnings:

[1] Covariance matrix calculated using the outer product of gradients (complex-step).

7.1 – SELECCIÓN DE MODELO

```
# 71. Evaluar autocorrelación restante en los residuos con Ljung-Box
```

```
from statsmodels.stats.diagnostic import acorr_ljungbox
```

```
residuos_vanish = ajuste_vanish.resid.iloc[1:].dropna()
```

```
residuos_lysol = ajuste_lysol.resid.iloc[1:].dropna()
```

```
ljung_vanish = acorr_ljungbox(  
    residuos_vanish,  
    lags=[10],  
    model_df=3,  
    return_df=True  
)
```

```
ljung_lysol = acorr_ljungbox(  
    residuos_lysol,  
    lags=[10],  
    model_df=0,  
    return_df=True  
)
```

```
print('Ljung-Box - Vanish')  
print(ljung_vanish)
```

```
print('\nLjung-Box - Lysol')  
print(ljung_lysol)
```

```
Ljung-Box - Vanish  
      lb_stat  lb_pvalue  
10  8.923518  0.258194
```

```
Ljung-Box - Lysol  
      lb_stat  lb_pvalue  
10  2.267001  0.993854
```

7.2 – VALIDACIÓN DEL MODELO

- Debido a que se trabaja con series de tiempo, los datos se dividieron respetando estrictamente el orden cronológico, evitando utilizar información futura durante el entrenamiento.
- De las 80 semanas disponibles:
- 64 semanas (80 %) entrenamiento
- 16 semanas (20 %) prueba
- Periodo de entrenamiento: 09/01/2022 – 26/03/2023
- Periodo de prueba: 02/04/2023 – 16/07/2023
- Esta estructura permitió evaluar los modelos utilizando semanas posteriores que no fueron observadas durante el entrenamiento.

7.2 – VALIDACIÓN DEL MODELO

45. Definir el tamaño de los conjuntos de entrenamiento y prueba

```
n_total = len(serie_modelo)
n_test = 16
n_train = n_total - n_test

print('Observaciones totales:', n_total)
print('Observaciones de entrenamiento:', n_train)
print('Observaciones de prueba:', n_test)

print('\nPorcentaje entrenamiento:',
      round(n_train / n_total * 100, 2), '%')

print('Porcentaje prueba:',
      round(n_test / n_total * 100, 2), '%')
```

Observaciones totales: 80
Observaciones de entrenamiento: 64
Observaciones de prueba: 16

Porcentaje entrenamiento: 80.0 %
Porcentaje prueba: 20.0 %

46. Dividir la serie de Vanish en entrenamiento y prueba

```
train_vanish = serie_modelo['VANISH'].iloc[:-n_test]
test_vanish = serie_modelo['VANISH'].iloc[-n_test:]

print('Training Vanish:')
print(train_vanish.index.min(), '→', train_vanish.index.max())
print('Observaciones:', len(train_vanish))

print('\nTesting Vanish:')
print(test_vanish.index.min(), '→', test_vanish.index.max())
print('Observaciones:', len(test_vanish))
```

Training Vanish:
2022-01-09 00:00:00 → 2023-03-26 00:00:00
Observaciones: 64

Testing Vanish:
2023-04-02 00:00:00 → 2023-07-16 00:00:00
Observaciones: 16

47. Dividir la serie de Lysol en entrenamiento y prueba

```
train_lysol = serie_modelo['LYSOL'].iloc[:-n_test]
test_lysol = serie_modelo['LYSOL'].iloc[-n_test:]

print('Training Lysol:')
print(train_lysol.index.min(), '→', train_lysol.index.max())
print('Observaciones:', len(train_lysol))

print('\nTesting Lysol:')
print(test_lysol.index.min(), '→', test_lysol.index.max())
print('Observaciones:', len(test_lysol))
```

Training Lysol:
2022-01-09 00:00:00 → 2023-03-26 00:00:00
Observaciones: 64

Testing Lysol:
2023-04-02 00:00:00 → 2023-07-16 00:00:00
Observaciones: 16

7.2 – VALIDACIÓN DEL MODELO

48. Visualizar la división temporal de Vanish

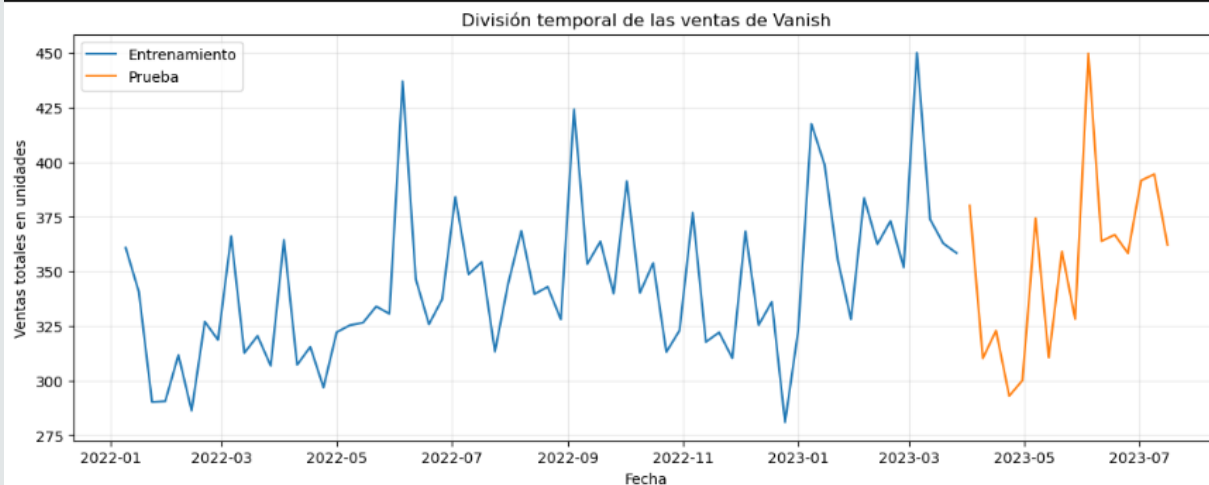
```
plt.figure(figsize=(14, 5))

plt.plot(
    train_vanish.index,
    train_vanish,
    label='Entrenamiento'
)

plt.plot(
    test_vanish.index,
    test_vanish,
    label='Prueba'
)

plt.title('División temporal de las ventas de Vanish')
plt.xlabel('Fecha')
plt.ylabel('Ventas totales en unidades')
plt.legend()
plt.grid(alpha=0.3)

plt.show()
```



49. Visualizar la división temporal de Lysol

```
plt.figure(figsize=(14, 5))

plt.plot(
    train_lysol.index,
    train_lysol,
    label='Entrenamiento'
)

plt.plot(
    test_lysol.index,
    test_lysol,
    label='Prueba'
)

plt.title('División temporal de las ventas de Lysol')
plt.xlabel('Fecha')
plt.ylabel('Ventas totales en unidades')
plt.legend()
plt.grid(alpha=0.3)

plt.show()
```



7.2 – VALIDACIÓN DEL MODELO

- Desempeño sobre datos no observados. Los modelos iniciales fueron evaluados utilizando:
 - MSE – Error Cuadrático Medio
 - MAE – Error Absoluto Medio
 - MAPE – Error Porcentual Absoluto Medio
- Resultados:

Marca	Modelo	MSE	MAE	MAPE
Vanish	ARIMA(3,1,0)	2370.32	39.73	11.96 %
Lysol	ARIMA(0,1,0)	0.0796	0.2366	10.24 %

7.2 – VALIDACIÓN DEL MODELO

```
# 72. Generar predicciones de Vanish para las 16 semanas del conjunto de prueba
```

```
pred_vanish = ajuste_vanish.forecast(  
    steps=len(test_vanish)  
)
```

```
pred_vanish.index = test_vanish.index
```

```
pred_vanish.head()
```

```
Date  
2023-04-02    411.566974  
2023-04-09    384.876935  
2023-04-16    376.574718  
2023-04-23    365.563143  
2023-04-30    392.903826  
Freq: 7D, Name: predicted_mean, dtype: float64
```

```
# 73. Generar predicciones de Lysol para las 16 semanas del conjunto de prueba
```

```
pred_lysol = ajuste_lysol.forecast(  
    steps=len(test_lysol)  
)
```

```
pred_lysol.index = test_lysol.index
```

```
pred_lysol.head()
```

```
Date  
2023-04-02     2.593  
2023-04-09     2.593  
2023-04-16     2.593  
2023-04-23     2.593  
2023-04-30     2.593  
Freq: 7D, Name: predicted_mean, dtype: float64
```

```
# 74. Importar las métricas necesarias para evaluar los modelos
```

```
from sklearn.metrics import mean_squared_error, mean_absolute_error
```

```
# 75. Crear una función para calcular las métricas de error
```

```
def evaluar_modelo(real, predicho, nombre):
```

```
    mse = mean_squared_error(real, predicho)  
    mae = mean_absolute_error(real, predicho)
```

```
    if (real == 0).any():  
        mape = np.nan  
    else:  
        mape = np.mean(  
            np.abs((real - predicho) / real)  
        ) * 100
```

```
    print(f'Evaluación del modelo - {nombre}')  
    print(f'-' * 40)  
    print(f'MSE: {mse:.4f}')  
    print(f'MAE: {mae:.4f}')
```

```
    if np.isnan(mape):  
        print('MAPE: No calculable porque existen valores reales iguales a 0.')  
    else:  
        print(f'MAPE: {mape:.2f}%')
```

```
# 76. Evaluar las predicciones del modelo de Vanish
```

```
evaluar_modelo(  
    test_vanish,  
    pred_vanish,  
    'Vanish - ARIMA(3,1,0)'  
)
```

```
Evaluación del modelo - Vanish - ARIMA(3,1,0)
```

```
-----  
MSE: 2370.3238  
MAE: 39.7330  
MAPE: 11.96%
```

7.2 – VALIDACIÓN DEL MODELO

```
# 77. Evaluar Las predicciones del modelo de Lysol
```

```
evaluar_modelo(  
    test_lysol,  
    pred_lysol,  
    'Lysol - ARIMA(0,1,0)'  
)
```

Evaluación del modelo - Lysol - ARIMA(0,1,0)

MSE: 0.0796

MAE: 0.2366

MAPE: 10.24%

```
# 78. Comparar Las ventas reales y Las predicciones de Vanish
```

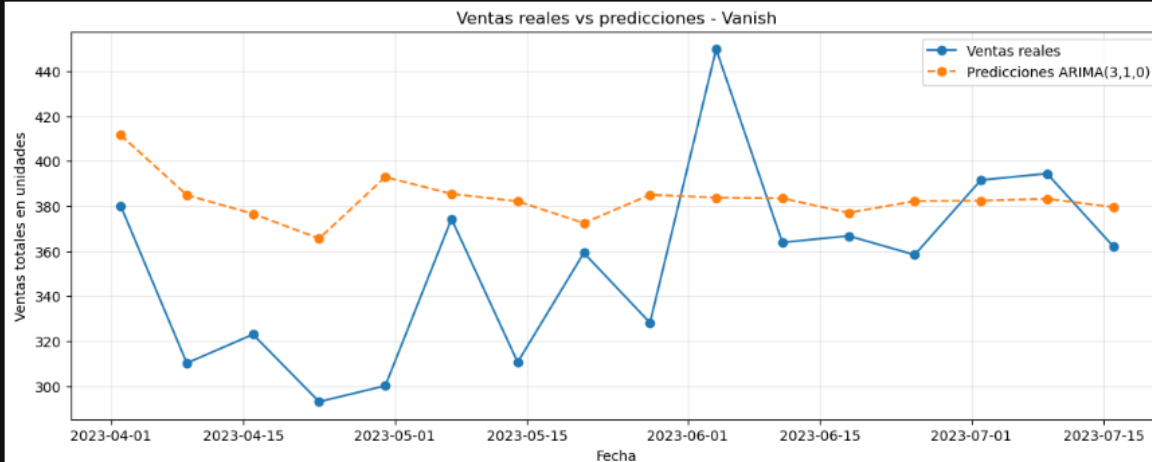
```
plt.figure(figsize=(14, 5))
```

```
plt.plot(  
    test_vanish.index,  
    test_vanish,  
    marker='o',  
    label='Ventas reales'  
)
```

```
plt.plot(  
    pred_vanish.index,  
    pred_vanish,  
    marker='o',  
    linestyle='--',  
    label='Predicciones ARIMA(3,1,0)'  
)
```

```
plt.title('Ventas reales vs predicciones - Vanish')  
plt.xlabel('Fecha')  
plt.ylabel('Ventas totales en unidades')  
plt.legend()  
plt.grid(alpha=0.3)
```

```
plt.show()
```



```
# 79. Comparar Las ventas reales y Las predicciones de Lysol
```

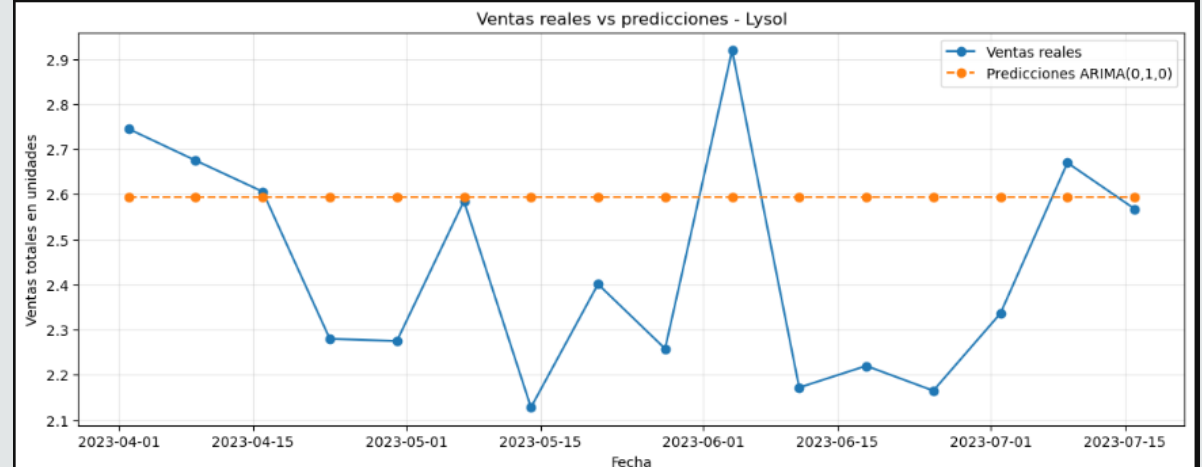
```
plt.figure(figsize=(14, 5))
```

```
plt.plot(  
    test_lysol.index,  
    test_lysol,  
    marker='o',  
    label='Ventas reales'  
)
```

```
plt.plot(  
    pred_lysol.index,  
    pred_lysol,  
    marker='o',  
    linestyle='--',  
    label='Predicciones ARIMA(0,1,0)'  
)
```

```
plt.title('Ventas reales vs predicciones - Lysol')  
plt.xlabel('Fecha')  
plt.ylabel('Ventas totales en unidades')  
plt.legend()  
plt.grid(alpha=0.3)
```

```
plt.show()
```



7.2 – VALIDACIÓN DEL MODELO

- Para evitar seleccionar hiperparámetros directamente utilizando el conjunto de prueba, el conjunto de entrenamiento original se dividió internamente en:
 - 52 semanas subentrenamiento
 - 12 semanas validación
- Se evaluaron diferentes configuraciones ARIMA utilizando directamente su capacidad para predecir las 12 semanas futuras de validación.
- La evaluación mostró
- Vanish: ARIMA(2,1,3) presentó una mejora relevante frente al ARIMA(3,1,0).
- Lysol: una configuración más compleja ARIMA(3,1,0) no presentó una mejora consistente frente al ARIMA(0,1,0).

7.2 – VALIDACIÓN DEL MODELO

80. Crear conjuntos internos de entrenamiento y validación para optimizar los modelos

```
n_validacion = 12

subtrain_vanish = train_vanish.iloc[:-n_validacion]
validacion_vanish = train_vanish.iloc[-n_validacion:]

subtrain_lysol = train_lysol.iloc[:-n_validacion]
validacion_lysol = train_lysol.iloc[-n_validacion:]

print('VANISH')
print('Subentrenamiento:', len(subtrain_vanish))
print('Validación:', len(validacion_vanish))

print('\nLYSOL')
print('Subentrenamiento:', len(subtrain_lysol))
print('Validación:', len(validacion_lysol))
```

```
VANISH
Subentrenamiento: 52
Validación: 12

LYSOL
Subentrenamiento: 52
Validación: 12
```

81. Verificar los periodos temporales del subentrenamiento y validación

```
print('VANISH')
print(
    'Subentrenamiento:',
    subtrain_vanish.index.min(),
    '-',
    subtrain_vanish.index.max()
)
print(
    'Validación:',
    validacion_vanish.index.min(),
    '-',
    validacion_vanish.index.max()
)

print('\nLYSOL')
print(
    'Subentrenamiento:',
    subtrain_lysol.index.min(),
    '-',
    subtrain_lysol.index.max()
)
print(
    'Validación:',
    validacion_lysol.index.min(),
    '-',
    validacion_lysol.index.max()
)
```

```
VANISH
Subentrenamiento: 2022-01-09 00:00:00 + 2023-01-01 00:00:00
Validación: 2023-01-08 00:00:00 + 2023-03-26 00:00:00
```

```
LYSOL
Subentrenamiento: 2022-01-09 00:00:00 + 2023-01-01 00:00:00
Validación: 2023-01-08 00:00:00 + 2023-03-26 00:00:00
```

82. Crear una función para optimizar ARIMA mediante validación temporal

```
def buscar_arima_validacion(
    subtrain,
    validacion,
    nombre,
    max_p=1,
    max_q=1
):
    resultados = []
    errores = []

    for p in range(max_p + 1):
        for q in range(max_q + 1):

            try:
                with warnings.catch_warnings():
                    warnings.simplefilter('ignore')

                    modelo = ARIMA(
                        subtrain,
                        order=(p, 1, q)
                    )

                    ajusta = modelo.fit()

                    predicción = ajusta.forecast(
                        steps=len(validacion)
                    )

                    predicción.index = validacion.index

                    mse = mean_squared_error(
                        validacion,
                        predicción
                    )

                    mae = mean_absolute_error(
                        validacion,
                        predicción
                    )

                    mape = np.mean(
                        np.abs(
                            (validacion - predicción)
                            / validacion
                        )
                    ) * 100

                    resultados.append([
                        'Marca': nombre,
                        'p': p,
                        'd': 1,
                        'q': q,
                        'MSE': mse,
                        'MAE': mae,
                        'MAPE': mape,
                        'AIC': ajusta.aic,
                        'BIC': ajusta.bic
                    ])

            except Exception as e:
                errores.append(
                    f'ARIMA({p},1,{q}): {e}'
                )

    resultados_df = pd.DataFrame(resultados)

    resultados_df = resultados_df.sort_values(
        by=['MAPE', 'MAE']
    ).reset_index(drop=True)

    return resultados_df, errores
```

7.2 – VALIDACIÓN DEL MODELO

83. Evaluar configuraciones ARIMA para Vanish mediante validación temporal

```
optimizacion_vanish, errores_vanish = buscar_arima_validacion(
    subtrain_vanish,
    validacion_vanish,
    'Vanish'
)
```

optimizacion_vanish.head(10)

	Marca	p	d	q	MSE	MAE	MAPE	AIC	BIC
0	Vanish	2	1	3	2652.445271	41.942191	10.610888	500.680955	512.271909
1	Vanish	1	1	3	3005.605637	44.828194	11.363499	501.059236	510.718364
2	Vanish	0	1	1	3336.226518	48.531309	12.327814	500.014476	503.878127
3	Vanish	2	1	2	3418.439032	48.973853	12.439885	504.025219	513.684347
4	Vanish	2	1	1	3440.897281	49.546137	12.597553	503.907831	511.635134
5	Vanish	1	1	1	3462.566854	49.828376	12.675211	501.919509	507.714986
6	Vanish	0	1	2	3467.122559	49.884040	12.690442	501.919920	507.715397
7	Vanish	0	1	3	3495.538239	50.210417	12.779222	503.914916	511.642218
8	Vanish	1	1	2	3679.676922	51.482300	13.101968	502.773910	510.501213
9	Vanish	0	1	0	3827.463367	53.352833	13.617615	525.424321	527.356147

84. Evaluar configuraciones ARIMA para Lysol mediante validación temporal

```
optimizacion_lysol, errores_lysol = buscar_arima_validacion(
    subtrain_lysol,
    validacion_lysol,
    'Lysol'
)
```

optimizacion_lysol.head(10)

	Marca	p	d	q	MSE	MAE	MAPE	AIC	BIC
0	Lysol	3	1	0	0.154389	0.328259	11.065120	141.391582	149.118885
1	Lysol	0	1	3	0.154040	0.328109	11.065518	141.445191	149.172494
2	Lysol	3	1	1	0.155811	0.329722	11.102188	143.324718	152.983846
3	Lysol	1	1	1	0.156321	0.332855	11.254838	139.506988	145.302465
4	Lysol	1	1	2	0.156499	0.333446	11.278331	141.504760	149.232062
5	Lysol	2	1	1	0.156586	0.333710	11.288770	141.503820	149.231123
6	Lysol	0	1	2	0.157658	0.335796	11.366262	139.518242	145.313719
7	Lysol	0	1	1	0.158803	0.337025	11.405594	137.555782	141.419434
8	Lysol	2	1	0	0.158459	0.337421	11.427484	139.534927	145.330403
9	Lysol	1	1	0	0.159266	0.337889	11.437900	137.572865	141.436517

7.2 – VALIDACIÓN DEL MODELO

```
# 85. Consultar el desempeño en validación de Los modelos originales
```

```
print('Modelo original Vanish - ARIMA(3,1,0)')
print(
    optimizacion_vanish[
        (optimizacion_vanish['p'] == 3) &
        (optimizacion_vanish['q'] == 0)
    ]
)
```

```
print('\nModelo original Lysol - ARIMA(0,1,0)')
print(
    optimizacion_lysol[
        (optimizacion_lysol['p'] == 0) &
        (optimizacion_lysol['q'] == 0)
    ]
)
```

```
Modelo original Vanish - ARIMA(3,1,0)
```

	Marca	p	d	q	MSE	MAE	MAPE	AIC	BIC
11	Vanish	3	1	0	4672.158367	60.699734	15.598808	495.713191	503.440494

```
Modelo original Lysol - ARIMA(0,1,0)
```

	Marca	p	d	q	MSE	MAE	MAPE	AIC	BIC
10	Lysol	0	1	0	0.164034	0.349417	11.895205	135.840945	137.77277

7.2 – VALIDACIÓN DEL MODELO

- Para Vanish, el modelo optimizado redujo aproximadamente:
 - 27.7 % el MSE
 - 19.8 % el MAE
 - 20.9 % el MAPE
- Para Lysol, el modelo más complejo deterioró ligeramente las tres métricas. Por ello se conservó el modelo ARIMA(0,1,0).

Marca	Modelo	MSE	MAE	MAPE
Vanish	ARIMA(3,1,0) original	2370.32	39.73	11.96 %
Vanish	ARIMA(2,1,3) final	1714.12	31.87	9.46 %
Lysol	ARIMA(0,1,0) final	0.0796	0.2366	10.24 %
Lysol	ARIMA(3,1,0) candidato	0.0851	0.2428	10.55 %

7.2 – VALIDACIÓN DEL MODELO

```
# 86. Entrenar Los modelos optimizados con todo el conjunto de entrenamiento
```

```
modelo_vanish_opt = ARIMA(  
    train_vanish,  
    order=(2, 1, 3)  
)
```

```
ajuste_vanish_opt = modelo_vanish_opt.fit()
```

```
modelo_lysol_opt = ARIMA(  
    train_lysol,  
    order=(3, 1, 0)  
)
```

```
ajuste_lysol_opt = modelo_lysol_opt.fit()
```

```
print('Modelo optimizado Vanish: ARIMA(2,1,3)')  
print('AIC:', ajuste_vanish_opt.aic)  
print('BIC:', ajuste_vanish_opt.bic)
```

```
print('\nModelo optimizado Lysol: ARIMA(3,1,0)')  
print('AIC:', ajuste_lysol_opt.aic)  
print('BIC:', ajuste_lysol_opt.bic)
```

```
Modelo optimizado Vanish: ARIMA(2,1,3)  
AIC: 622.3190084156519  
BIC: 635.177816774001
```

```
Modelo optimizado Lysol: ARIMA(3,1,0)  
AIC: 161.24467535024996  
BIC: 169.81721425581608
```

```
# 87. Generar predicciones finales con Los modelos optimizados
```

```
pred_vanish_opt = ajuste_vanish_opt.forecast(  
    steps=len(test_vanish)  
)
```

```
pred_vanish_opt.index = test_vanish.index
```

```
pred_lysol_opt = ajuste_lysol_opt.forecast(  
    steps=len(test_lysol)  
)
```

```
pred_lysol_opt.index = test_lysol.index
```

```
# 88. Evaluar Los modelos optimizados sobre el conjunto de prueba final
```

```
evaluar_modelo(  
    test_vanish,  
    pred_vanish_opt,  
    'Vanish optimizado - ARIMA(2,1,3)'  
)
```

```
print('\n')
```

```
evaluar_modelo(  
    test_lysol,  
    pred_lysol_opt,  
    'Lysol optimizado - ARIMA(3,1,0)'  
)
```

```
Evaluación del modelo - Vanish optimizado - ARIMA(2,1,3)
```

```
-----  
MSE: 1714.1246  
MAE: 31.8744  
MAPE: 9.46%
```

```
Evaluación del modelo - Lysol optimizado - ARIMA(3,1,0)
```

```
-----  
MSE: 0.0851  
MAE: 0.2428  
MAPE: 10.55%
```

7.2 – VALIDACIÓN DEL MODELO

89. Comparar Las métricas del modelo original y optimizado

```
def obtener_metricas(real, predicho):
```

```
    mse = mean_squared_error(real, predicho)
    mae = mean_absolute_error(real, predicho)
```

```
    mape = np.mean(
        np.abs((real - predicho) / real)
    ) * 100
```

```
    return mse, mae, mape
```

```
mse_v_orig, mae_v_orig, mape_v_orig = obtener_metricas(
    test_vanish,
    pred_vanish
)
```

```
mse_v_opt, mae_v_opt, mape_v_opt = obtener_metricas(
    test_vanish,
    pred_vanish_opt
)
```

```
mse_l_orig, mae_l_orig, mape_l_orig = obtener_metricas(
    test_lysol,
    pred_lysol
)
```

```
mse_l_opt, mae_l_opt, mape_l_opt = obtener_metricas(
    test_lysol,
    pred_lysol_opt
)
```

```
comparacion_modelos = pd.DataFrame({
    'Marca': [
        'Vanish', 'Vanish',
        'Lysol', 'Lysol'
    ],
    'Modelo': [
        'Original ARIMA(3,1,0)',
        'Optimizado ARIMA(2,1,3)',
        'Original ARIMA(0,1,0)',
        'Optimizado ARIMA(3,1,0)'
    ],
    'MSE': [
        mse_v_orig, mse_v_opt,
        mse_l_orig, mse_l_opt
    ],
    'MAE': [
        mae_v_orig, mae_v_opt,
        mae_l_orig, mae_l_opt
    ],
    'MAPE': [
        mape_v_orig, mape_v_opt,
        mape_l_orig, mape_l_opt
    ]
})
```

comparacion_modelos

	Marca	Modelo	MSE	MAE	MAPE
0	Vanish	Original ARIMA(3,1,0)	2370.323795	39.733006	11.955581
1	Vanish	Optimizado ARIMA(2,1,3)	1714.124553	31.874402	9.460387
2	Lysol	Original ARIMA(0,1,0)	0.079556	0.236562	10.236421
3	Lysol	Optimizado ARIMA(3,1,0)	0.085091	0.242776	10.545698

7.3 – PREDICCIONES FUTUROS

- Una vez seleccionados los modelos definitivos, estos fueron reentrenados utilizando las 80 semanas históricas disponibles. Modelos finales:
 - Vanish ARIMA(2,1,3)
 - Lysol ARIMA(0,1,0)}
- Se generaron predicciones para las siguientes 12 semanas, del 23 de julio al 8 de octubre de 2023.
- Vanish: El modelo pronostica aproximadamente entre:338 – 395 unidades semanales, con un promedio cercano a 365 unidades por semana.
- Lysol: El modelo proyecta aproximadamente 2.568 unidades semanales durante el horizonte analizado.

7.3 – PREDICCIONES FUTUROS

```
# 90. Reentrenar Los modelos finales utilizando Las 80 semanas históricas disponibles
```

```
modelo_final_vanish = ARIMA(
    serie_modelo['VANISH'],
    order=(2, 1, 3)
)
```

```
ajuste_final_vanish = modelo_final_vanish.fit()
```

```
modelo_final_lysol = ARIMA(
    serie_modelo['LYSOL'],
    order=(0, 1, 0)
)
```

```
ajuste_final_lysol = modelo_final_lysol.fit()
```

```
print('Modelo final Vanish: ARIMA(2,1,3)')
print('AIC:', ajuste_final_vanish.aic)
print('BIC:', ajuste_final_vanish.bic)
```

```
print('\nModelo final Lysol: ARIMA(0,1,0)')
print('AIC:', ajuste_final_lysol.aic)
print('BIC:', ajuste_final_lysol.bic)
```

```
Modelo final Vanish: ARIMA(2,1,3)
AIC: 786.402766697818
BIC: 800.6194538126201
```

```
Modelo final Lysol: ARIMA(0,1,0)
AIC: 180.073545058078
BIC: 182.44299291054503
```

```
# 91. Generar pronósticos para Las próximas 12 semanas
```

```
n_futuro = 12

forecast_vanish = ajuste_final_vanish.get_forecast(
    steps=n_futuro
)
```

```
forecast_lysol = ajuste_final_lysol.get_forecast(
    steps=n_futuro
)
```

```
pred_futuro_vanish = forecast_vanish.predicted_mean
pred_futuro_lysol = forecast_lysol.predicted_mean
```

```
# 92. Obtener intervalos de confianza del 95 % para Las predicciones
```

```
ic_vanish = forecast_vanish.conf_int(alpha=0.05)
ic_lysol = forecast_lysol.conf_int(alpha=0.05)

print('Periodo pronosticado:')
print(pred_futuro_vanish.index.min(), '-', pred_futuro_vanish.index.max())
```

```
Periodo pronosticado:
```

```
2023-07-23 00:00:00 → 2023-10-08 00:00:00
```

```
# 93. Crear una tabla comparativa con Las predicciones futuras
```

```
tabla_pronostico = pd.DataFrame({
    'Vanish_Prediccion': pred_futuro_vanish,
    'Vanish_Limite_Inferior': ic_vanish.iloc[:, 0],
    'Vanish_Limite_Superior': ic_vanish.iloc[:, 1],
    'Lysol_Prediccion': pred_futuro_lysol,
    'Lysol_Limite_Inferior': ic_lysol.iloc[:, 0],
    'Lysol_Limite_Superior': ic_lysol.iloc[:, 1]
})
```

```
tabla_pronostico.index.name = 'Fecha'
```

```
tabla_pronostico.round(3)
```

	Vanish_Prediccion	Vanish_Limite_Inferior	Vanish_Limite_Superior	Lysol_Prediccion	Lysol_Limite_Inferior	Lysol_Limite_Superior
Fecha						
2023-07-23	378.612	316.646	440.578	2.568	1.104	4.032
2023-07-30	339.167	274.451	403.883	2.568	0.498	4.638
2023-08-06	394.806	329.141	460.471	2.568	0.033	5.103
2023-08-13	337.937	272.228	403.645	2.568	-0.360	5.496
2023-08-20	385.972	316.399	455.545	2.568	-0.705	5.841
2023-08-27	351.901	282.071	421.732	2.568	-1.018	6.154
2023-09-03	370.997	298.303	443.691	2.568	-1.305	6.441
2023-09-10	365.003	292.309	437.697	2.568	-1.572	6.708
2023-09-17	361.351	287.553	435.149	2.568	-1.823	6.959
2023-09-24	370.736	296.584	444.888	2.568	-2.061	7.197
2023-10-01	359.167	284.672	433.661	2.568	-2.287	7.423
2023-10-08	370.213	294.926	445.500	2.568	-2.503	7.639

7.3 – PREDICCIONES FUTUROS

- El modelo de Vanish conserva variaciones semanales en el pronóstico, mientras que Lysol mantiene un comportamiento prácticamente constante debido a la estructura de su $ARIMA(0,1,0)$.
- En ambos modelos se calcularon intervalos de confianza del 95 %.
- La amplitud de estos intervalos aumenta conforme se amplía el horizonte, reflejando una mayor incertidumbre en las predicciones más lejanas. En Lysol, el límite inferior llega a valores negativos. Estos valores no representan ventas posibles, sino una limitación estadística del intervalo generado por el modelo. Los pronósticos deben interpretarse como una herramienta de apoyo para la planeación y no como valores futuros exactos.

7.3 – PREDICCIONES FUTUROS

94. Visualizar el pronóstico futuro de Vanish

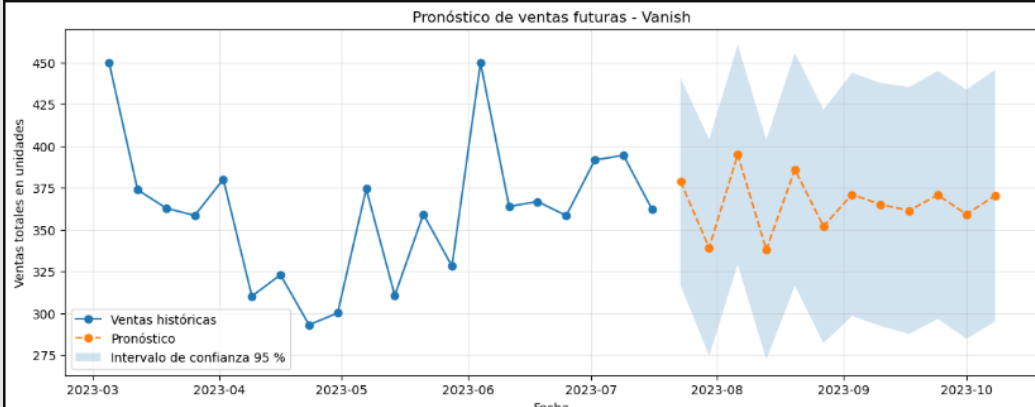
```
plt.figure(figsize=(14, 5))

plt.plot(
    serie_modelo['VANISH'].iloc[-28:].index,
    serie_modelo['VANISH'].iloc[-28:],
    marker='o',
    label='Ventas históricas'
)

plt.plot(
    pred_futuro_vanish.index,
    pred_futuro_vanish,
    marker='o',
    linestyle='--',
    label='Pronóstico'
)

plt.fill_between(
    pred_futuro_vanish.index,
    ic_vanish.iloc[:, 0],
    ic_vanish.iloc[:, 1],
    alpha=0.2,
    label='Intervalo de confianza 95 %'
)

plt.title('Pronóstico de ventas futuras - Vanish')
plt.xlabel('Fecha')
plt.ylabel('Ventas totales en unidades')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



95. Visualizar el pronóstico futuro de Lysol

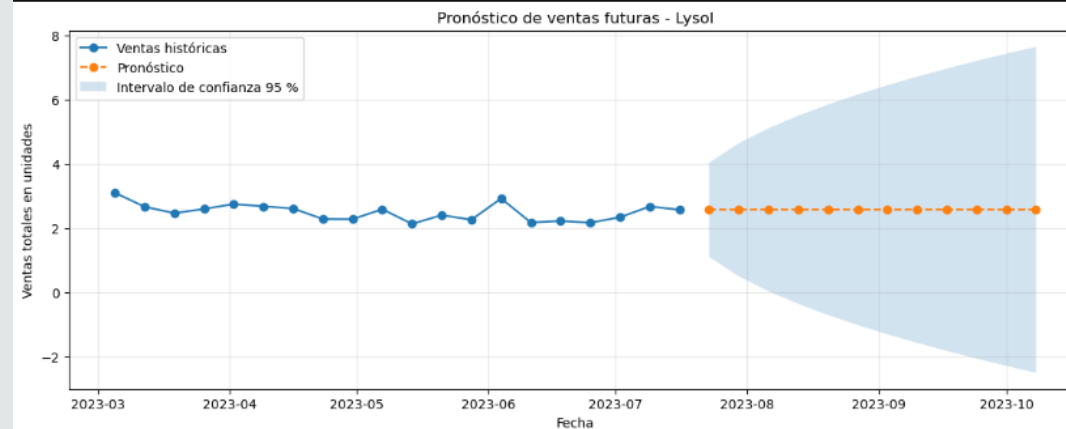
```
plt.figure(figsize=(14, 5))

plt.plot(
    serie_modelo['LYSOL'].iloc[-28:].index,
    serie_modelo['LYSOL'].iloc[-28:],
    marker='o',
    label='Ventas históricas'
)

plt.plot(
    pred_futuro_lysol.index,
    pred_futuro_lysol,
    marker='o',
    linestyle='--',
    label='Pronóstico'
)

plt.fill_between(
    pred_futuro_lysol.index,
    ic_lysol.iloc[:, 0],
    ic_lysol.iloc[:, 1],
    alpha=0.2,
    label='Intervalo de confianza 95 %'
)

plt.title('Pronóstico de ventas futuras - Lysol')
plt.xlabel('Fecha')
plt.ylabel('Ventas totales en unidades')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```



8 – CONCLUSIONES Y RECOMENDACIONES



8.1 – RESUMEN DE LOS HALLAZGOS CLAVE

- El proyecto permitió transformar distintas fuentes de información en una base analítica integrada y avanzar desde la exploración de datos hasta la generación de pronósticos.
- Se consolidaron 122,002 registros de ventas y 20 variables, integrando producto, categoría, segmento, calendario y región.
- El EDA mostró distribuciones fuertemente asimétricas, con numerosos valores extremos que fueron investigados y no eliminados automáticamente.
- TOTAL AUTOS SCANNING MEXICO concentra aproximadamente 50% del valor de ventas, destacando claramente frente a las demás regiones.
- El clustering identificó 350 productos y aisló un grupo de sólo 3 productos CLORALEX que concentra aproximadamente 26.67% del valor total vendido.
- Los modelos ARIMA permitieron generar pronósticos a 12 semanas para Vanish y Lysol, con MAPE de 9.46% y 10.24%, respectivamente.

8.2 – IMPACTO DEL ANÁLISIS EN LA ESTRATEGIA EMPRESARIAL

- Los resultados permiten pasar de una visión descriptiva de las ventas a una gestión basada en datos.

Resultado	Aplicación estratégica
Concentración regional	Priorizar análisis y acciones comerciales donde se genera mayor valor.
Segmentación de productos	Diferenciar estrategias entre productos de alto desempeño y portafolio general.
Tendencias temporales	Anticipar cambios en demanda y mejorar la planeación.
Pronósticos de ventas	Apoyar decisiones de inventario, abastecimiento y objetivos comerciales.
Identificación de outliers	Detectar operaciones excepcionales y evitar decisiones basadas únicamente en promedios.

8.3 - RECOMENDACIONES

- 1. Productos de alto desempeño: Analizar los factores detrás de los 3 productos CLORALEX identificados como excepcionales y evaluar cuáles pueden replicarse en otras líneas.
- 2. Gestión diferenciada del portafolio: Evitar aplicar la misma estrategia a los 350 productos; utilizar desempeño, segmento y comportamiento comercial para establecer acciones específicas.
- 3. Concentración regional: Profundizar el análisis de TOTAL AUTOS SCANNING MEXICO, debido a su participación cercana al 50% de las ventas en valor.
- 4. Planeación predictiva: Actualizar periódicamente los modelos de Vanish y Lysol e incorporar sus pronósticos a procesos de inventario, abastecimiento y planeación comercial.
- 5. Valores extremos: Mantener los outliers bajo análisis individual antes de excluirlos, ya que pueden representar ventas reales de alto impacto.

8.4 – FUTURAS MEJORAS

- El proyecto puede evolucionar incorporando mayor profundidad de datos y modelos:
 - Ampliar el histórico de ventas para mejorar el análisis temporal.
 - Incorporar variables como precio, promociones, inventario, descuentos y disponibilidad.
 - Implementar validación temporal continua y monitoreo del error de los pronósticos.
 - Evaluar periódicamente la estabilidad de los clusters ante nuevos productos y nuevos periodos.
 - Comparar ARIMA y K-Means con otros algoritmos de Machine Learning y Series de Tiempo.
 - Automatizar la actualización de datos, análisis, visualizaciones y pronósticos.
- El proyecto demuestra cómo un proceso integral de Ciencia de Datos puede transformar registros de ventas en conocimiento accionable, desde la comprensión del comportamiento histórico hasta la segmentación y predicción del desempeño futuro.